

AWS DEVELOPER ASSOCIATE

IAM User url - <https://aws-shamanth-92.signin.aws.amazon.com/console>

IAM username - iamadmin

Access key - AKIAW3MEBATESMAR6EEM /

BgwuzmbHIIHKkcfHG97wZxaJa8GdmF6iQT+jtZUG

AWS Accounts

- Container for identities(users) and resources.
- Every account has a account root user. Has full access to everything within the account. Cannot be restricted.
- Other identities can be created using IAM and have full or limited rights. They start with no permissions. Access has to be granted.
- Always set up MFA for the account.
- AWS Budgets
- IAM - Identity and Access Management
- Least Privilege Access
- IAM is a globally resilient service.
- IAM lets you create - user, group and roles.
 - User - Identities that need access to your account.
 - Group - Collection of related users
 - Roles - Granting external access to your account.
- IAM Policy - to allow or deny access to AWS services when attached to IAM identities.
- IAM Access Keys
 - An IAM user can have 2 access keys. Not more than that.
 - Formed from 2 parts - Access Key ID and Secret Access Key
 - Rotating Access Keys means creating a brand new set and deleting the old one.

AWS Fundamentals

- 3 different network zones - public internet, aws public and aws private.
- Regions and Edge Locations.
- AWS Regions
 - Geographic Separation - Isolated Fault Domain
 - Geopolitical Separation - Different Governance
 - Location Control - Performance
- Availability Zones

- Globally Resilient, Region Resilient and AZ Resilient Services.
- Virtual Private Cloud (VPC)
A virtual network inside AWS.
Regional services - within 1 account and 1 region.
Two types - Default and Custom.
- EC2
IAAS - Infrastructure as a Service.
AZ Resilient Service.
On Demand Billing.
- S3
Regionally resilient
Hosting data
Default storage service in AWS
Objects and Buckets
Bucket names are globally unique
Number of Buckets - 100 soft limit, 1000 hard limit
Unlimited objects, size 0 bytes to 5TB
Its an object store system.
- AWS CloudFormation - IAC (Infrastructure as Code)
- AWS CloudWatch
Collects and manages operational data
Metrics, Logs and Events.
Namespace, Datapoints and Dimensions.
- Shared Responsibility Model
- High Availability, Fault Tolerance and Disaster Recovery.
- Route 53 - Globally Resilient.

IAM, Accounts and AWS Organizations

- IAM Policy - To grant and deny permissions to AWS services.
Priority Rules - Explicit DENY - Explicit ALLOW - Default DENY.
Inline Policies and Managed Policies (This type if preferred).
- IAM Users
ARN (Amazon Resource Name) - Uniquely identify resources within any AWS accounts.
5000 IAM Users per account.
- Security Token Service (STS)
Generates temporary credentials for IAM Roles.
Temporary credentials include access key id, expiration, secret access key and session token.
sts:AssumeRole calls are made.

- AWS Organizations
- Service Control Policies

CloudTrail

- Logs API actions which affect AWS accounts.
- Logs API calls/activities as a CloudTrail Event.
- Enabled by default.
- 90 days data stored by default in Event History.
- No cost for 90 day history.
- Management Events (Control Plane) and Data Events.
- Regional Service.
- Management events are enabled by default.
- Logs can be stored in S3 buckets.
- Global services such as IAM, STS and CloudFront log data as global service events to us-east-1.
- Regional services log data to their specific regions.
- Logs are not real time. There is a delay.

S3 (Simple Storage Service)

S3 Security:

- Private by default - Only accessible by the root user.
- S3 bucket policy. (Resource Policy).
- Access Control Lists (ACLs) - Legacy, not used anymore.
- Block Public Access.

S3 Static Website Hosting:

- Allows access via HTTP.
- Index and Error Documents. (HTML)
- Website Endpoint is created.
- Offloading and Out of Band pages.
- Transfer data into S3 is free and out of S3 has cost.

S3 Object Versioning and MFA Delete:

- Once enabled, cannot be disabled. Disabled by default. Can be suspended and then enabled again.
- Delete Marker.

- Version Delete.
- Billed for all versions.
- MFA Delete can be enabled in versioning configuration.

S3 Performance Optimization:

- Usually S3 uploads single stream of data which is slow and unreliable.
- Multipart Upload is the solution. Data is broken up. Min data size is 100mb for multipart.
- 10,000 max parts, 5mb to 5gb.
- Parts can fail and be restarted.
- S3 Transfer Acceleration.

Key Management Service (KMS):

- Regional and Public Service.
- Create, Store and Manage Cryptographic Keys.
- Symmetric and Asymmetric Keys.
- FIPS 140-2 (L2) compliance.
- Data Encryption Keys (DEKs).
- KMS Keys are isolated to a region.
- AWS Managed and Customer Managed keys.
- Supports rotation.

S3 Server Side Encryption (SSE):

- Client Side vs Server Side. (Encryption at Rest)
- Client Side - Encrypted by the user.
- Server Side - Encrypted by S3 server.
- 3 types of SSE - SSE-C, SSE-S3 and SSE-KMS.
- SSE-S3 is the default.

S3 Object Storage Classes:

- Default Storage Class - S3 Standard. Objects are replicated across at least 3 AZs in the AWS region. Used for frequently accessed data which is important and non replaceable.
- S3 Standard Infrequent Access (IA)
Has retrieval fee.
Minimum duration charge.
Minimum capacity charge.
Used for long lived data, which is important but where access is infrequent.

- S3 One Zone IA
Data only stored in one AZ. No replication.
Used for long lived data which is non critical and replaceable and where access is infrequent.
- S3 Glacier - Instant
Used for long lived data, accessed once per quarter with millisecond access.
- S3 Glacier - Flexible
Objects cannot be made publicly accessible. Any access of data requires a retrieval process.
Expedited, Standard, Bulk and Faster.
First byte latency = minutes or hours.
Used for archival data where frequent or real time access isn't needed.
- S3 Glacier Deep Archive (Cheapest)
Retrieval is longer - hours or days.
Used for archival data that is very rarely accessed.
- S3 Intelligent Tiering
Used for long lived data with changing or unknown patterns.

S3 Replication:

- Replication of objects between source and destination S3 bucket.
- 2 types - Cross Region (CRR), Same Region (SRR).
- By default replication is not retroactive, Versioning needs to be ON.
- By default - One way replication - source to destination.
- By default unencrypted.
- No system events, Glacier or Glacier Deep Archive.
- By default Deletes not replicated.

S3 Presigned URLs:

- A way to give another person or application access to an object inside an S3 bucket using your credentials in a safe and secure way.
- S3 Select and Glacier Select - Use SQL like statements to select part of the object pre filtered by S3.
- S3 Event Notifications.
- S3 Access Logs.
- S3 Requestor Pays.
- S3 Object Lock.

CloudHSM

HSM - Hardware Security Module.

AWS provisioned, fully customer managed.

Fully FIPS 140-2 Level 3.

Virtual Private Cloud

- Regional Service - All AZs in the region.
- Isolated Network inside AWS.
- Nothing IN or OUT without explicit configuration.
- Allows Hybrid Networking.
- Default or Dedicated Tenancy.
- IPv4 Private CIDR blocks and Public IPs.
- Have fully featured DNS provided by R53.

Subnets:

- Private by default. Need configurations to make them public.
- AZ resilient.
- A subnetwork of a VPC - within a particular AZ.
- Cannot be in multiple AZs. 1 Subnet => 1 AZ.
- Cannot overlap with other subnets.
- VPC Router - Network+1.
- Reserved DNS - Network+2.

VPC Routing and Internet Gateway:

- Present in every VPC which routes traffic.
- Runs in all AZs of the VPC.
- Controlled by route tables.
- Higher Prefix is used while routing.
- Local target means within the VPC. Local routes are un-editable.

Internet Gateway: Regionally resilient gateway attached to a VPC. 1 to 1 relationship between VPC and Internet Gateway. Gateways traffic between the VPC and the Internet or AWS Public Zone.

Network Access Control Lists (NACL):

- Filters traffic crossing the subnet boundary.
- Contains 2 sets of rules - Inbound and Outbound.
- NACLs are stateless. (Individual rules required for request and response).
- Allows explicit allows and denies.

- Only affects data crossing the subnet boundary.
- Custom NACLs.
- Each subnet can have one NACL (Default or Custom).
- A NACL can be associated with many subnets.

Security Groups:

- Stateful. If request is allowed then response is automatically allowed.
- No explicit DENY. Only Allow or Implicit Deny.
- Typically used in conjunction with NACLs.

Network Address Translation and NAT Gateways:

- Process of giving a private resource outgoing only access to the internet. (NAT)
- NAT Gateway is the AWS implementation that's available within a VPC.
- IP masquerading - hiding CIDR blocks behind one IP.
- NAT Gateways run from a public subnet.
- Uses Elastic IPs.
- NAT Gateways - AZ resilient.
- Managed AWS Service.
- NAT Gateways cannot be associated with a Security Group.

EC2

Virtualization:

- Process of running more than one OS on a piece of physical hardware or server.
- SR-IOV.
- AWS Nitro.
- Hypervisor.

EC2 Architecture:

- EC2 Instances are virtual machines.
- Run on EC2 hosts - Shared or Dedicated.
- AZ resilient service.
- EC2 uses - Traditional OS+Application Compute, Long Running Compute, Server style applications, monolithic application stacks, migrating application workloads and disaster recovery.

Instance Types:

Divided into 5 categories:

- General Purpose - Default.
- Compute Optimized.
- Memory Optimized.
- Accelerated Computing.
- Storage Optimized.

<https://aws.amazon.com/ec2/instance-types/>

Storage:

Types:

- Direct
- Network
- Ephemeral
- Persistent

Storage Categories:

- Block
- File
- Object

Elastic Block Store (EBS):

- Block Storage - raw disk allocations (volume), can be encrypted using KMS.
- AZ resilient.
- Attached to one EC2 instance.
- Billed based on GB-month.
- Volume Snapshots can be created in S3.
- GP2 - Default General Purpose SSD Storage. **(Now GP3)**
- IOPS
- GP2 - good for boot volumes, low latency interactive apps, dev and test environments.
- GP3 - SSD based.
- Base price is 20% cheaper than GP2.
- Use cases for GP3 - virtual desktops, medium sized single instance DBs, boot volumes, low latency interactive apps, dev and test environments.
- Provisioned IOPS SSD (io1 and io2).
- Used for high performance applications.
- Hard Disk Drive (HDD) storage.

- st1 and sc1.
- st1 - Big data, data warehouses and log processing.
- sc1 - Less frequently accessed workloads.

Instance Store Volumes:

- Block Storage Devices.
- Physically connected to one EC2 host. Local to an EC2 host.
- Since they are locally attached, they provide the highest storage performance in AWS.
- Attached at Launch time.
- Lost on instance move, resize or hardware failure.
- Instance store volumes are temporary.

Imp - Lesson "Choosing between Instance Store and EBS". Refer for remembering some statistics for comparing the two storage types.

EBS Snapshots:

- Efficient way to backup EBS volumes to S3.
- Incremental.
- Regional resilient.
- Snaps restore lazily - fetched gradually.
- Fast Snapshot Restore - for immediate restore. Costs extra.

EBS Encryption:

- Be default, no encryption.
- Can be encrypted using KMS.
- Each volume uses 1 unique Data Encryption key.
- Snapshots and future volumes use the same DEK.

Amazon Machine Images (AMI):

- Images of the EC2.
- Used to launch EC3 instance.
- AWS or community provided.
- AMIs are regional.
- AMI Lifecycle - Launch, Configure, Create Image and Launch.
- Block Device Mapping.
- AMI Baking.
- AMI cannot be edited.

- AMIs can be copied between regions.
- Permissions - Private, public or specific accounts.

Purchase Options:

- On Demand - default. Per second billing while instance is running. Defined upfront. No discounts.
- Spot - selling unused EC2 host capacity for up to 90% discount. Spot instances are not reliable. Never use them for workloads which cant tolerate interruptions.
- Reserved - for long term consistent usage of EC2. Reservations are for 1 or 3 year terms and paid for the entire term.
- Dedicated Hosts - EC2 host allocated to you. Pay for the host.
- Dedicated Instances.

Reserved Instance types - Standard, Scheduled, Capacity Reservations, Savings Plan.

Instance Metadata:

- EC2 service provides data to instances.
- IP address to access Instance metadata is <http://169.254.169.254/latest/meta-data>
- All information about the environment the instance is in.
- Networking information.
- Authentication.
- User data.
- Not authenticated or encrypted.

CloudWatch

- Ingestion, Storage and Management of metrics.
- Public Service.
- AWS Service integration.
- Agent Integration.
- On Premise and Application Integration.
- CloudWatch alarms.
- Namespace - container for metrics.
- Datapoint - Timestamp, value, unit of measure.
- Metric - Time ordered set of data points.
- Dimensions - name value pair.

CloudWatch Logs:

- Ingestion of logs.
- CW agent needs to be installed to ingest logging from system or custom applications.
- Regional service.
- Logging sources.
- Log events.
- Log stream.
- Log groups.
- Subscriptions.

X-Ray

- Distributed tracing application.
- Track sessions through an application.
- Tracing Header - unique trace ID used to track a request through the distributed application.
- Segments - Data blocks.
- Subsegments.
- Service Graph - JSON document detailing services and resources which make up the application.
- Service Map - Visual version of the service graph showing traces.

Elastic Container Service (ECS)

- Managed container based compute service.
- Runs in 2 modes - ec2 and fargate.
- Containers run inside an ECS cluster.
- ECR - Elastic Container Registry.
- Container Definition, Task Definition, Service Definition.
- Task Roles - best practice way of giving containers within ECS permissions to access AWS products and services.

EC2 Bootstrapping

- Allow EC2 Build Automation.
- Enabled using EC2 User data accessed via the metadata IP.
- <http://169.254.169.254/latest/user-data>
- Only on Launch.
- Anything in User Data is executed by the instance OS.

SSM Parameter Store

- Storage for configuration and secrets.

- Stores 3 different types of parameters - String, StringList and SecureStrings.
- Store data like License codes, Database strings, Full configs etc.
- aws ssm get-parameters --names 'name of the stored value'.
- aws ssm get-parameters-by-path --path 'path of the stored value'.

EC2 Placement Groups

- 3 types - Cluster, Spread, Partition.
- Cluster - To achieve absolute highest level of performance from EC2. Should be launched in a single AZ. 10Gbps single stream performance. Use Cases: Performance, fast speeds, low latency.
- Spread - spans multiple AZs. Provides infrastructure isolation. 7 instances per AZ. Use case: Small number of critical instances that need to be kept separated from each other.
- Partition - 7 partitions per AZ. Great for topology aware applications such as Cassandra.

CloudFormation

- Template written in either YAML or JSON.
- Contain logical resources.
- Templates are used to create Stacks.
- Stacks create physical resources based on the logical resources from the template.
- Stack keeps logical and physical resources in sync.
- **Template Parameters** - accept input. Can be referenced from within Logical resources.
- **Pseudo Parameters** - injected by AWS into the template.
- **Intrinsic Functions:**
 - Ref and Fn::GetAtt - Ref can be used to reference the main value of a parameter or a logical resource. GetAtt means get attribute.
 - Fn::GetAZs and Fn::Select - GetAZ is to get a list of all AZs in the region.
 - Fn::Join and Fn::Split.
 - Fn::Base64 and Fn::Sub.
 - Fn::Cidr.
- **CF Mappings** - Templates can contain a Mappings object. Use the !FindInMap intrinsic function.
- **CF Outputs** - optional outputs section. Values declared in this section are visible as outputs when using the CLI, visible in the console UI, accessible from a parent stack when using nesting, can be exported allowing cross stack references.
- Use public SSM parameters to refer to certain parameters like AMI id for an EC2 instance.
<https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-public-parameters.html>

- **CF Conditions** - created in the optional Conditions section. If condition evaluates to false no physical resource is created for the corresponding logical resource.
- **CF DependsOn** - establish dependencies between resources.
- **CF Signal** - cfn-signal command (included in aws-cfn-bootstrap package).
- **CF Nested Stacks**
 - Resources in a single stack share a lifecycle.
 - Nested Stacks - Contains a root & parent stack which is created manually.
 - Benefits - Overcome the 500 resource limit of one stack, Modular templates, code reuse, Installation process easier, use only when everything is lifecycle linked.
- **CF Cross Stack References**
 - CFN Stacks are designed to be isolated and self-contained. Cross Stack References are used to overcome this.
 - Export is used to export the output of a stack to a unique name in that region of that account.
 - ImportValue can be used instead of Ref to reference region exported values.
 - Uses - Service oriented architectures, different lifecycles, stack reuse (reuse the resources not template).
- **CF StackSets**
 - to deploy and manage infrastructure across many accounts and regions.
 - StackSets contain stack instances which reference stacks.
 - StackSets are applied in an admin account.
 - Concurrent Accounts.
 - Retain Stacks.
- **CF DeletionPolicy** - can be defined on each resource in the template. Default policy is delete. Can also be Retain or Snapshot.
- **CF Init**
 - Configuration management system.
 - Configuration directives are stored in template.
 - cfn-init helper script to access CF init data.
- **cfn-hup**
 - For EC2 bootstrapping.
 - Detects changes in the resources metadata.
 - Helps in re running cfn-init.
- **CF Change sets**
 - When stack is updated 3 things might occur - No Interruption, some interruption, replacement.
 - Change Sets let you preview changes.

- Different versions can be created.
- **CF Custom Resources**
 - Type of logical resource which lets CF integrate with anything it doesn't yet or doesn't natively support.

****R53**

- Two types of DNS zones in R53 - public and private.
- R53 Hosted Zone is a DNS db for a domain.
- R53 - Globally resilient.
- Hosted Zones host DNS records.

CloudFront

- CDN.
- Edge Location - Local cache of data.
- Regional Edge Cache - Larger version of an edge location.
- **Behaviors**
 - Distributions are the unit of configuration in CF.
 - Default Behavior has wild card or star path pattern.
 - For any request incoming to an edge location, they are pattern matched against any behaviors for that distribution using path pattern.
- Default TTL is 24 hours.
- Cache Invalidations are performed on a distribution and applies to all edge locations.
- SSL supported by default.
- For CF, certificates have to be always added in us east 1 (Northern Virginia) region.
- Self signed certificates don't work, have to be publicly trusted certificates.
- Certificates have to match the DNS name.
- Origins - S3 bucket, AWS Media Package channel Endpoints, AWS Media Store Container Endpoints and web servers (custom origins).
- **AWS Certificate Manager (ACM)**
 - Lets you run a public or private certificate authority (CA).
 - Generate or import certificates.
 - Automatically renews generated certificates.
 - Supported AWS Services only. (Cannot be used with EC2).
 - Regional service.
- **CF Geo Restriction**
 - Restricting content to a particular location.
 - Applicable only for Countries.

- Applies to entire distribution.

Databases

- **Wide Column Store**
 - DynamoDB is an example.
 - Minimum one key known as partition key.
- Running databases directly on EC2 is a bad idea.
- **RDS**
 - Managed Database Server as a Service.
 - Multiple Databases on one DB Server/instance.
 - Every instance has its own dedicated storage provided by EBS.
 - Backups are stored in S3.
 - Billed for resource allocation.
 - Multi AZ (Instance) - Synchronous replication between primary and standby, not free tier, one standby replica and it cannot be used for reads and writes, 60-120 secs failover.
 - Multi AZ (Cluster) - 1 writer and 2 reader db instances in different AZs, failover is faster ~35s.
- **Amazon Aurora**
 - A single primary instance and 0 or more replicas.
 - Replicas can be used for read operations during normal operation.
 - Uses cluster volume. Shared storage.
 - Can have up to 15 replicas.
 - Storage based on SSD - high IOPS, low latency.
 - Storage is billed on what's used.
 - No free tier option.
 - Allows Backtrack.
- **Aurora Serverless**
 - No need to provision resources.
 - Uses ACU - Aurora Capacity Units.
 - Consumption billing per second basis, hence cost effective compared to Aurora.

Elastic File System (EFS)

- Implementation of NFSv4.
- EFS Filesystems can be mounted in Linux.
- Shared between many EC2 instances.
- Private service but can be accessed from on-premises.

- Only supported for Linux.
- 2 performance modes - General Purpose(default) and Max I/O.
- 2 throughput modes - Bursting and Provisioned.
- 2 storage classes - Standard(default) and Infrequent Access.

FSx for Windows File Server

- Fully managed native Windows file servers/shares.
- Integrates with Directory Service or Self Managed Active Directory.
- Single AZ or Multi AZ within a VPC.
- On demand and Scheduled backups.
- Accessible using VPC, Peering, VPN and Direct Connect.

FSx for Lustre

- Managed Lustre file system.
- Designed for HPC - Linux clients, (POSIX).
- Use Cases - Machine Learning, Big Data, Financial Modelling.
- 100 Gb/s throughput and sub millisecond latency.
- Deployment types - Persistent and Scratch.
- Accessible over VPN or Direct Connect.

Elastic Load Balancer

- 3 types - Classic (v1)(Not Good), Application (v2), Network (v2).
- Cross Zone Load Balancing.
- **Application Load Balancer**
 - Layer 7 LB.
 - Listens on HTTP and/or HTTPS.
 - Must have SSL certs if HTTPS is used.
 - Slower than NLB.
 - Rules direct connections and are processed in priority order.
- **Network Load Balancers**
 - Layer 4.
 - TCP, TLS and UDP.
 - No headers, No cookies.
 - Very fast.
 - No detailed health checking.

AWS CI/CD

- BuildSpec file - collection of build commands and related settings in YAML format that CodeBuild uses to run a build.
- AppSpec file - YAML or JSON file used by CodeDeploy to manage a deployment.
- CodePipeline is a continuous delivery (CD) tool.
- **AWS CodeBuild**
 - Fully managed.
 - Pay only for the resources consumed during builds.
 - Used for builds and tests.
 - Customized via buildspec.yml (in root of the source).
- **AWS CodeDeploy**
 - Appspec.yml (YAML or JSON) manage deployments.
 - Agent needs to be installed if deploying to EC2 or on premises.
 - Lifecycle hooks - ApplicationStop, DownloadBundle, BeforeInstall, Install, AfterInstall, ApplicationStart, ValidateService.

Event Driven Architecture

AWS Lambda:

- Function as a Service (FaaS) product.
- Lambda function - a piece of code lambda runs.
- Billed for the duration that a function runs.
- Serverless.
- Lambda functions can run up to 900s/15min (function timeout).
- Common uses - Serverless Applications(S3, API gateway and Lambda), File Processing, Database Triggers, Serverless CRON, Realtime Stream Data Processing.
- 2 networking modes - public (default) and VPC.
- Lambda execution roles are IAM roles attached to lambda functions which control permissions the lambda function receives.
- Lambda resource policy controls what services and accounts can invoke lambda functions.
- Logging/tracing stored in Cloudwatch and X-ray.
- 3 ways to invoke lambda functions - Synchronous, Asynchronous and Event Source mappings.
- Different versions of lambda functions are possible.
- Lambda functions can be published and this creates an immutable version.
- Qualified ARN points at a specific version.
- An Alias is a pointer to a function version.
- Lambda Layers.
- Lambda Runtime API.

- Lambda Resource Policy - who can do what with the lambda function.

SNS

- Pub Sub messaging service.
- Public service.
- Coordinates the sending and delivery of messages.
- Messages are ≤ 256 kb payloads.
- SNS topic.
- Used for notifications.

SQS

- Public, Fully Managed, highly available queues - Standard or FIFO.
- Messages up to 256KB in size.
- VisibilityTimeout - for error handling and automatic reprocessing.
- Fanout architecture.
- Standard - at least once, FIFO - exactly once.
- Standard vs FIFO
 - FIFO - 300 messages per second without batching and 3000 with.
 - Standard - Infinite number of transactions per second.
 - FIFO - guarantees order, exactly once processing.
 - Standard - faster, no message ordering, messages can be delivered more than once.
- SQS Extended Client Library - Used when handling messages over 256KB. Allows large payloads (stored in S3).
- SQS Delay Queues - Postpone the delivery of messages to consumers.
- SQS Dead Letter Queues.

Step Functions

- State Machines
 - Serverless Workflow.
 - Maximum Duration 1 year.
 - Standard workflow (1 year) and Express workflow (in minutes).
 - Started via API Gateway, lambda etc.
 - Amazon States Language.

API Gateway

- Create and manage APIs.
- Public Service.

- Connect using API endpoint DNS name.
- Endpoint types - Edge Optimized, Regional, Private.
- Cache TTL default is 300 seconds. Configurable min 0 to max 3600s. Cache can be encrypted. Cache size is 500mb to 237gb.
- Proxying - Information is sent to integrations and received back without any translation.
- Integrations - Mock, HTTP, HTTP proxy, AWS, AWS_PROXY.
- The current API state needs to be deployed to a stage.

Kinesis

- Scalable streaming service.
- Producers send data to Kinesis stream.
- Public service.
- Shard Architecture.
- Kinesis Data Record.
- **Kinesis Data Firehose**
 - Fully managed service to load data for data lakes, data stores and analytics services.
 - Near real time delivery.
 - Pay as you go service.
 - Deliver data to - HTTP endpoints, Splunk, Redshift, ElasticSearch and S3.
- **Kinesis Data Analytics**
 - Real time processing of streaming data using SQL.

Amazon Cognito

- Authentication, Authorization and user management for web/mobile apps.
- User Pools - Sign in and get a JWT.
- Identity Pools - Allows you to offer access to Temporary AWS credentials.
- Web Identity Federation.

DynamoDB

- NoSQL Public database service.
- Highly resilient.
- Really fast.
- A table is a grouping of items with the same primary key. Item max size 400kb.
- Two types of backups - On Demand, Point in Time Recovery.
- On demand billing.
- Query and Scan.
- WCU (Write Capacity Unit) Calculation per item - (Item Size/1KB).

- RCU (Read Capacity Unit) per item = (Item Size/4KB).
- Indexes - Local Secondary Indexes and Global Secondary Indexes.
- Stream - Time ordered list of item changes in a table. 4 ways to see changes - KEYS_ONLY, NEW_IMAGE, OLD_IMAGE and NEW_AND_OLD_IMAGES.
- Triggers - Item changes generate an event. Action can be taken based on data change using Streams + Lambda.
- **DynamoDB Accelerator (DAX)**
 - Operates within a VPC.
 - Cluster service.
 - Maintains two caches - Item cache and Query cache.
- **ElastiCache**
 - In memory database for high end performance requirements.
 - Redis and Memcached.
 - Reduces database workloads.
 - Can be used to store session data.
 - Allows cost effective scaling of read heavy workloads.
- Amazon Athena - Serverless interactive querying service.

Elastic Beanstalk

- Platform as a Service.
- Provides managed environments for applications.
- Focus on code, low infrastructure overhead.
- Fully customizable.
- Not free of cost.
- Platforms - Built in languages, docker and custom(Packer).
- Great for small development teams.
- Databases should be created outside beanstalk.
- Environments - Web server and Worker.
- Deployment Policies - All at once, Rolling, Rolling with additional batch, Immutable and Traffic Splitting.
- Ebextensions are a way to customize EB environments.

Udemy Solutions Architect Associate - https://media.datacumulus.com/aws-saa/AWS%20Certified%20Solutions%20Architect%20Slides%20v44.pdf?_gl=1*1iythk0*_ga*MTYxNTUwMDEyMy4xNzQ0NzQ0NDQ0*_ga_6GZZTGX7H*MTc0NDc0NDQ0NC4xLjAuMTc0NDc0NDQ0NC42MC4wLjA.

Udemy Developer Associate - https://media.datacumulus.com/aws-dva/AWS%20Certified%20Developer%20Slides%20v40.pdf?_gl=1*1fjxeag*_ga*MTYxNTUwMDEyMy4xNzQ0NzQ0NDQ0*_ga_6GZZTGGX7H*MTc0NDc0NDQ0NC4xLjEuMTc0NDc0NDY3MC42MC4wLjA.

Notes for SAA-C03:

S3:

Storage classes for frequently accessed objects

For performance-sensitive use cases (those that require millisecond access time) and frequently accessed data, Amazon S3 provides the following storage classes:

- **S3 Standard** (`STANDARD`) – The default storage class. If you don't specify the storage class when you upload an object, Amazon S3 assigns the S3 Standard storage class.
- **S3 Express One Zone** (`EXPRESS_ONEZONE`) – Amazon S3 Express One Zone is a high-performance, single-zone Amazon S3 storage class that is purpose-built to deliver consistent, single-digit millisecond data access for your most latency-sensitive applications. S3 Express One Zone is the lowest latency cloud object storage class available today, with data access speed up to 10x faster and with request costs 50 percent lower than S3 Standard. With S3 Express One Zone, your data is redundantly stored on multiple devices within a single Availability Zone.

Storage class for automatically optimizing data with changing or unknown access patterns

S3 Intelligent-Tiering (`INTELLIGENT_TIERING`) is an Amazon S3 storage class that's designed to optimize storage costs by automatically moving data to the most cost-effective access tier, without performance impact or operational overhead. S3 Intelligent-Tiering is the only cloud storage class that delivers automatic cost savings by moving data on a granular object level between access tiers when access patterns change. S3 Intelligent-Tiering is the ideal storage class when you want to optimize storage costs for data that has unknown or changing access patterns. There are no retrieval fees for S3 Intelligent-Tiering.

For a small monthly object monitoring and automation fee, S3 Intelligent-Tiering monitors access patterns and automatically moves objects that have not been accessed to lower-cost access tiers. S3 Intelligent-Tiering delivers automatic storage cost savings in three low-latency and high-throughput access tiers.

S3 Intelligent-Tiering automatically stores objects in three access tiers:

- **Frequent Access** – Objects that are uploaded or transitioned to S3 Intelligent-Tiering are automatically stored in the Frequent Access tier.
- **Infrequent Access** – S3 Intelligent-Tiering moves objects that have not been accessed in 30 consecutive days to the Infrequent Access tier.
- **Archive Instant Access** – With S3 Intelligent-Tiering, any existing objects that have not been accessed for 90 consecutive days are automatically moved to the Archive Instant Access tier.

Storage classes for infrequently accessed objects

The **S3 Standard-IA** and **S3 One Zone-IA** storage classes are designed for long-lived and infrequently accessed data. (IA stands for *infrequent access*.) S3 Standard-IA and S3 One Zone-IA objects are available for millisecond access (similar to the S3 Standard storage class). Amazon S3 charges a retrieval fee for these objects, so they are most suitable for infrequently accessed data.

For example, you might choose the S3 Standard-IA and S3 One Zone-IA storage classes to do the following:

- For storing backups.
- For older data that is accessed infrequently, but that still requires millisecond access. For example, when you upload data, you might choose the S3 Standard storage class, and use lifecycle configuration to tell Amazon S3 to transition the objects to the S3 Standard-IA or S3 One Zone-IA class.

These storage classes differ as follows:

- **S3 Standard-IA** (`STANDARD_IA`) – Amazon S3 stores the object data redundantly across multiple geographically separated Availability Zones (similar to the S3 Standard storage class). S3 Standard-IA objects are resilient to the loss of an Availability Zone. This storage class offers greater availability and resiliency than the S3 One Zone-IA class.
- **S3 One Zone-IA** (`ONEZONE_IA`) – Amazon S3 stores the object data in only one Availability Zone, which makes it less expensive than S3 Standard-IA. However, the data is not resilient to the physical loss of the Availability Zone resulting from disasters, such as earthquakes and floods. The S3 One Zone-IA storage class is as durable as S3 Standard-IA, but it is less available and less resilient.

Storage classes for rarely accessed objects

The **S3 Glacier Instant Retrieval** (`GLACIER_IR`), **S3 Glacier Flexible Retrieval** (`GLACIER`), and **S3 Glacier Deep Archive** (`DEEP_ARCHIVE`) storage classes are designed for low-cost,

long-term data storage and data archiving. These storage classes require minimum storage durations and retrieval fees making them most effective for rarely accessed data.

Amazon S3 provides the following S3 Glacier storage classes:

- **S3 Glacier Instant Retrieval** (`GLACIER_IR`) – Use for long-term data that's rarely accessed and requires milliseconds retrieval. Data in this storage class is available for real-time access.
- **S3 Glacier Flexible Retrieval** (`GLACIER`) – Use for archives where portions of the data might need to be retrieved in minutes. Data in this storage class is archived, and not available for real-time access.
- **S3 Glacier Deep Archive** (`DEEP_ARCHIVE`) – Use for archiving data that rarely needs to be accessed. Data in this storage class is archived, and not available for real-time access.

EBS:

Type	Use Case	Key Traits
gp3	General-purpose workloads	Default volume; customizable IOPS & throughput; lower cost than gp2
gp2	Legacy general-purpose (still supported)	Performance scales with size; simple to use but less flexible than gp3
io2	High-performance databases, critical apps	Very high durability (99.999%); consistent IOPS; supports Multi-Attach
io1	Older version of io2	High IOPS; supports critical workloads, but io2 is more efficient
st1	Big data, log processing, data warehousing	HDD-based; throughput optimized; cost-effective for sequential access
sc1	Cold storage, infrequent access	Lowest cost; good for archival; very low performance and throughput
Magnetic	Legacy use only (previously “standard”)	Basic performance; rarely used today

Feature	Description
Snapshots can be deleted	Yes, anytime
Snapshots are incremental	Only changed data is stored after the first snapshot
Storage optimized	You’re only billed for what’s actually stored
Data not lost	AWS keeps necessary blocks even if older snapshots are deleted

Feature	Description
Durable & Reliable	Data is automatically replicated within its Availability Zone.
Persistent Storage	Data is retained even if the instance is stopped.
Scalable	You can increase volume size, change volume type, or adjust performance.
Snapshots	You can back up EBS volumes using snapshots stored in Amazon S3.
Encrypted	Supports encryption for data at rest and in transit. (KMS)
Attachable	Volumes can be attached to a single EC2 instance at a time (except for multi-attach-enabled io2 volumes).

Feature	Fast Snapshot Restore (FSR)
Goal	Full performance from the start after restore
Problem it solves	Slow initial performance of lazy-loaded volumes
Enabled on	Snapshot + Availability Zone
Cost	Extra charge per snapshot per AZ
Best for	Prod systems, disaster recovery, large volume launches

- **AZ-Scoped:** EBS volumes are **only available within the same AZ** as the attached instance.
- **Persistent:** Data on EBS persists even if you stop or terminate the EC2 instance (unless `DeleteOnTermination` is set to `true` for the root volume).
- **Attach/Detach:** EBS volumes can be **detached** and attached to another instance in the same AZ.
- **IOPS:** io1/io2 volumes allow **provisioning up to 64,000 IOPS** (depends on instance type).
- **Snapshots & AMIs:** Snapshots can be used to create AMIs for rapid EC2 deployment.

EFS:

Feature	Description
Backup	Managed by AWS Backup, supports scheduling & vaults
Replication	Cross-region read-only copy, helps with disaster recovery
Encryption	At rest (with KMS) and in transit (TLS)
Access Control	POSIX + IAM + EFS Access Points

Feature	Description
File system	Like a shared folder — supports files and directories
Shared access	Many EC2 instances (or containers) can mount it at once
Fully managed	AWS handles scaling, availability, and durability
Scalable	Grows/shrinks automatically as files are added or removed
POSIX-compliant	Works like a Linux file system (permissions, paths, etc.)
Pay-as-you-go	Pay only for what you use (per GB/month)
Accessible across AZs	Unlike EBS, it's regional and available in multiple Availability Zones

Feature	Can enable after creation?	Notes
Replication	✅ Yes	Creates a read-only replica in another region
Encryption at rest	❌ No	Must be enabled at creation; requires new EFS for changes
Encryption in transit	✅ Yes	Use <code>-o tls</code> with EFS mount helper for secure communication

Feature	EFS	EBS	S3
Type	File Storage	Block Storage	Object Storage
Mountable by	Multiple EC2s	One EC2 (or Multi-Attach io2)	Web/API based access
Durability	High	High	Very High
Scalability	Auto-scales instantly	Manual sizing	Unlimited
Access Pattern	Shared filesystem	Like a hard drive	Upload/download of objects
Example Use Case	Shared media for web apps	OS disk for a VM	Static assets, backups

FSx:

FSx Variant	Best For	Underlying File System
FSx for Windows File Server	Windows-based apps needing SMB access	Windows NTFS

FSx Variant	Best For	Underlying File System
FSx for Lustre	High-performance workloads (HPC, ML, etc.)	Lustre (Linux)
FSx for OpenZFS	Linux-based apps needing snapshots, clones	OpenZFS
FSx for NetApp ONTAP	Enterprise NAS and hybrid cloud use cases	NetApp ONTAP

Feature	Description
Fully managed	AWS handles setup, patching, backups, scaling, and recovery
High performance	Optimized IOPS and throughput depending on the file system
Secure	Supports encryption at rest (KMS), in transit, and VPC access
Integrated backups	Automatic daily backups or manual snapshot options
Multi-AZ (optional)	Some variants support failover across Availability Zones

FSx Type	Example Use Cases
Windows File Server	File shares for Windows apps, Active Directory home directories
Lustre	Machine learning, high-performance computing, video rendering pipelines
OpenZFS	Linux workloads, snapshots, copy-on-write backups
NetApp ONTAP	Hybrid environments, enterprise NAS replacement, disaster recovery

AWS DataSync:

Online data transfer and discovery service that simplifies data migration and helps you quickly, easily, and securely transfer your file or object data to, from, and between AWS storage services.

Use Case	Why Use DataSync?
Migrate on-prem file servers to AWS	Move data to S3, EFS, or FSx without manual effort
Transfer backups to S3	Automatically sync backup files regularly
Sync data between AWS regions	Enable geo-redundancy or disaster recovery setups
Replace rsync or custom scripts	No need to build/maintain your own tools

AWS Transfer Family:

Secure transfer service that enables you to transfer files into and out of AWS storage services. Transfer Family is part of the AWS Cloud platform. AWS Transfer Family offers fully managed support for the transfer of files over SFTP, AS2, FTPS, FTP, and web browser-based transfers directly into and out of AWS storage services.

AWS Transfer Family supports transferring data from or to the following AWS storage services.

- Amazon Simple Storage Service (Amazon S3) storage.
- Amazon Elastic File System (Amazon EFS) Network File System (NFS) file systems.

The following are some common use cases for using Transfer Family with Amazon S3:

- Data lakes in AWS for uploads from third parties such as vendors and partners.
- Subscription-based data distribution with your customers.
- Internal transfers within your organization.

The following are some common use cases for using Transfer Family with Amazon EFS:

- Data distribution
- Supply chain
- Content management
- Web serving applications

AWS Storage Gateway:

AWS Storage Gateway is a hybrid cloud storage service that lets your on-premises applications use AWS cloud storage seamlessly.

Type	What it emulates	Use Case
File Gateway	NFS/SMB file shares	Store files in S3 with local caching
Volume Gateway	iSCSI block volumes	Backup/replicate local disks to EBS/S3
Tape Gateway	Virtual tape library (VTL)	Replace physical backup tapes with S3/Glacier-based storage

Use Case	Gateway Type
On-prem file servers to S3	File Gateway
Virtual tape backups to AWS	Tape Gateway
Disk-to-cloud disaster recovery	Volume Gateway

Use Case	Gateway Type
Cloud-based archive storage	File / Tape
Hybrid cloud data sharing	File Gateway

Feature	File Gateway	Volume Gateway	Tape Gateway
Protocol	NFS, SMB	iSCSI	iSCSI (VTL)
Storage backend	Amazon S3	S3 (backups), EBS	S3 Glacier / Deep Archive
Local caching	Yes	Yes (optional)	Yes
Use case	File shares & backup	Block-level storage	Tape backup replacement

AWS Backup:

AWS Backup is a fully managed backup service that helps you centralize, automate, and manage backups across many AWS services and even some on-premises resources. Instead of manually creating snapshots or backup scripts for every service, AWS Backup lets you define **backup policies**, **schedules**, **retention**, and **lifecycle rules** — all in one place.

ASG Scaling Policies:

Scaling Policy Type	How It Works	Best For	Example Use Case
Manual Scaling	You manually set the desired number of instances	Full control, rarely changing workloads	Admin manually sets desired capacity to 4
Target Tracking Scaling	Automatically adjusts to maintain a target metric (e.g., CPU = 60%)	Hands-off, dynamic load handling	Keep average CPU at 50%
Step Scaling	Responds to CloudWatch alarm breaches with scaling steps	Tiered scaling based on metric severity	Add 1 instance if CPU > 60%, 2 if > 80%
Simple Scaling	Triggers a scaling action after a single CloudWatch alarm	Basic automation with cooldown	Add 1 instance when CPU > 70% for 5 minutes
Scheduled Scaling	Scales based on defined time schedules	Predictable load patterns	Scale out at 9AM, scale in at 5PM on weekdays
Predictive Scaling	Uses machine learning to forecast future load and	Seasonal traffic, regular patterns	Retail app scales up automatically before

Scaling Policy Type	How It Works	Best For	Example Use Case
	scale in advance		Black Friday

EC2 Instance Types:

Family	Optimized For	Naming Example	Use Cases
General Purpose	Balanced CPU, memory, networking	t3 , t4g , m6i	Web servers, dev/test environments, small databases
Compute Optimized	High-performance CPUs	c6g , c7i	High performance computing (HPC), batch processing, ad serving, video encoding, gaming, scientific modeling, distributed analytics, and CPU-based ML inference.
Memory Optimized	High memory-to-CPU ratio, designed to deliver fast performance for workloads that process large data sets in memory.	r6i , x2idn	In-memory databases (Redis), real-time big data analytics
Storage Optimized	High local storage throughput, designed for workloads that require high, sequential read and write access to very large data sets on local storage.	i4i , d3	NoSQL databases, data warehousing, Elasticsearch
Accelerated Computing	GPU or FPGA acceleration	p4d , g5 , inf2	ML/AI training, HPC, video processing, inference workloads

VPC:

Feature	VPC Sharing	VPC Peering	VPC Endpoints
Purpose	Share VPC subnets across accounts	Connect two VPCs privately	Connect to AWS services privately
Scope	Intra-org accounts	Intra or cross-account/region	Service-level within same region
Transitive?	N/A	✗ No	N/A

Feature	VPC Sharing	VPC Peering	VPC Endpoints
Billing Owner	Participant account	Each account pays own resources	VPC owner pays
Use Case	Central VPC in org-wide setups	App-to-app VPC communication	Access AWS services without NAT/IGW

CF StackSets:

AWS CloudFormation StackSets is an extension of CloudFormation that lets you deploy and manage stacks across multiple AWS accounts and Regions — all from a single admin account.

Think of it as CloudFormation at scale: it helps you centrally manage infrastructure as code across your entire organization or environment.

A **StackSet** is a collection of:

- A **CloudFormation template** (defining resources like VPCs, IAM roles, etc.)
- **Deployment targets** (accounts and regions)
- **Operations settings** (such as failure tolerance and concurrency)

Requires **IAM roles**:

- **Admin role** in the administrator account.
- **Execution role** in target accounts.

Route 53:

Function	Description
Domain Registration	Register new domain names (e.g., <code>example.com</code>)
DNS Routing	Translate domain names into IP addresses (e.g., <code>www.example.com</code> → <code>192.0.2.1</code>)
Health Checks	Monitor the health of endpoints and route traffic only to healthy ones
Traffic Management	Route traffic based on location, latency, geoproximity, or custom rules
Private DNS	Use with Amazon VPCs for internal DNS resolution

Routing Policy	Description
Simple	Basic DNS resolution to a single IP or resource

Routing Policy	Description
Weighted	Split traffic by weights (e.g., 70% to one server, 30% to another)
Latency-based	Route to the region with the lowest latency
Failover	Primary/secondary setup using health checks
Geolocation	Route users based on their country or continent
Geoproximity (with Traffic Flow)	Route based on user's proximity to resources and bias settings
Multivalue Answer	Return multiple healthy IPs, simulating load balancing

Use Case	How Route 53 Helps
Register and manage your domain	Built-in domain registration and renewal
Route traffic to AWS services	Seamlessly integrates with EC2, ELB, S3, etc.
Route across regions or accounts	Use latency or geolocation routing
Implement high availability	Combine failover + health checks
Internal DNS within VPC	Use Private Hosted Zones

Amazon Aurora:

Amazon Aurora is a fully managed relational database engine designed by AWS for high performance and availability. It is MySQL- and PostgreSQL-compatible but offers up to 5x the performance of standard MySQL and 3x for PostgreSQL, with the scalability and reliability of AWS cloud infrastructure.

Feature	Description
MySQL/PostgreSQL-compatible	Use familiar drivers and tools without rewriting apps
Fully managed by RDS	Automated backups, patching, monitoring, and failover
High performance	5x faster than MySQL, 3x faster than PostgreSQL
High availability	Up to 15 read replicas and automatic failover across AZs
Auto-scaling storage	Scales automatically from 10 GB up to 128 TB
Aurora Global Databases	Multi-region replication with low latency
Aurora Serverless	On-demand, auto-scaling database — pay only for usage
Backtrack	Roll back a database without restoring from a backup (MySQL only)

Feature	Description
Fault-tolerant design	Six copies of data across 3 Availability Zones

Aurora Serverless:

Amazon Aurora Serverless is an on-demand, auto-scaling configuration for Amazon Aurora where the database automatically starts up, shuts down, and scales based on your application's needs — without managing instances.

Feature	Description
Auto-scaling	Scales compute capacity up/down based on actual usage (in Aurora Capacity Units - ACUs)
On-demand	Starts only when a query is made; pauses when idle (v1 only)
Fully managed	No need to manage database instances or clusters
Cost-efficient	Pay per second of usage; ideal for variable or infrequent workloads
Instant provisioning (v2)	v2 adds near-instant scaling and continuous operation

Use Case	Best Fit?
Infrequent or unpredictable workloads	✅ Yes
Development, test, or QA environments	✅ Yes
E-commerce or SaaS apps with usage spikes	✅ Yes (v2 preferred)
High-throughput transactional systems	❌ No (Standard Aurora better)
Multi-AZ and failover-critical workloads	✅ Aurora Serverless v2 only

Feature	Aurora Standard	Aurora Serverless (v2)
Instance management	You choose and manage	Fully managed, auto-scaling
Auto scaling	Manual or limited	Seamless & granular
Startup time	Always on	Instant (v2), delayed (v1)
Availability	Multi-AZ	✅ Yes (v2 only)
Use case	Steady, high-throughput apps	Variable or bursty workloads
Billing model	Per-hour/per-instance	Per-second per ACU

Amazon DocumentDB:

Amazon DocumentDB is a fully managed NoSQL document database service designed to

store, query, and index JSON-like documents at scale. It is MongoDB-compatible, meaning you can use your existing MongoDB drivers and tools with minimal changes.

Feature	Description
MongoDB compatibility	Supports MongoDB API (3.6, 4.0, 5.0 compatibility)
Fully managed	Automated backups, patching, replication, and monitoring
Scalable	Storage auto-scales up to 128 TiB; supports read replicas
Highly available	Multi-AZ deployment with automatic failover
Integrated with AWS	Works with IAM, VPC, KMS, CloudWatch

- Content management systems
- Catalogs and metadata stores
- Mobile and web applications requiring flexible schema

DynamoDB:

Amazon DynamoDB is a fully managed NoSQL database service that delivers single-digit millisecond performance at any scale. It supports both key-value and document data models, making it ideal for high-performance, internet-scale applications.

Feature	Description
Fully managed	No server provisioning, patching, or maintenance
NoSQL data model	Supports key-value and document structures
Scalable performance	Automatically scales throughput and storage
Consistent low latency	<10 ms reads/writes at virtually any scale
Durable & available	Replicated across 3 Availability Zones
On-demand or provisioned	Choose billing model based on workload patterns
Global tables	Multi-region replication for low-latency global apps
DAX (DynamoDB Accelerator)	In-memory caching for microsecond response times
Built-in security	Integration with IAM, VPC, KMS (encryption), CloudTrail
Streams & Triggers	Event-driven architecture via DynamoDB Streams + Lambda

- Real-time bidding, gaming, and IoT apps
- Session management and user profile storage
- Shopping carts and product catalogs

- Serverless apps (integrates tightly with Lambda)

Pricing Model:

Mode	Description
On-demand	Pay per read/write request, no capacity planning
Provisioned	Specify read/write capacity units; can auto-scale
Storage	Billed per GB/month
Optional features	DAX, Streams, Global Tables cost extra

DynamoDB Global Tables provide multi-region, fully active, fully managed replication of your DynamoDB tables — allowing you to read and write to the same table from multiple AWS Regions, with automatic data synchronization and low-latency access for global users.

Feature	Description
Multi-region replication	Data is replicated automatically across selected regions
Active-active design	You can read and write to any replica in any region
Automatic conflict resolution	Uses last-writer-wins to resolve write conflicts
Highly available	Built-in regional failover — app can switch to another region
Disaster recovery	Resilient to Region-level outages, Eventually Consistent only
Global latency reduction	Users access the closest region to reduce response time

DAX:

Amazon DAX is a fully managed, in-memory cache for Amazon DynamoDB that improves read performance by delivering microsecond response times, even at millions of requests per second.

- Applications with **frequent read access** to the same data
- **Gaming, social media, or leaderboards**
- Real-time dashboards or recommendation engines
- Reducing **read load** on DynamoDB and improving cost-efficiency
- Best suited for **eventually consistent** reads

Amazon KeySpaces:

Amazon Keyspaces is a managed, serverless, scalable implementation of Apache Cassandra

— a wide-column NoSQL database.

Amazon Neptune:

Amazon Neptune is a fully managed graph database service designed for highly connected data.

Use cases: social networks, recommendation engines, fraud detection.

Amazon QLDB (Quantum Ledger Database):

Amazon QLDB is a fully managed ledger database that provides an immutable, cryptographically verifiable transaction log.

Use for audit logs, financial records, compliance tracking, or supply chain systems.

Amazon ElastiCache:

Amazon ElastiCache is a fully managed, in-memory data store and cache service that supports:

- Redis (key-value store with rich data types)
- Memcached (simple, high-speed caching)

Use when you need fast, in-memory access to frequently used data.

General-purpose in-memory caching layer for any app or DB (Both SQL and NoSQL).

Amazon RDS:

Amazon RDS is a fully managed relational database service that makes it easy to set up, operate, and scale databases in the cloud. It handles routine database tasks such as provisioning, patching, backups, and scaling — so you can focus on your application.

Feature	Description
Fully managed	AWS handles provisioning, backups, patching, monitoring
Supports multiple engines	MySQL, PostgreSQL, MariaDB, Oracle, and Microsoft SQL Server
Automatic backups	Daily snapshots + point-in-time restore (up to 35 days)
Multi-AZ deployments	High availability via synchronous replication to standby in another AZ
Read replicas	Offload read traffic to replicas (MySQL, PostgreSQL, MariaDB, Aurora)
Monitoring	CloudWatch metrics, Enhanced Monitoring, and Performance Insights
Encryption	At rest (via KMS) and in transit (via SSL)
Scaling	Vertical (instance type) or horizontal (read replicas)
Maintenance windows	Apply updates during user-defined periods

- Web and mobile apps with structured data
- Business-critical applications requiring high availability
- Migrating on-premise databases to the cloud
- Applications needing complex queries and transactions

Feature	Benefit
Multi-AZ	Automatic failover for HA
Read Replicas	Scale read workloads and disaster recovery

How RDS Provides HA:

Feature	Description
Multi-AZ Deployment	Creates a primary and a standby DB instance in different AZs
Synchronous replication	Real-time data replication to standby to ensure no data loss
Automatic failover	If the primary instance fails, RDS promotes the standby to primary
Managed by AWS	No manual intervention needed during failover

How to Achieve DR in RDS:

Strategy	Purpose
Manual cross-region snapshots	Copy backups to another AWS region
Read Replicas in other regions	For MySQL, PostgreSQL, MariaDB — can be promoted to primary
Aurora Global Databases	For Aurora only — built-in cross-region replication
Point-in-time recovery	Recover data from specific timestamp using backups
Automated and manual snapshots	Protect against app-level data loss

Amazon Redshift:

Amazon Redshift is a fully managed, petabyte-scale data warehouse service designed for online analytical processing (OLAP) — meaning it's optimized for complex queries across large datasets, rather than high-volume transactional operations.

Use Case	Why Redshift?
Business Intelligence (BI)	Fast, complex queries on structured data
Data lakes with S3	Redshift Spectrum lets you query S3 directly
Reporting dashboards	Scales to handle 1000s of users and dashboards
Historical data analysis	Ideal for querying large amounts of past records

Feature	Traditional Database (e.g., RDS, DynamoDB)	Data Warehouse (e.g., Redshift)
Purpose	Handle day-to-day transactions (OLTP)	Run complex queries on large datasets (OLAP)
Data access pattern	Frequent inserts/updates	Large-scale reads, aggregations, reporting
Query type	Simple queries (e.g., CRUD)	Complex joins, filters, group-bys
Data structure	Normalized (many tables, strict relationships)	Denormalized (fewer, wider tables)
Example use case	Banking system, e-commerce orders	Sales reporting, business intelligence dashboards
Performance optimized for	Fast reads/writes of individual records	High-throughput analytics on large sets
Storage model	Row-based storage	Columnar storage

You can write to Redshift, but it's best for batch inserts or data loading for analytics, not for handling frequent, real-time transactions like a traditional database.

Service	Description
Amazon RDS	Managed relational DB (MySQL, PostgreSQL, Oracle, etc.)
Amazon Aurora	Cloud-native relational DB (MySQL/PostgreSQL compatible)
Amazon Redshift	Data warehouse optimized for analytics (OLAP SQL)
Amazon QLDB	Immutable ledger DB with SQL-like query language

Service	Description
Amazon DynamoDB	Key-value & document store with microsecond latency
Amazon DocumentDB	Document store (MongoDB compatible)
Amazon Keyspaces	Wide-column store (Apache Cassandra compatible)

Service	Description
Amazon ElastiCache	In-memory key-value store (Redis or Memcached)
Amazon Neptune	Graph database (supports Gremlin and SPARQL)
Amazon DAX	In-memory cache for DynamoDB (not a DB itself)

Analytics:

Amazon Athena:

A serverless interactive query service that lets you use standard SQL to analyze data directly in Amazon S3.

AWS Data Exchange:

A service for finding, subscribing to, and using third-party data in the AWS Cloud.

AWS Data Pipeline:

A workflow orchestration service to move and transform data between AWS services and on-prem sources on a schedule.

Amazon EMR:

A big data platform for processing vast amounts of data using Apache Spark, Hadoop, Hive, HBase, and more.

AWS Glue:

A serverless data integration and ETL (Extract, Transform, Load) service that prepares data for analytics and ML.

Amazon Kinesis:

A platform for real-time data streaming. It lets you collect, process, and analyze data as it moves through your system. Useful for log processing, real-time analytics, and application monitoring. Includes Kinesis Data Streams, Data Firehose, Data Analytics, and Video Streams.

- Kinesis Data Streams - **A service that lets you collect and process real-time streaming data (like logs, events, or metrics) with millisecond latency.**
- Kinesis Data Firehose - **A fully managed delivery service that automatically loads streaming data into AWS destinations like S3, Redshift, OpenSearch, or a 3rd-party tool.**

Amazon Lake Formation:

A service to help you set up a secure data lake quickly. It simplifies collecting data from different sources, cataloging it, and setting up access controls. It integrates with services like S3, Glue, and Athena.

Amazon MSK:

A fully managed service that makes it easy to run Apache Kafka on AWS. It handles provisioning, patching, and scaling so you can focus on your streaming application logic. Ideal for building real-time pipelines and streaming apps.

Amazon Opensearch Service:

A managed service that makes it easy to deploy and operate OpenSearch (formerly Elasticsearch) for search, log analytics, and observability. It supports full-text search, dashboards (with OpenSearch Dashboards), and real-time log monitoring.

Amazon QuickSight:

A scalable business intelligence service for creating and sharing interactive dashboards and visualizations. It can connect to many data sources (like S3, Redshift, Athena) and lets users build charts and insights without needing to manage servers.

Application Integration:

Amazon AppFlow:

A fully managed integration service that allows you to securely transfer data between SaaS applications (like Salesforce, Slack, ServiceNow) and AWS services (like S3, Redshift). It can move data bi-directionally, apply transformations, and support event-driven or scheduled flows.

AWS AppSync:

A managed GraphQL API service that lets you easily build APIs that combine data from multiple sources (e.g., DynamoDB, Lambda, HTTP endpoints). It supports real-time updates via subscriptions and can handle offline scenarios for mobile/web apps.

Amazon MQ:

A managed message broker service for Apache ActiveMQ and RabbitMQ. It's used to migrate or run traditional message-based applications (like Java apps using JMS) on AWS, without re-architecting to newer messaging services like SQS.

Amazon EventBridge:

Amazon EventBridge is a serverless event bus service that makes it easy to build event-driven applications by connecting AWS services, SaaS applications, and your own apps using events.

Feature	Description
Event bus	Central hub for sending and receiving events
Built-in integrations	Connects to over 200 AWS services and SaaS providers
Custom events	Your app can publish events directly to EventBridge
Event filtering & routing	Rules to selectively route events to specific targets
Serverless	Fully managed, no infrastructure to manage
Schema registry	Catalogs event schemas for easy discovery and documentation

Common Use Cases:

- Decouple microservices using an event-driven architecture
- Trigger Lambda functions or Step Functions based on events
- Monitor AWS resource changes (e.g., EC2 state changes, S3 uploads)

- Connect SaaS tools (like Auth0, Zendesk) with your AWS services
- Build workflows that react in real time to application events

Example Event Flow:

1. **Event source:** S3, DynamoDB, or your app sends an event (like an object upload).
2. **Event bus:** EventBridge receives and processes the event.
3. **Rules:** Filters match relevant events and route them to the right targets.
4. **Targets:** Services like Lambda, SNS, SQS, Step Functions, or EC2.

Amazon SNS:

Amazon SNS is a fully managed, pub/sub messaging service that enables you to send messages (notifications) to multiple subscribers instantly and simultaneously.

Feature	Description
Pub/Sub Model	Publishers send messages to topics; subscribers receive them
Multiple protocols	Supports HTTP/S, Lambda, SQS, email, SMS, mobile push
Scalable and durable	Automatically scales to handle millions of messages
Message filtering	Basic attribute-based filtering for subscribers
Fan-out delivery	One published message can be delivered to multiple endpoints simultaneously
Fully managed	No servers to manage; AWS handles delivery and retries

Use Cases:

- Sending **alerts and notifications** (e.g., billing alerts, system events)
- Fan-out messages to multiple microservices (like broadcasting to SQS queues, Lambda functions, or HTTP endpoints)
- Mobile push notifications for apps
- Decoupling application components

Amazon SQS:

Types of SQS Queues:

Queue Type	Key Features	Use Cases
Standard Queue	- Unlimited throughput - At-least-once delivery - Best-effort ordering (no	Decoupling microservices, high-volume event ingestion, general-

Queue Type	Key Features	Use Cases
	guaranteed order)	purpose queuing
FIFO Queue	- First-In-First-Out delivery- Exactly-once processing - Limited to ~3,000 transactions/sec	Order-sensitive workflows, payment processing, event logs where ordering matters

Visibility Timeout:

The visibility timeout is the period of time that a message is “hidden” from other consumers after it’s retrieved from the queue.

- When a consumer receives a message, it has visibility timeout seconds to process it.
- If the consumer successfully processes the message and deletes it, the message is removed.
- If the message is not deleted (e.g., processing failed), it becomes visible again after the timeout expires → another consumer can try again.

Default: 30 seconds. Adjustable up to 12 hours.

Dead Letter Queues:

Feature	Description
How it works	You configure a max receive count (e.g., 5). If a message fails that many times, it’s moved to the DLQ.
Why it’s useful	Helps you debug and isolate problematic messages without blocking normal processing.
Separate queue	DLQ is a separate SQS queue you can monitor or inspect.

Types of Polling:

Polling Type	Description
Short polling	SQS immediately responds, even if no messages available. Fast but can be wasteful.
Long polling	Consumer waits for up to 20 seconds. If no messages, reduces empty responses → more efficient!

AWS Step Functions:

AWS Step Functions is a fully managed orchestration service that helps you coordinate multiple AWS services into serverless workflows. It's like a visual flowchart for your app logic, with built-in error handling and retry logic.

Feature	Description
State machines	Define workflows as states and transitions
Visual editor	Build and view workflows in the AWS Console
Built-in error handling	Automatic retries and catch/try blocks
Integration with 200+ AWS services	No glue code needed to connect services
Express workflows	Lower-cost, high-speed workflows for short tasks

State machines - In AWS Step Functions, a state machine is the entire workflow definition — it's a blueprint for how your application logic flows from one step to the next.

Containers:

Amazon ECS:

Amazon ECS is a fully managed container orchestration service that makes it easy to run, stop, and manage containers on a cluster of virtual machines.

Two Launch Types - **EC2**: run containers on your own EC2 instances , **Fargate**: run containers **serverlessly** (no EC2 management).

Specifically designed for Docker containers.

- ECS uses the **Docker image format** for defining container images.
- It uses the **Docker runtime** under the hood (via the **containerd** core engine).
- Tasks in ECS are defined by **task definitions** that **specify Docker images**, networking, CPU/memory, etc.

A task definition in ECS is like a blueprint or recipe for running one or more containers together as a single unit (a task).

Amazon EKS:

Amazon EKS is a fully managed Kubernetes service that makes it easy to run Kubernetes clusters on AWS — without having to manage the control plane yourself.

Amazon ECS Anywhere:

Amazon ECS Anywhere extends the Elastic Container Service (ECS) to run containers on your own infrastructure — not just within AWS.

Use cases:

- Hybrid environments (on-prem + cloud)

- Edge computing (like factories, retail stores)
- Compliance needs (run containers close to data sources)

How it Works:

1. Register your on-prem instances as **ECS-capable** using the ECS Anywhere agent.
2. ECS schedules tasks on these external instances.
3. ECS continues to **orchestrate** and **monitor** them like cloud tasks.

Amazon EKS Anywhere:

Amazon EKS Anywhere is an extension of Amazon EKS that allows you to run Kubernetes clusters on your own infrastructure (like on-premises data centers or edge environments).

Amazon EKS Distro:

Amazon EKS Distro is a production-ready Kubernetes distribution built from the same open-source components that Amazon EKS uses.

Feature	Amazon EKS	Amazon EKS Anywhere	Amazon EKS Distro (EKS-D)
Where it runs	Fully in AWS cloud	On-premises (bare metal or VMware)	Anywhere you want — self-managed
Control plane	Fully managed by AWS	Self-managed on-premises	Self-managed, your responsibility
Kubernetes version	AWS-tested & supported	Same as EKS (AWS-tested)	Same as EKS (AWS-tested)
Node management	Managed node groups (optional)	You provision and manage the nodes	You provision and manage everything
Integration	Deep AWS integrations (IAM, CloudWatch, etc.)	Limited (no built-in control plane in AWS)	None — completely self-managed
Use case	Run K8s in AWS cloud	Hybrid & edge deployments with EKS tooling	DIY Kubernetes clusters, fully portable
Updates and security	AWS automatically handles control plane updates	AWS provides lifecycle tools (you do it)	AWS publishes security-tested K8s binaries (you do it)
Best for	Cloud-native workloads in AWS	Hybrid cloud & edge with EKS experience	Fully portable, consistent EKS-like K8s

Best for **DIY** and maximum flexibility — AWS publishes but you run it!

Amazon ECR:

Amazon ECR is a fully managed container image registry that lets you store, manage, and share Docker container images.

Developer Tools:

AWS X-Ray:

AWS X-Ray is a distributed tracing service that helps you analyze and debug applications — especially those built with microservices or serverless architectures.

Feature	Description
Request tracing	Visualizes the flow of requests across services
Service map	Shows how different services and resources interact
Root cause analysis	Pinpoints slow services or errors in complex systems
Filter and search	Focus on problem areas in your distributed app
Integrates with AWS services	Works with Lambda, ECS, API Gateway, and more

Use Cases:

- Debugging **slow API calls** or **intermittent errors**
- Identifying **bottlenecks** in microservices architectures
- Gaining **end-to-end visibility** of requests across your stack
- Monitoring performance impacts of new deployments

Front End Web and Mobile:

AWS Amplify:

A frontend and mobile development platform that helps you build, deploy, and host web and mobile apps quickly.

- Integrated hosting, authentication, GraphQL/REST APIs, and analytics
- Works with **React, Angular, iOS, Android, and more**

AWS Device Farm:

A testing service for mobile and web applications on real devices.

Amazon Pinpoint:

A customer engagement service for sending targeted SMS, email, push notifications, and voice messages. Used for marketing campaigns, transactional messaging, and user engagement.

Amazon API Gateway:

Amazon API Gateway is a fully managed service that lets you create, deploy, and manage APIs

at any scale. Supports REST APIs, HTTP APIs, and WebSocket APIs.

Compute:

AWS Batch:

AWS Batch is a fully managed batch computing service that lets you run large-scale batch jobs in the AWS cloud.

Use Cases:

- Data analytics: ETL tasks, log processing, big data workloads.
- Image/video processing: Rendering or transformations in bulk.
- Scientific simulations: HPC tasks like genome sequencing or CFD.
- Machine learning: Large-scale model training or hyperparameter tuning.

Feature	AWS Batch	AWS Step Functions
Primary Purpose	Run large-scale batch jobs (like video processing, data analysis, HPC)	Orchestrate workflows and coordinate multiple AWS services and steps
Job Type	Batch jobs (compute-heavy, often independent)	State machine -style workflows with decision logic and error handling
Compute Management	Provisions and scales EC2 or Fargate compute resources automatically	Doesn't run jobs itself — calls AWS services (like Lambda, ECS, Batch!)
Parallelism	Runs many jobs in parallel (arrays, HPC tasks)	Supports parallel states (but for orchestration, not compute)
Dependency Handling	Job dependencies (A must finish before B)	Flexible branching and decision trees
Typical Use Cases	Data ETL, ML model training, log processing, video rendering	Chaining steps across services (e.g., S3 → Lambda → DynamoDB)
Integration with AWS	Runs containerized jobs using ECS	Orchestrates 200+ AWS services, including Batch!

AWS Beanstalk:

AWS Elastic Beanstalk is a fully managed service for deploying and running web applications and services. It's designed to help developers quickly deploy and manage applications without worrying about the underlying infrastructure.

Feature	Description
Supports multiple languages	Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker.
Managed environment	AWS automatically provisions EC2 instances, load balancers, auto scaling, and monitoring.
Environment customization	Use configuration files or the console to tweak environment settings.
Built-in monitoring	Integrated with CloudWatch for logs and metrics.
Rolling updates	Zero-downtime deployments and version tracking.

Use Cases:

- Deploying web apps, REST APIs, or backend services
- Quickly building development, test, or production environments
- Managing the lifecycle of apps without manually provisioning AWS resources

Feature	Elastic Beanstalk	ECS (Elastic Container Service)	EKS (Elastic Kubernetes Service)
Abstraction level	High — platform-as-a-service for app deployment	Medium — container orchestration service	Low — fully managed Kubernetes control plane
Deployment model	Upload app code directly (ZIP, WAR, etc.)	Deploy containers via task definitions (Docker)	Deploy and manage Kubernetes clusters
Complexity	Easiest — AWS does most of the heavy lifting	Intermediate — need to understand containers	Advanced — need to understand Kubernetes
Supported apps	Web apps, APIs, background tasks (traditional apps)	Container-based apps, microservices	Large-scale container orchestration, hybrid workloads
Customization	Limited — within Beanstalk environment settings	High — manage container definitions, networking	Highest — fully control K8s objects and resources
Scaling	Managed auto-scaling built-in	Fine-grained auto-scaling with ECS and ALB	Kubernetes-native auto-scaling and scheduling
Use of containers	Optional (supports Docker images)	Core feature — runs Docker containers	Core feature — Kubernetes manages

Feature	Elastic Beanstalk	ECS (Elastic Container Service)	EKS (Elastic Kubernetes Service)
			containers

When to use which?

Elastic Beanstalk

- You want to deploy quickly with minimal operational overhead.
- Your app is a traditional web app (Java, Node.js, PHP, etc.).
- You don't want to learn container orchestration.
- Best for small-to-medium apps or when you want to focus on the app itself, not infrastructure.

Amazon ECS

- You want to run containerized apps without managing servers.
- You need fine-grained control over container resources, networking, and scaling.
- Best for microservices or API backends in a fully AWS-native environment.

Amazon EKS

- You need Kubernetes for portability or because your team already uses K8s.
- You're working on complex, multi-service container workloads.
- You want full Kubernetes features like CRDs, Helm, and native service discovery.
- Best for large-scale container orchestration or hybrid/multi-cloud strategies.

AWS Outposts:

AWS Outposts is a fully managed service that extends AWS infrastructure, services, APIs, and tools to your on-premises data center or co-location facility.

Use Cases:

- Low-latency apps that must run close to data sources (e.g., factory floors, hospitals).
- Data residency and compliance requirements.
- Hybrid architectures: run some workloads on-prem while others run in the cloud.
- Edge computing for real-time analytics or processing.

AWS ships a pre-configured Outposts rack to your facility. Connect it to your on-prem network. Use it like any other AWS region — same APIs, same tools.

AWS Serverless Application Repository:

A managed repository of serverless apps and components.

- Lets you find, deploy, and publish reusable serverless applications.

- You can share apps within your organization or publicly.
- Great for quickly assembling common patterns (e.g., image resizing, Slack bots).

VMware Cloud on AWS:

A hybrid cloud service that runs VMware's vSphere-based environment on AWS infrastructure.

AWS Wavelength:

A service that brings AWS compute and storage to the edge of 5G networks. Best for real-time apps like gaming, AR/VR, live video streaming, and IoT.

Management and Governance:

AWS Compute Optimizer:

AWS Compute Optimizer is a recommendation service that uses machine learning to help you right-size your compute resources.

Use Cases:

- Identifying underutilized or over-provisioned EC2 instances
- Optimizing performance for Lambda functions
- Adjusting Auto Scaling group configurations for better efficiency
- Reducing overall AWS costs by matching resource needs to actual usage

AWS Config:

AWS Config is a fully managed service that continuously monitors and records your AWS resource configurations and changes over time.

Feature	Description
Configuration history	Records detailed snapshots of your AWS resource states
Resource relationships	Tracks how resources are related (e.g., EC2 instance → security group)
Compliance evaluation	Evaluates resources against rules and compliance policies
Change tracking	See exactly who changed what, when, and how (via API activity tracking)
Integrated with CloudTrail	Works with CloudTrail for deeper change event analysis

Use Cases:

- Auditing resource configurations for compliance
- Troubleshooting why something changed or broke
- Security posture tracking: Are resources secure and compliant?

- Building custom compliance dashboards

AWS Control Tower:

AWS Control Tower is a service that helps you set up and govern a secure, multi-account AWS environment — often called a landing zone.

AWS Health Dashboard:

The AWS Health Dashboard provides personalized, real-time information about the health of AWS services that impact your account.

Feature	Description
Service status updates	Shows issues, planned maintenance, and health events affecting AWS services globally and your account specifically.
Account-specific view	Displays impact on your resources — tailored to your environment.
Event details	Provides remediation guidance and status updates .
Notifications	Integrates with CloudWatch and SNS for proactive alerts.

AWS License Manager:

A service for managing software licenses across AWS and on-prem environments.

Amazon Managed Grafana:

A fully managed service for running Grafana dashboards to visualize your metrics, logs, and traces.

Amazon Managed Service for Prometheus:

A fully managed Prometheus-compatible monitoring service for containerized workloads.

AWS Proton:

AWS Proton is a fully managed service for deploying and managing container-based and serverless applications. Provides a self-service interface for developers to deploy apps using standardized, vetted templates created by platform teams.

AWS Service Catalog:

AWS Service Catalog lets you create, organize, and manage approved IT resources (products) in a centralized catalog. Ensures governance and compliance by letting users only deploy pre-approved resources.

AWS Trusted Advisor:

AWS Trusted Advisor is a real-time guidance and best practices tool that scans your AWS environment and gives recommendations to help you:

- ✓ Optimize costs
- ✓ Improve security
- ✓ Increase performance
- ✓ Boost fault tolerance
- ✓ Service limits — Avoid hitting AWS service limits.

Use Cases:

- Lower your AWS bill by eliminating unused resources or switching to reserved pricing.
- Identify security misconfigurations (e.g., open S3 buckets, IAM best practices).
- Improve reliability by catching under-provisioned or misconfigured resources.

AWS Well-Architected Tool:

The AWS Well-Architected Tool helps you assess, measure, and improve your workloads using AWS's proven architectural guidance, ensuring your cloud environments are secure, reliable, efficient, and cost-effective.

AWS Systems Manager:

AWS Systems Manager is a unified management service that helps you automate and manage your AWS resources and infrastructure, whether in the cloud or on-premises.

Feature	Description
Run Command	Remotely execute scripts and commands on managed instances.
Session Manager	Securely access EC2 instances without SSH or bastion hosts .
Patch Manager	Automate patching for OS and applications.
Parameter Store	Central location to securely store config values and secrets .
Inventory & Compliance	Track and audit resources for config and security compliance .
Automation	Create workflows to automate operational tasks like backups and deployments.

Media Services:

Amazon Elastic Transcoder:

Amazon Elastic Transcoder is a media transcoding service in the cloud that converts media files (like videos) from one format to another. Common use cases include generating different video resolutions (e.g., 480p, 720p) for streaming platforms, or converting video to audio-only files.

Amazon Kinesis Video Streams:

Amazon Kinesis Video Streams (KVS) is a service for securely streaming video data from connected devices (like security cameras or drones) to AWS. Use cases include smart home

security, live video monitoring, and ML-based video processing (like object detection or facial recognition).

Migration and Transfer:

AWS Application Discovery Service:

AWS Application Discovery Service is designed to help you plan your migration to AWS by automatically discovering on-premises servers and applications.

It gathers detailed information like:

- Server configuration (CPU, memory, network).
- Usage patterns (CPU, disk, network utilization).
- Running processes, network dependencies, and connections.

How it works:

- Install Discovery Agents on servers or use Agentless Discovery (via VMware connector).
- It collects data and sends it to AWS.
- You get insights into what's running, what's connected, and how workloads are structured.

AWS Application Migration Service:

AWS MGN is the primary AWS service for lift-and-shift server migrations. It replicates your entire source servers (on-prem or in other clouds) to AWS in real-time. Automatically converts them to run as native EC2 instances in AWS.

How it works:

- Install the AWS Replication Agent on your source servers.
- It continuously replicates data (disk, memory) to AWS, keeping the target up-to-date.

AWS Database Migration Service (DMS):

AWS DMS helps you migrate databases to AWS quickly and securely. Supports over 20 database engines (like Oracle, MySQL, PostgreSQL, SQL Server, Aurora). Handles homogeneous (same engine) and heterogeneous (different engines) migrations.

AWS Migration Hub:

AWS Migration Hub is a centralized console for tracking and managing migrations across your AWS services. It aggregates data from different migration tools (like MGN, DMS, Discovery Service) into one view. Visualizes your migration progress — from initial discovery to final deployment.

Security, Identity, and Compliance:

AWS Artifact:

AWS Artifact is your self-service portal for compliance reports and security documentation (like SOC, PCI, ISO certifications).

- Lets you download audit reports and agreements directly from AWS.
- Key tool for proving your environment's compliance to auditors and regulators.

AWS Audit Manager:

A compliance automation service that helps you continuously assess your AWS resource configurations and activities against industry standards.

- Generates audit-ready reports to make compliance easier (e.g., GDPR, HIPAA, CIS Benchmarks).

AWS Certificate Manager(ACM):

ACM is a service for provisioning, managing, and deploying SSL/TLS certificates for your AWS resources (like ALB, CloudFront, API Gateway).

- Simplifies certificate renewal and deployment to secure your applications.

AWS CloudHSM:

AWS CloudHSM is a managed hardware security module (HSM) service that provides dedicated hardware appliances in the AWS cloud for high-security cryptographic operations. The HSMs are FIPS 140-2 Level 3 compliant, making them suitable for highly regulated industries.

Feature	Description
Dedicated hardware	Unlike shared cryptography services (like KMS), you get your own physical HSM cluster .
Full control	You manage keys and cryptographic operations — AWS can't access your keys.
Standards compliance	Certified for FIPS 140-2 Level 3 — meets strict security requirements.
Integration with apps	Works with applications using standard cryptographic APIs (PKCS#11, JCE, CNG).
Scalability & reliability	Cluster multiple HSMs for high availability and scaling.

Use Cases:

- Financial services: Secure payment processing and encryption of sensitive data (like PCI DSS compliance).

- Healthcare and government: Meet data security requirements for PHI and PII.
- Digital signing: Sign documents or transactions with hardware-backed keys.
- Bring your own key (BYOK) scenarios: Let you control your keys used in other AWS services.

Amazon Detective:

A security investigation service that uses machine learning to analyze, visualize, and understand security data from AWS resources.

AWS Directory Service:

A fully managed directory service for Microsoft Active Directory (AD) and AWS resources.

AWS Firewall Manager:

A centralized security management service for configuring and managing AWS firewall rules across multiple accounts.

Amazon GuardDuty:

A threat detection service that uses machine learning and threat intelligence to monitor AWS accounts for malicious activity.

- Detects anomalies like suspicious API calls, data exfiltration, or unusual logins.
- Sends alerts so you can take action quickly.

Aspect	Amazon GuardDuty	Amazon Detective
Primary Role	Detects suspicious or malicious activity in AWS environments (threat detection).	Helps investigate and analyze security findings (root cause analysis).
Data Source	Continuously monitors AWS logs (VPC Flow Logs, CloudTrail, DNS logs).	Uses the findings from GuardDuty, CloudTrail, and VPC Flow Logs to build investigation graphs.
Outcome	Generates alerts (findings) on suspicious activity.	Provides interactive visualizations to help you understand what happened and why.
Use Case	"What's wrong right now?"	"Why did this happen? How is it related?"
Integration	Runs independently, generates alerts automatically.	Works with GuardDuty findings to provide deeper investigation insights.

Amazon Inspector:

A security assessment service that automatically scans AWS workloads (like EC2 instances and container images) for vulnerabilities and deviations from best practices.

- Identifies software vulnerabilities and unpatched packages.
- Provides actionable remediation steps to improve security posture.

Amazon Macie:

A data security and privacy service that uses machine learning to automatically discover, classify, and protect sensitive data in AWS (like PII in S3 buckets).

AWS Network Firewall:

A managed, highly available firewall service for VPC-level network protection.

AWS Resource Access Manager (RAM):

A service that lets you securely share AWS resources across accounts, organizational units (OUs), or organizations in AWS Organizations.

AWS Secrets Manager:

A fully managed service for securely storing and managing sensitive information (like API keys, database passwords, or certificates).

- Automatically rotates secrets to improve security posture.
- Provides secure and auditable access to secrets for applications.

AWS Security Hub:

A centralized security and compliance service that aggregates findings from multiple AWS security services (like GuardDuty, Macie, Inspector) and third-party tools.

AWS Shield:

A managed DDoS protection service that safeguards your applications from volumetric and targeted DDoS attacks. Shield Standard is included for all AWS customers, while Shield Advanced adds additional protections and 24/7 DDoS response support.

AWS WAF:

A service that protects web applications from common exploits (like SQL injection, XSS) and custom traffic filtering.

AWS IAM Identity Center:

A centralized identity and access management service that provides single sign-on (SSO) for AWS accounts and integrated apps.

- Connects with existing identity providers (like Microsoft AD or Okta) for centralized user authentication and authorization.
- Supports multi-account access and user provisioning for secure AWS access.

Amazon Cognito:

A user authentication and access management service for web and mobile apps.

- **IAM Identity Center** → AWS resource and app-level SSO **for employees/administrators**.
- **Cognito** → End-user authentication for **your app's users** (like customers, partners).

AWS Key Management Service:

AWS KMS is a fully managed encryption service that helps you create, manage, and control cryptographic keys used to encrypt data across your AWS environment. It's designed to make encryption easy and secure, while integrating with many AWS services like S3, EBS, RDS, DynamoDB, and more.

Feature	Description
Customer Master Keys (CMKs)	Central to KMS — create and manage symmetric or asymmetric keys for encryption and signing.
Automatic key rotation	Rotate keys automatically (e.g., every year) for best security practices.
Integration with AWS services	Many AWS services can encrypt data at rest using KMS-managed keys.
Access control	Uses IAM policies and key policies to tightly control who can use or manage keys.
Auditability	Integration with CloudTrail — every key usage request is logged.
Encryption APIs	Use AWS SDKs or APIs to encrypt, decrypt, and sign data within your apps.

Use Cases:

- Encrypting S3 objects or EBS volumes to protect sensitive data at rest.
- Encrypting database data (RDS, DynamoDB).
- Generating data keys for app-level encryption (like encrypting fields in a customer database).
- Digital signing and verifying signatures for sensitive transactions.

Serverless:

AWS Fargate:

AWS Fargate is a serverless compute engine for running containers in Amazon ECS and Amazon EKS. It eliminates the need to provision, configure, or manage EC2 instances for container workloads.

Feature	Description
Serverless	No need to manage servers — Fargate handles compute infrastructure automatically.

Feature	Description
Per-second billing	Pay only for the resources your containers use (CPU, memory).
Security	Each task or pod runs in its own isolated environment.
Scalability	Automatically scales to match container workload demands.
Integration	Works with ECS and EKS for orchestrating containers.

Service	What it is	Where it fits
Fargate	A serverless container compute engine	PaaS (Platform as a Service) – AWS manages the infrastructure and OS layers for you (no server management needed).
ECS	A container orchestration service	IaaS/PaaS hybrid – You can run it with Fargate (PaaS-like) or EC2 (IaaS-like) instances.
EKS	A managed Kubernetes control plane	IaaS/PaaS hybrid – AWS manages the Kubernetes control plane (PaaS-like), but you manage the worker nodes (IaaS) unless you use Fargate .

Storage:

AWS Backup:

AWS Backup is a fully managed backup service that helps you centralize and automate data protection across AWS services.

Feature	Description
Centralized backup	Manage backups for services like EC2, RDS, EFS, DynamoDB, S3, and more from one place.
Policy-based backup	Create backup plans to automate scheduling, retention, and lifecycle management.
Cross-region & cross-account backup	Supports cross-region and cross-account backups for disaster recovery.
Backup vaults	Store backups securely with encryption and access control.
Monitoring	Track backup activity and compliance with AWS Backup Audit Manager.

Use Cases:

- Disaster recovery for AWS workloads.
- Compliance with data retention policies (e.g., GDPR, HIPAA).

- Centralized backup for multi-account or hybrid environments.

Networking and Content Delivery:

AWS Client VPN:

A fully managed VPN service that allows users (like employees) to securely connect to AWS and on-prem resources from anywhere.

- Provides a secure TLS-based VPN endpoint.
- Ideal for remote work, secure access to VPCs.

AWS PrivateLink:

A private connectivity service that lets you access AWS services or third-party SaaS services securely and privately within your VPC.

AWS Direct Connect:

A dedicated network connection from your on-premises data center to AWS.

- Offers low latency and consistent performance compared to VPN over the internet.
Bypasses the public internet for more reliable, consistent, and lower-latency performance.

AWS Site-to-Site VPN:

A managed VPN service that lets you connect your on-prem network to an AWS VPC via an encrypted IPsec VPN tunnel over the internet.

AWS Transit Gateway:

A centralized hub that connects multiple VPCs and on-premises networks (via Direct Connect or VPN).

- Replaces complex VPC peering meshes with a hub-and-spoke architecture.
- Simplifies routing, management, and scalability for large networks.

Service	Main Purpose	Typical Use Case
Direct Connect	Dedicated, low-latency private link	Large data transfer, consistent hybrid cloud performance
Site-to-Site VPN	Encrypted tunnel over internet	Quick hybrid connection, backup to Direct Connect
Transit Gateway	Central hub for hybrid/cloud networking	Managing multiple VPCs and hybrid connections

Encrypting Data at Rest:

Data is stored on a physical disk (like an S3 bucket, EBS volume, RDS database) in an encrypted form so that even if someone gets access to the disk, they can't read the data

without the decryption key.

How it's done in AWS:

- AWS services like S3, EBS, RDS, DynamoDB support server-side encryption (SSE).
- KMS (Key Management Service) is often used to manage the encryption keys (customer-managed or AWS-managed keys).
- Encryption/decryption happens automatically by the service when data is read or written — you don't handle the keys directly if using AWS-managed options.

Encrypting Data in Transit:

Data is moving across networks (like between an app and S3, or between users and an API) and is protected by encryption during the transfer.

How it's done in AWS:

- Use TLS/SSL (HTTPS) for data moving over the internet (like S3 HTTPS endpoints).
- For AWS services talking to each other (e.g., Lambda to RDS), you enable encryption protocols (like TLS).
- AWS-managed services often support encryption in transit by default (like AWS API Gateway, S3 HTTPS, RDS SSL endpoints).

Service	Encryption at Rest	Encryption in Transit
Amazon S3	✓ SSE-S3, SSE-KMS, SSE-C (customer-provided keys)	✓ HTTPS (TLS) for all data transfers
Amazon EBS	✓ KMS-based encryption for volumes and snapshots	✓ Encrypted communication to/from EC2
Amazon RDS	✓ KMS-based encryption for data at rest	✓ SSL/TLS for database connections
Amazon DynamoDB	✓ Encryption at rest with AWS KMS	✓ HTTPS for API operations
Amazon Redshift	✓ KMS-based encryption for data at rest	✓ SSL/TLS for client connections
Amazon Aurora	✓ Encryption at rest using KMS	✓ SSL/TLS for database connections
Amazon SQS	✓ Server-side encryption with KMS	✓ HTTPS for sending/receiving messages
Amazon SNS	✓ Server-side encryption with KMS	✓ HTTPS for message delivery
Amazon CloudWatch Logs	✓ Encryption at rest with KMS	✓ HTTPS for data ingestion and API calls

Service	Encryption at Rest	Encryption in Transit
AWS Lambda	✓ Environment variables encrypted with KMS	✓ HTTPS for invocation
AWS Secrets Manager	✓ Secrets encrypted with KMS	✓ HTTPS for API operations
AWS Systems Manager Parameter Store	✓ Parameter values encrypted with KMS	✓ HTTPS for API access
Amazon EFS	✓ KMS-based encryption at rest	✓ TLS for client connections
AWS Backup	✓ Uses KMS for backup vault encryption	✓ HTTPS for backup/restore API calls
Amazon Kinesis (Streams, Firehose)	✓ Server-side encryption with KMS	✓ HTTPS for ingestion and delivery

Implementing access policies for encryption keys in AWS:

Key Policies: The Primary Control:

- Every CMK has a key policy that defines who can use or manage the key.
- Key policies are attached directly to the key and are the primary way to manage access.
- By default, only the AWS account that creates the CMK has full access.

IAM Policies: Identity-Based Access Control

- In addition to key policies, you can use IAM policies to control who can call KMS APIs (like `kms:Encrypt` or `kms:CreateKey`).
- However, even if an IAM policy grants permission, the key policy must also allow the action.
- So it's both key policies and IAM policies working together.

Grants: Temporary, Fine-Grained Access

- You can create grants for temporary or scoped permissions (like a Lambda function using a key just for a session).
- Grants are helpful for dynamic access needs.

Best Practices:

Use least privilege — only grant what's needed.

Keep key policies simple and easy to audit.

Consider using automatic key rotation for long-term security.

Regularly review and update key policies to meet compliance needs.

Topic	AWS Service	Automatic Support?
Key rotation	AWS KMS	✅ Automatic rotation for customer-managed CMKs
Certificate renewal	AWS Certificate Manager	✅ Automatic for ACM-provided certificates; manual for imported ones

AWS Snow Family:

Service	Form Factor	Use Case	Capacity	Notes
AWS Snowcone	Small, portable device	Edge computing and small data migrations (e.g., sensors, rugged environments)	8 TB usable storage	Weighs 4.5 lbs; can run EC2 instances and IoT applications; supports offline and online transfers.
AWS Snowball Edge Storage Optimized	Rugged, suitcase-sized device	Large-scale data migrations and edge computing with more storage capacity	80 TB usable storage	Provides local storage and compute capabilities; supports EC2 and Lambda functions at the edge.
AWS Snowball Edge Compute Optimized	Rugged, suitcase-sized device	Edge computing with more compute power (e.g., analytics, ML inference)	42 TB usable storage + GPU options	Includes up to 52 vCPUs and optional GPU (NVIDIA Tesla V100); supports EC2 and Lambda.
AWS Snowmobile	Truck-sized shipping container	Massive data migrations (exabyte-scale), e.g., data center decommissioning	100 PB per Snowmobile	Physically transported in a 45-foot container with security; requires AWS assistance for deployment.

AWS Lambda:

✅ **Serverless Compute**

- AWS Lambda lets you run code without provisioning or managing servers.
- You only pay for what you use: billed based on number of requests and duration.

✓ Event-Driven

- Lambda functions automatically trigger in response to events from services like S3 (object uploads), DynamoDB (streams), API Gateway (HTTP requests), SNS (notifications), SQS (queue messages), and CloudWatch Events (schedules).

✓ Scaling & Availability

- Lambda automatically scales with incoming events — each request can invoke a new instance.
- It handles high availability and fault tolerance automatically.

✓ Execution Time & Resources

- Maximum execution timeout is **15 minutes**.
- Memory allocation ranges from **128 MB to 10 GB** (in 1 MB increments).
- Includes ephemeral `/tmp` disk storage of **512 MB** per execution
- No persistent storage (use S3, DynamoDB, etc. for persistence).

✓ IAM Role (Execution Role)

- Lambda assumes an **IAM execution role** to access other AWS resources securely.

✓ Deployment

- You can deploy as a **.zip file** or **container image** (up to 10 GB).
- Supports languages like Node.js, Python, Java, Go, Ruby, and .NET, or custom runtimes.

✓ Monitoring & Logging

- Lambda automatically integrates with **CloudWatch Logs** for logging function output and errors.
- Use CloudWatch metrics and X-Ray for tracing and performance analysis.

✓ VPC Integration

- Lambda functions can be configured to run inside a VPC for secure access to private resources (like RDS databases).

✓ Concurrency & Throttling

- Default concurrency limit per region (can be adjusted with **Reserved Concurrency**).

- Throttles and returns errors if concurrency limit is reached.

Auto Scaling

- AWS Lambda **automatically scales** the number of function instances in response to incoming requests.
- When an event occurs (like an API call, file upload to S3, or DynamoDB update), Lambda **instantiates a new container** to process that event.
- **No manual scaling configuration** needed — Lambda automatically scales **horizontally** to handle parallel executions.
- If there are thousands of events, Lambda can create thousands of instances **concurrently** (up to your account's concurrency limits).

High Availability

- Lambda is **natively highly available**. It runs your functions in multiple **Availability Zones (AZs)** within a region.
- If there's a failure in one AZ, your function still executes in another AZ seamlessly.
- No need for load balancers or replication setups — Lambda **handles it all** automatically.
- Your functions remain available as long as **AWS region** itself is operational.

AWS CloudTrail:

AWS CloudTrail is a **service that records AWS API calls** for your account. It **tracks who did what** in your AWS environment.

- **Tracks API activity** (AWS Management Console, SDKs, CLI, etc.).
- Records:
 - **Who** made the call (IAM user/role).
 - **When** the call was made.
 - **What** resources were affected.
 - **Where** the call originated (IP address).
- **Event history:** By default, you can view 90 days of API activity in the CloudTrail console.
- **Trails:**
 - **Single-region trail** – captures activity in one region.
 - **Multi-region trail** – captures activity in all regions.
 - Stores events in **S3 buckets** (encrypted).
- **Integration with CloudWatch Logs** for real-time alerts and metric filters.
- CloudTrail is **enabled by default** – but you must create a trail to **store logs in S3** long-term.

- CloudTrail logs are **encrypted** with S3 SSE by default, and you can use KMS for additional control.
- **Insights** (CloudTrail Insights) help detect unusual API activity (like spikes in resource creation).
- Works across **multiple accounts** (via AWS Organizations integration).

AWS Organizations:

AWS Organizations lets you **centrally manage multiple AWS accounts**. It's useful for consolidating billing, applying policies, and structuring accounts hierarchically.

✓ Account Management

- Create **Organizational Units (OUs)** to group accounts (e.g., dev, test, prod).
- Create new AWS accounts **programmatically** (great for scaling environments).
- Consolidate **billing** across accounts (all charges go to the management account).

✓ Service Control Policies (SCPs)

- Apply **policies** to OUs or accounts to set **maximum permissions** (like guardrails).
- SCPs **don't grant permissions** — they only **restrict them** (IAM policies still define actual permissions).

✓ Consolidated Billing

- Single bill for all accounts.
- Share volume discounts and savings plans across accounts.

✓ Integration

- Works with AWS Control Tower, AWS SSO, and other AWS management services.

Service Control Policies:

Service Control Policies are a type of policy in AWS Organizations that define the **maximum set of permissions** for accounts or Organizational Units (OUs). Think of them as “**guardrails**” — they set boundaries for what resources and actions can be used in an account.

✓ SCPs do NOT grant permissions

- They only **restrict** what IAM policies can grant.
- The **effective permissions** are the **intersection** of IAM policies and SCPs.

✓ Apply to everyone in the target account

- SCPs apply to **root users, IAM users, and roles** in member accounts.
- Even the **root user** of a member account is bound by SCPs.

✓ Default Behavior

- **FullAWSAccess SCP** is attached by default — it allows all AWS services/actions unless restricted.

✓ Targeted Control

- You can attach SCPs to:
 - **Individual accounts**
 - **Organizational Units (OUs)**
 - **Root of the organization**

✓ Explicit Deny

- SCPs can explicitly **deny** actions regardless of what IAM policies allow (like IAM's `Deny`).

✓ Inheritance

- SCPs attached to an OU **apply to all child OUs and accounts**.

AWS Global Accelerator:

AWS Global Accelerator is a **networking service** that improves the **availability and performance** of your applications for **global users** by leveraging the **AWS global network**.

✓ Anycast IP Addresses

- Provides you with **two static IP addresses** (or bring your own).
- These IPs are **fixed entry points** to your application, no matter where your users are.
- Traffic is routed to the nearest healthy AWS endpoint.

✓ Global Network

- Uses the **AWS global backbone** to route traffic — reducing latency and improving performance.

✓ Health Checks & Failover

- Monitors endpoints (e.g., ALB, NLB, EC2).
- If an endpoint fails health checks, Global Accelerator automatically **fails over** to the next closest healthy endpoint.

✓ Traffic Distribution

- You can **weight** or **prioritize** traffic to endpoints.
- Example: 80% of traffic to a primary region, 20% to a backup region.

✓ Integrated with AWS Resources

- Works with **Elastic Load Balancers (ALB, NLB)**, EC2 instances, or Elastic IPs.
- Global Accelerator is **not a CDN** — it doesn't cache content (that's what CloudFront does!).
- It **optimizes network paths**, not content.
- Charges are **based on data transfer** and **accelerator hours**.
- Endpoints can be in **multiple AWS regions**.

Placement Groups:

Placement groups let you **control how EC2 instances are placed on underlying hardware** to meet specific performance or fault-tolerance needs.

Types of Placement Groups

✓ 1 Cluster Placement Group

- Packs instances **close together within a single AZ**.
- Low latency and **high network throughput** between instances.
- Best for **HPC (High Performance Computing)**, tightly-coupled apps, or apps that need **10 Gbps+** speeds.
- ⚠ Limited by AZ capacity — may fail to launch if there's not enough capacity.

✓ 2 Spread Placement Group

- Spreads instances **across distinct hardware** in the same AZ or region (1 instance per rack).
- Best for **small number of critical instances** that must **not share** underlying hardware.
- ⚠ Maximum of **7 running instances per AZ** in the group.

✓ 3 Partition Placement Group

- Spreads instances across **logical partitions** — each partition uses separate racks with independent power/network.
- Great for **large, distributed systems** like HDFS, Kafka, Cassandra.
- Can have **many instances**, unlike spread groups.

Disaster Recovery Methods:

✓ 1 Backup and Restore

- **Most cost-effective, but slowest to recover.**
- Regularly back up data (e.g., S3, EBS Snapshots, RDS snapshots).
- In a disaster, **restore data** and deploy resources as needed.
- **RTO**: hours or more.
- **Use case**: Small business with less critical workloads.

✓ 2 Pilot Light

- Minimal version of the environment is always running (like a pilot flame).
- Core components (e.g., database) are running, but other components (like web servers) are turned off.
- In disaster, quickly **scale up** to full production.
- **RTO**: minutes to hours.
- **Use case**: Critical data with less critical app tier.

✓ 3 Warm Standby

- A scaled-down **fully functional** copy of the environment is always running.
- In a disaster, **scale up resources** to handle full production load.
- **RTO**: minutes.
- **Use case**: Apps needing faster recovery than pilot light.

✓ 4 Multi-Site (Active-Active)

- Fully active **two or more environments** in different AWS Regions/AZs.
- Traffic is load balanced across regions.
- **RTO**: near-zero, almost instant.
- **Use case**: Mission-critical apps (like e-commerce or global SaaS).

EC2 Hibernate:

✓ EC2 Hibernate **pauses** your instance and saves its **in-memory state (RAM)** to the **root EBS volume**.

✓ When you start it again, the instance **resumes from where it left off**

S3 Object Lock:

With S3 Object Lock, you can store objects using a write-once-read-many (WORM) model. Object Lock can help prevent objects from being deleted or overwritten for a fixed amount of time or indefinitely. You can use Object Lock to help meet regulatory requirements that require WORM storage or to simply add another layer of protection against object changes and deletion.

Before you lock any objects, you have to enable a bucket to use S3 Object Lock. You enable Object Lock when you create a bucket. After you enable Object Lock on a bucket, you can lock objects in that bucket. When you create a bucket with Object Lock enabled, you can't disable Object Lock or suspend versioning for that bucket.

S3 Object Lock provides two retention modes:

-Governance mode

-Compliance mode

These retention modes apply different levels of protection to your objects. You can apply either retention mode to any object version that is protected by Object Lock.

The screenshot shows the AWS Management Console interface for editing Object Lock settings. The left sidebar shows the 'Amazon S3' navigation menu. The main content area is titled 'Edit Object Lock' and includes a warning box stating that once Object Lock is enabled, it cannot be disabled or suspended. Below this, the 'Object Lock' status is 'Enabled'. The 'Default retention' is set to 'Enable'. The 'Default retention mode' is set to 'Governance', which is highlighted by a green box. A green circle with a line points to this box, containing the text 'S3 Object Lock Retention Modes'. The 'Default retention period' is set to '365 Days'. At the bottom, there are 'Cancel' and 'Save changes' buttons.

In governance mode, users can't overwrite or delete an object version or alter its lock settings unless they have special permissions. With governance mode, you protect objects against being deleted by most users, but you can still grant some users permission to alter the retention settings or delete the object if necessary. You can also use governance mode to test retention-period settings before creating a compliance-mode retention period.

In compliance mode, a protected object version can't be overwritten or deleted by any user, including the root user in your AWS account. When an object is locked in compliance mode, its retention mode can't be changed, and its retention period can't be shortened. Compliance mode helps ensure that an object version can't be overwritten or deleted for the duration of the retention period.

To override or remove governance-mode retention settings, a user must have the `s3:BypassGovernanceRetention` permission and must explicitly include `x-amz-bypass-governance-retention:true` as a request header with any request that requires overriding governance mode.

Legal Hold vs. Retention Period

With Object Lock, you can also place a legal hold on an object version. Like a retention period, a legal hold prevents an object version from being overwritten or deleted. However, a legal hold doesn't have an associated retention period and remains in effect until removed. Legal holds can be freely placed and removed by any user who has the `s3:PutObjectLegalHold` permission.

Legal holds are independent from retention periods. As long as the bucket that contains the object has Object Lock enabled, you can place and remove legal holds regardless of whether the specified object version has a retention period set. Placing a legal hold on an object version doesn't affect the retention mode or retention period for that object version.

For example, suppose that you place a legal hold on an object version while the object version is also protected by a retention period. If the retention period expires, the object doesn't lose its WORM protection. Rather, the legal hold continues to protect the object until an authorized user explicitly removes it. Similarly, if you remove a legal hold while an object version has a retention period in effect, the object version remains protected until the retention period expires.

DVA-C02:

For Cloud Guru - Skip IAM, EC2.

Exam Domains:

Development with AWS Services

Security

Deployment

Troubleshooting and Optimization

S3 Encryption:

Encryption in Transit - SSL/TLS, HTTPS.

Encryption at Rest - SSE-S3(default), SSE-KMS, SSE-C.

Client Side Encryption.

Amazon Athena:

Amazon Athena is a **serverless, interactive query service** that lets you run **SQL queries directly on data stored in Amazon S3** — no database to manage or ETL process required.

- You write queries using **standard SQL (Presto/Trino)**.
- Athena scans your S3 data directly and returns results in seconds.
- You **pay only for the data scanned**.

Concept	What You Need to Know for the Exam
Purpose	Query structured or semi-structured data (CSV, JSON, Parquet, ORC, etc.) stored in S3.
Serverless	No provisioning, scaling, or maintenance required.
Data Catalog	Uses AWS Glue Data Catalog to define schema and tables.
Integration	Works with S3, Glue, QuickSight, and Lambda.
Cost model	Pay per query based on bytes scanned. Compressing or using columnar formats reduces cost.
Security	IAM for permissions, S3 bucket policies for access, and encryption support (KMS).

Amazon Kinesis:

Amazon Kinesis is a suite of services that let you **collect, process, and analyze streaming data in real time**. It's ideal when data is **continuously produced** — such as application logs, IoT sensor data, or clickstreams — and you need to react immediately.

Service	Purpose	Typical Use Case
Kinesis Data Streams (KDS)	Low-level stream service for custom real-time processing	Build custom consumer apps or Lambda consumers that process data record-by-record
Kinesis Data Firehose	Fully managed delivery stream (no consumer code)	Automatically load streaming data into S3, Redshift, OpenSearch, or custom HTTP endpoints
Kinesis Data Analytics (KDA)	Run SQL queries on streaming data	Filter, aggregate, or transform data before storing it
Kinesis Video Streams	Stream video and audio data	Used for real-time video processing and analytics

Amazon OpenSearch Service:

Amazon OpenSearch Service (formerly **Amazon Elasticsearch Service**) is a **fully managed search and analytics engine** used to **index, search, and visualize data in near real time**. It's built on **OpenSearch / Elasticsearch** and is often paired with **Kibana or OpenSearch Dashboards** for visualization.

Use Case	Example
Full-text search	Website or app search bars (e.g., “find products by name, description, tags”)
Log & metrics analytics	Centralized analysis of CloudWatch, VPC, or application logs
Real-time monitoring dashboards	Ops dashboards with visualizations of system health, error rates, etc.

Perfect — this is one of the **most important sections** for the **AWS Developer Associate (DVA-C02)** exam, because it directly tests your understanding of **CI/CD on AWS** using **CodeCommit, CodeBuild, CodeDeploy, and CodePipeline** — collectively called the **AWS Developer Tools suite**.

Let’s break it down **exactly how the exam expects you to know it** 📌

🧩 Overview Summary

Service	Purpose	What It Replaces / Comparable To
CodeCommit	Private Git-based source control	GitHub, GitLab, Bitbucket
CodeBuild	Build and test source code in a fully managed environment	Jenkins build agents, GitHub Actions runners
CodeDeploy	Deploy apps automatically to EC2, Lambda, or on-prem servers	Jenkins deploy jobs, Ansible
CodePipeline	Orchestrate the entire CI/CD workflow	Jenkins pipeline, GitLab CI/CD

⚙️ 1. AWS CodeCommit

Purpose

A **fully managed Git repository service** that stores source code securely in AWS.

Key Points

- Supports standard Git commands (push, pull, clone).

- Integrated with **IAM** for authentication and authorization.
- Repositories are encrypted at rest and in transit.
- Works seamlessly with CodePipeline and CodeBuild as a CI/CD source stage.
- Can be accessed via:
 - SSH keys
 - HTTPS Git credentials for IAM users
 - Federated access (Cognito or SSO)

Exam Tips

- If a question mentions **source control inside AWS** → **CodeCommit**.
- If developers must **avoid hardcoding credentials** → use **IAM roles or federated access**.
- Supports **repository triggers** (SNS, Lambda, or CloudWatch Events) on push/merge events.

2. AWS CodeBuild

Purpose

A **fully managed build service** that compiles source code, runs tests, and produces artifacts.

Key Concepts

Concept	Description
buildspec.yml	YAML file in your repo that defines the build commands and environment variables.
Environment	Managed build environment (Docker container) — choose runtime, OS, and compute size.
Artifacts	Build outputs stored in S3 or passed to CodePipeline.
Caching	Reuses dependencies (npm, Maven, etc.) to speed up builds.
Reports	Test results can be exported to S3 or viewed in the console.

Typical buildspec.yml

```
version: 0.2
phases:
  install:
    commands:
```

```
- npm install
build:
  commands:
    - npm run build
artifacts:
  files:
    - '**/*'
base-directory: dist
```

Exam Tips

- **Fully serverless:** no build servers to manage.
- Can **pull code from CodeCommit, GitHub, S3, or Bitbucket.**
- Can **push artifacts to S3 or CodeDeploy.**
- You pay per build minute (no idle cost).
- Common IAM issue: CodeBuild needs permissions to read the source and write artifacts.
- Use **environment variables** to inject secrets (via Parameter Store or Secrets Manager).

3. AWS CodeDeploy

Purpose

Automates **application deployments** to:

- **EC2 instances**
- **On-premises servers**
- **Lambda functions**
- **ECS services**

Deployment Types

Type	Description	Use Case
In-place (rolling)	Updates existing EC2 or on-prem instances directly	Long-running, stateful apps
Blue/Green	Creates a new environment (new instances, new Lambda version, or ECS task set) and switches traffic gradually	Safer, minimal downtime deployment

Application Types

Platform	Deployment Behavior
EC2 / On-Prem	Uses the CodeDeploy agent and appspec.yml file
Lambda	Shifts traffic between Lambda versions/aliases
ECS	Updates ECS task definitions and handles traffic shifting via load balancer

AppSpec File Examples

EC2 (appspec.yml)

```
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/html
hooks:
  AfterInstall:
    - location: scripts/restart.sh
      timeout: 300
      runas: root
```

Lambda (appspec.yml)

```
version: 0.0
Resources:
  - myLambdaFunction:
      Type: AWS::Lambda::Function
      Properties:
        Name: "MyFunction"
        Alias: "live"
        CurrentVersion: "1"
        TargetVersion: "2"
```

Deployment Configs

Config	Description
AllAtOnce	Deploy to all instances at once (fast but risky)
HalfAtATime	Deploy to 50% at a time
OneAtATime	Deploy sequentially (safe but slow)

Exam Tips

- **Lambda + traffic shifting (alias-based deployments)** = important DVA topic.
 - Understand **hooks lifecycle** (BeforeInstall, AfterInstall, ApplicationStart, etc.).
 - If you see “roll back automatically if health check fails” → CodeDeploy can do this automatically.
 - Blue/Green deployments integrate with **ALB/NLB** for traffic routing.
-

4. AWS CodePipeline

Purpose

A fully managed **CI/CD orchestration service** that automates build, test, and deploy stages.

Pipeline Structure

Stage	Example Service Used
Source	CodeCommit, GitHub, or S3
Build	CodeBuild
Test (optional)	CodeBuild, Lambda, or manual approval
Deploy	CodeDeploy, ECS, CloudFormation, or Elastic Beanstalk

Example Flow:

```
[CodeCommit]
  ↓
[CodeBuild]
  ↓
[CodeDeploy]
  ↓
[Production]
```

Integrations

- Can integrate with **third-party sources** (GitHub, Bitbucket).
- Can use **manual approval actions**.
- Supports **CloudWatch Events** and **SNS notifications**.
- Can run **parallel actions** or **conditional branches**.

Exam Tips

- CodePipeline **does not host code or build** — it orchestrates other AWS services.
- If a question says “*automate from commit to deploy across multiple stages*” → CodePipeline.
- Common exam pattern:
 - CodeCommit → CodeBuild → CodeDeploy → Lambda/EC2/ECS
- Can use **Artifacts S3 buckets** between stages.
- Each stage runs in sequence and can **retry automatically** on transient failures.

5. Common Exam Scenarios

Scenario	Correct AWS Tool
“Need private Git repo in AWS”	CodeCommit
“Need to compile, test, and package app automatically”	CodeBuild
“Need to deploy app to EC2 or Lambda with rollback support”	CodeDeploy
“Need to automate CI/CD workflow across multiple stages”	CodePipeline

Scenario	Correct AWS Tool
“Need to trigger builds on code push”	CodePipeline + CodeCommit trigger
“Need blue/green deployment for Lambda”	CodeDeploy + Lambda alias routing

💡 6. IAM & Security Notes

- Each service has a **service role**:
 - AWSCodeCommitServiceRole
 - AWSCodeBuildServiceRole
 - AWSCodeDeployRole
 - AWSCodePipelineServiceRole
- The **pipeline role** needs permission to call actions in other services.
- Always apply the **least privilege principle**.
- Artifacts between stages are stored in **S3** and must be **encrypted**.

🧩 8. Quick Summary Table

Service	Core Role	Exam Keyword Clues
CodeCommit	Source control (Git)	“Store code in AWS,” “trigger pipeline on push”
CodeBuild	Compile & test code	“buildspec.yml,” “run tests,” “create artifacts”
CodeDeploy	Automated deployments	“Lambda alias,” “Blue/Green,” “EC2 rollout”
CodePipeline	Orchestrate CI/CD stages	“Automate from commit to deploy,” “multi-stage workflow”

Hook	Purpose	Example
BeforeInstall	Run commands before the new version is copied . Often used to stop services, back up files, or clean directories.	<code>service nginx stop</code>
Install	Copies the new application revision to the target instance. (<i>Handled automatically</i>)	—

Hook	Purpose	Example
AfterInstall	Run after files are copied — for configuration, setting file permissions, or dependency installation.	<code>npm install</code> , <code>chmod 755</code>
ApplicationStart	Start the application or service with the new version.	<code>systemctl start myapp</code>
ValidateService	Final test to verify deployment success (health checks, curl endpoints).	<code>curl http://localhost/health</code>

Method	Description	Pros	Cons
In-Place (Rolling) Deployment	Updates the existing instances in batches with the new version. The CodeDeploy agent installs the new app, stops the old one, and starts the new one on the same host.	Simple, fast setup.	Some downtime possible; if deployment fails, old code might be partially replaced.
Blue/Green Deployment	Launches a new set of instances (green) with the new version while keeping the old ones (blue) running. After testing, CodeDeploy switches traffic (via Load Balancer) to the green environment.	Zero downtime, quick rollback.	Uses more resources (two environments).

Method	Description	Notes
Canary	Shifts a small percentage of traffic (e.g., 10%) to the new version for a specified time, then shifts 100% if no issues.	Safer rollout, minimal impact if failure.
Linear	Gradually increases traffic by fixed percentages (e.g., +10% every 10 minutes).	Useful for gradual confidence buildup.
All-at-Once	Instantly shifts all traffic to the new version.	Fastest but riskiest.

Method	Description	Notes
Blue/Green (ECS)	Creates a new task set (green) in the ECS service, tests it, and then reroutes traffic from the old task set (blue) using the load balancer.	Supports automatic rollback if health checks fail.

Method	Rollback Mechanism
In-Place (Rolling)	CodeDeploy re-installs the previous app revision on the same instances if a deployment fails (using the last successful revision).
Blue/Green	CodeDeploy simply redirects traffic back to the “blue” (old) environment using the load balancer. The failed “green” instances are terminated.

Method	Rollback Mechanism
Canary / Linear / All-at-Once	CodeDeploy re-points the alias to the previous Lambda version if alarms or validation hooks fail. No code is redeployed — just alias redirection.

Method	Rollback Mechanism
Blue/Green	If health checks fail, CodeDeploy switches traffic back to the previous ECS task set (blue) and terminates the failed one (green).

Amazon EventBridge

Amazon EventBridge is a **serverless event bus** that lets you build **event-driven applications** by connecting event sources (like AWS services, SaaS apps, or your own apps) to event targets (like Lambda, Step Functions, SNS, SQS, etc.) — **without writing glue code**.

Concept	Description
Event Bus	A stream of events — can be default , partner (SaaS) , or custom .
Event Source	Who generates the event (e.g., AWS service, SaaS app, or your app).
Event Rule	Filters and routes events based on pattern matching (JSON rules).
Event Target	The service that receives the event (Lambda, SQS, Step Functions, SNS, etc.).
Schema Registry	Stores event schemas so developers can generate strongly typed code bindings.

Type	Purpose
Default Event Bus	Every AWS account automatically has one — all AWS service events go here.
Custom Event Bus	You create these for your own apps and microservices. Your app can publish custom events to it.

Type	Purpose
Partner Event Bus	Receives events from SaaS providers integrated with AWS (like Zendesk, Datadog, Shopify).

Target	Typical Use
AWS Lambda	Run custom code when an event occurs
Step Functions	Start workflows
SNS / SQS	Fan-out or queue events for async processing
API Gateway / EventBridge Pipe	Forward events to APIs or Kinesis
Kinesis Data Streams / Firehose	Real-time streaming pipelines
EC2 Systems Manager / ECS Tasks	Automate operational tasks

Scenario	Service Combo
Trigger a Lambda when an S3 object is created	S3 → EventBridge → Lambda
Start a Step Function when a CloudWatch alarm changes	CloudWatch → EventBridge → Step Functions
Notify a Slack app when a user signs up	App → EventBridge custom bus → Partner target (e.g., Slack webhook)
Route SaaS app events to your system	Partner Event Bus → Lambda or SQS
Periodic job (like a cron)	Use EventBridge Scheduler (replaces CloudWatch Scheduled Events)

Amazon SQS:

Amazon SQS is a **fully managed message queuing service** that lets you **decouple microservices** or distributed systems.

It acts as a **buffer** between producers (senders) and consumers (receivers) — ensuring messages are **durable**, **reliable**, and **asynchronous**.

Concept	Description
Producer	Sends messages to the queue.
Queue	Stores messages temporarily until they're processed.
Consumer	Retrieves and processes messages, then deletes them.
Message	Up to 256 KB payload (can reference S3 for larger data).

Concept	Description
Retention period	1 minute to 14 days (default: 4 days).

Type	Description	Use Case
Standard Queue	Unlimited throughput; at-least-once delivery (can deliver duplicates); messages may be out of order.	High throughput tasks (e.g., processing logs, jobs, analytics).
FIFO Queue	First-In-First-Out order; exactly-once processing; limited throughput (3,000 msg/s with batching, 300 msg/s without).	Financial transactions, order systems, workflows where order matters.

- **Producer sends message** → stored redundantly across multiple AZs.
- **Consumer receives message** → message becomes *invisible* for a time (visibility timeout).
- **Consumer processes it.**
- **Consumer deletes message** → only then is it permanently removed.

Setting	Description	Default
Visibility Timeout	Time a message stays “hidden” after being received (prevents duplicate processing).	30 sec
Message Retention	How long messages stay in queue before deletion.	4 days
Delivery Delay	Delay new messages becoming visible.	0–15 min
Maximum Message Size	256 KB (or larger if using S3 pointer).	—
Batching	Send/receive/delete up to 10 messages at once (reduces cost).	—

A **Dead-Letter Queue** stores messages that **can’t be processed successfully** after a set number of receives (maxReceiveCount).

- Each source queue can target **one DLQ**.
- Commonly used for debugging and auditing failed events.

Concept	Description
MessageGroup	Ensures ordered delivery for that group (like “user-123” or “order-456”).

Concept	Description
MessageDeduplicationId	Prevents duplicate message delivery within a 5-minute deduplication window.
Content-Based Deduplication	If enabled, SQS uses SHA-256 hash of message body as the deduplication ID.

Feature	Description
IAM Policies	Control who can <code>SendMessage</code> , <code>ReceiveMessage</code> , <code>DeleteMessage</code> .
SSE (Server-Side Encryption)	Encrypt messages at rest using SSE-SQS (AWS-managed KMS key) or a CMK .
KMS Integration	Per-queue KMS key for compliance/security.
VPC Endpoint	Access queues privately without public internet.

Scenario	Correct Approach
“Need to decouple producer and consumer”	Use SQS
“Messages appear multiple times”	Standard queue’s at-least-once delivery
“Preserve order”	FIFO queue
“Reduce cost / empty receives”	Enable Long Polling
“Consumer fails multiple times”	Use DLQ
“Retry but prevent duplicate processing”	Use visibility timeout correctly
“Serverless event-driven processing”	SQS → Lambda trigger
“Encryption required”	Enable SSE-SQS with KMS

Amazon SNS:

Amazon Simple Notification Service (SNS) is a **fully managed pub/sub (publish-subscribe)** messaging service that enables **event-driven communication** between **publishers** and **subscribers**.

Concept	Description
Topic	A communication channel for a specific event or data type. Publishers send messages to a topic.
Publisher	The component or app that sends a message to a topic.

Concept	Description
Subscriber	The endpoint that receives the messages (SQS, Lambda, email, SMS, HTTP, etc.).
Subscription	The link between a topic and a subscriber endpoint.
Message	The payload sent by a publisher (max size: 256 KB).

Protocol	Description
SQS	Sends message to a queue for decoupled, reliable processing.
Lambda	Triggers a Lambda function automatically.
HTTP / HTTPS	Sends POST request to a webhook or API endpoint.
Email / Email-JSON	Sends email notifications.
SMS	Sends text messages to mobile devices.
Mobile Push (Platform Apps)	Pushes to Apple, Google, etc. via platform endpoints.
Kinesis Data Firehose (<i>new</i>)	Send streaming data to analytics destinations.

Use Case	Example
Fan-out messaging	1 event → multiple SQS queues and Lambdas.
Push notifications	Send alerts to users or admins.
Decoupling services	Producer doesn't know who the consumers are.
Mobile apps	Push messages via APNs (Apple) or FCM (Google).
Cross-region messaging	Global notification delivery.

SNS supports **message filtering** by using **subscription filter policies** (JSON-based). It allows **subscribers to receive only certain messages** based on message attributes.

Mechanism	Description
Retries	SNS automatically retries failed deliveries (HTTP, Lambda, SQS).
Backoff	Retries use exponential backoff (like SQS).
DLQ for Lambda	For failed Lambda deliveries, messages can go to a DLQ (SQS or SNS).
Durability	Messages stored redundantly across multiple AZs.

Feature	Description
FIFO SNS Topic	Preserves order of messages for subscribers (must use FIFO SQS or Lambda).
Exactly-once delivery	No duplicates.
Naming rule	Ends with <code>.fifo</code>
Use Case	Ordered processing pipelines (financial transactions, logs).

Scenario	Correct Solution
“Send message to multiple systems at once”	SNS topic (fan-out)
“Only send some messages to specific subscribers”	SNS filter policies
“Decouple microservices asynchronously”	SNS → SQS
“Trigger Lambda when message arrives”	SNS → Lambda
“Cross-account publish”	Topic policy
“Encrypt messages”	KMS integration
“Send push notifications to mobile devices”	SNS Mobile Push
“Maintain message order”	FIFO SNS topic
“Avoid message loss for failed delivery”	DLQ for Lambda / retry policy

AWS Step Functions:

AWS Step Functions is a **serverless orchestration service** that lets you **coordinate multiple AWS services or Lambda functions** into **workflows (state machines)** — without writing complex workflow logic in code.

Concept	Description
State Machine	The entire workflow — defined in Amazon States Language (ASL) (JSON-based).
State	Each step in the workflow (task, choice, wait, etc.).
Execution	A single run of the workflow.
Input/Output	Each state takes JSON input, performs actions, and passes output to the next.
Task	A single work unit — can invoke a Lambda, ECS task, Glue job, etc.

State Type	Description	Example Use
Task	Runs a single unit of work (e.g., Lambda, ECS task, or API call).	Call a Lambda to validate order
Choice	Conditional branching based on input values.	If amount > 1000 → manager approval
Parallel	Run multiple branches concurrently.	Process images + update DB simultaneously
Wait	Pause execution for seconds or until timestamp.	Wait 5 minutes before next step
Pass	Passes input to output unchanged (for testing/debug).	Placeholder or mock
Fail	Stops execution and marks it failed.	Validation error
Succeed	Marks successful completion.	Final step
Map	Iterates over an array of items (like a loop).	Process each record in a dataset

Type	Description	Use Case
Standard Workflow	Long-running workflows (up to 1 year)	Order processing, data pipelines
Express Workflow	High-volume, short-lived workflows (up to 5 minutes)	Event-driven tasks, real-time analytics

Feature	Standard	Express
Duration	Up to 1 year	Up to 5 minutes
Execution Start Rate	~2,000/s	>100,000/s
Pricing	Per state transition	Per duration + invocations
Visibility	Step-by-step logging	Aggregate metrics only
Use Case	Reliable, auditable workflows	High-throughput, transient events

Field	Purpose
InputPath	Filter or select part of input passed to the state.
ResultPath	Where to insert the task's result in the JSON.
OutputPath	What data to pass to the next state.

Feature	Description	Exam Focus
Workflow Type	Standard (long-running) / Express (high-volume)	Choose correct type
States	Task, Choice, Parallel, Wait, Map, Pass, Fail, Succeed	Know usage
Service Integrations	Call AWS services directly (Lambda optional)	"No glue code"
Error Handling	Built-in retry, catch, timeout	Retry with exponential backoff
Security	IAM roles per state machine	"Access denied" → missing permissions
Input/Output	JSON transformations	InputPath, OutputPath, ResultPath
Cost	Pay per state transition (Standard)	Pricing question
Max Duration	1 year (Standard), 5 minutes (Express)	Duration limits

Amazon ElastiCache:

Amazon ElastiCache is a **fully managed, in-memory data store and cache** service that improves application performance by **retrieving data from memory instead of disk**.

It supports two open-source engines:

-  **Redis**
-  **Memcached**

Benefit	Description
Low latency	Microsecond response times for frequent reads.
Reduced DB load	Offload read traffic from RDS / DynamoDB.
Scalable	Easy to scale horizontally or vertically.
Session storage	Store user session data across web servers.
Pub/Sub & Leaderboards	Real-time messaging, gaming leaderboards, rate limiting.

If question mentions *"persistence, replication, or pub/sub"* → **Redis**

If it says *"simple, multi-node, scale-out cache"* → **Memcached**

Use Case	Example
Database query caching	Cache frequent SQL queries to reduce RDS load.
Session storage	Web sessions shared across EC2 instances.
Leaderboard or ranking	Store top player scores (Redis sorted sets).
Pub/Sub messaging	Real-time notifications in gaming/chat apps.
Rate limiting	API throttling with Redis INCR/EXPIRE.
Temporary token store	Store OAuth tokens or JWTs.

Redis supports **read replicas** for scaling reads.

Memcached supports **multiple nodes** with **client-side sharding**.

Feature	Description
Snapshots	Periodic backups to S3 for disaster recovery.
Restore	Create a new cluster from a snapshot.
Automatic failover	Primary fails → replica promoted automatically (Multi-AZ).

Memcached has no persistence — restart wipes all data.

Strategy	Description	Example
Lazy Loading (Cache-Aside)	Load into cache on first request (most common).	If cache miss → fetch → store.
Write-Through	Write to cache + DB simultaneously.	Immediate cache consistency.
Write-Behind (Write-Back)	Write to cache first → async write to DB later.	Fast writes, but risk data loss.
TTL (Time-to-Live)	Expire cache items automatically.	Prevent stale data.

If you see *“cache stale data issue”* → set **TTL** or **invalidate cache** on updates.

Scenario	Correct Solution
“Reduce read load on RDS”	ElastiCache
“Maintain session state across EC2 instances”	Redis
“Need persistent cache across restarts”	Redis with snapshots

Scenario	Correct Solution
“Simple distributed cache, no replication”	Memcached
“Cluster must support pub/sub”	Redis
“Application needs caching for DynamoDB”	DAX, not ElastiCache
“High read throughput, unordered data”	Memcached
“Multi-AZ, automatic failover”	Redis replication group
“Reduce API response time”	Cache responses in Redis

Amazon RDS:

Amazon RDS (Relational Database Service) is a **fully managed relational database service** that handles provisioning, patching, backups, replication, and scaling — so developers can focus on their app logic instead of database administration.

Feature	Description
Automated Backups	Daily snapshots + transaction logs; point-in-time recovery.
Manual Snapshots	User-initiated backups retained indefinitely.
Multi-AZ Deployments	Automatic failover to standby in another AZ.
Read Replicas	Offload read traffic; up to 15 for Aurora, 5 for other engines.
Storage Auto-Scaling	Grows storage up to max size automatically.
Monitoring	CloudWatch, Enhanced Monitoring, Performance Insights.
Maintenance Window	Periodic patching/upgrades by AWS.

Type	Use Case
General Purpose (gp3/gp2)	Balanced price/performance (default).
Provisioned IOPS (io1/io2)	Consistent high I/O for heavy workloads.
Magnetic (deprecated)	Legacy. Avoid in new setups.

Feature	Multi-AZ	Read Replica
Purpose	High availability	Performance (read scaling)
Sync/Async	Synchronous	Asynchronous
Use For	Failover	Read scaling
Can Promote?	Yes, auto on failure	Yes, manual

Feature	Multi-AZ	Read Replica
Region	Same region	Same or cross-region
DNS endpoint	Single endpoint	Separate endpoint per replica

RDS Proxy manages **database connection pooling** between your app (especially Lambda) and RDS.

Benefit	Description
Connection Pooling	Reuses connections to avoid “too many connections” errors.
Failover Handling	Keeps connections alive during Multi-AZ failover.
IAM Auth Support	No password in code.
Improves scalability	Especially for serverless apps (Lambda, Fargate).

Type	Description
Automated Backups	Daily + point-in-time recovery (retention: 1–35 days).
Manual Snapshot	Created manually; retained until deleted.
Restore	Create a new DB instance from snapshot.

- In **Multi-AZ**, AWS automatically:
 - Promotes standby to primary.
 - Updates DNS endpoint.
 - Restores sync replication later.
- Failover typically completes in **30–120 seconds**.

Feature	Description
Parameter Group	Controls DB engine settings (e.g., <code>max_connections</code>).
Option Group	Adds specific features (e.g., Oracle TDE, SQL Server Agent).

Scenario	Correct Solution
“Lambda times out connecting to RDS”	Add RDS Proxy or increase timeout.
“Need automatic failover”	Enable Multi-AZ .
“Need read scaling”	Use Read Replicas .
“Need restore to yesterday 2 PM”	Point-in-Time Recovery (PITR) .

Scenario	Correct Solution
“Reduce read latency”	Add ElastiCache .
“Encrypt RDS data”	Enable KMS encryption .
“Access RDS from private VPC”	Use VPC Endpoint + Security Group rules .
“Reduce database connections from Lambda”	RDS Proxy .
“MySQL + performance + auto-scaling”	Aurora MySQL .

If the question says “*high performance, fully managed, MySQL-compatible*” → **Aurora**.

If question says “*handle read-heavy traffic without downtime*” → **Read Replicas**.

If it says “*automatic scaling of read capacity*” → **Aurora Auto Scaling**.

If you see “*automatically fail over*” → **Multi-AZ**.

If you see “*improve read throughput*” → **Read Replica**.

If the question says “*Lambda functions occasionally time out due to RDS connection limits*” → use **RDS Proxy**.

If Lambda needs to access RDS securely → **RDS Proxy + IAM authentication**.

Common question: modifying DB behavior (like enabling slow query logs) → **use Parameter Group**.

“Restore to a specific time yesterday” → **Automated Backup (PITR)**

“Clone environment for testing” → **Manual Snapshot** → **Restore new instance**

AWS X-Ray:

AWS X-Ray is a **distributed tracing and analysis service** that helps developers **debug and monitor production, distributed, and serverless applications** — from request to database query.

It lets you:

- Trace requests as they travel through services.
- See where **latency, errors, and throttling** occur.
- Understand **dependencies** across microservices.

Feature	Description
Request Tracing	Tracks requests across services (API Gateway → Lambda → DynamoDB → etc.)
Performance Analysis	Identifies slow components (“Which API call is causing latency?”)
Error/Exception Analysis	Shows which segments failed and why.

Feature	Description
Service Map	Visual representation of your application's components and their connections.

- **X-Ray SDK / Agent** captures trace data (called **segments**) from your application.
- The **X-Ray Daemon** (or embedded agent in Lambda) buffers trace data and sends it to the **X-Ray service**.
- The **X-Ray Console** visualizes traces, latency, and errors.

Term	Description
Trace	The complete path of a single request across services.
Segment	Data about one component (e.g., a Lambda function, EC2 instance).
Subsegment	Granular trace inside a segment (e.g., DB call inside Lambda).
Annotation	Custom indexed metadata (e.g., <code>userId</code> , <code>region</code>) for filtering.
Metadata	Non-indexed data for debugging (e.g., payloads).
Sampling	Controls what % of requests are traced to reduce overhead/cost.
Service Map	Visual graph showing all connected components.

X-Ray doesn't trace *every* request by default — instead, it uses **sampling**.

Default Rule

- 1 request per second (baseline)
- Plus 5% of additional requests

Feature	Description
IAM Permissions	Use <code>AWSXrayWriteOnlyAccess</code> for agents; <code>AWSXrayReadOnlyAccess</code> for viewing traces.
Encryption	Data encrypted at rest (KMS optional).
Service Role	Lambda or EC2 instance must have <code>xray:PutTraceSegments</code> and <code>xray:PutTelemetryRecords</code> .

Scenario	Correct Service
"Trace end-to-end request latency across multiple microservices"	AWS X-Ray
"Lambda and API Gateway integration with tracing"	Enable Active Tracing

Scenario	Correct Service
"Add custom business metadata to traces"	Annotations
"View dependency graph of your services"	Service Map
"Reduce tracing overhead in production"	Sampling Rules
"No traces visible from EC2 app"	Missing X-Ray Daemon or IAM permissions
"Compare average latency by API"	X-Ray console traces

If a question says *"trace the path of a request through multiple AWS services"* → it's **AWS X-Ray** (not CloudWatch Logs).

If question says *"trace requests through Lambda and API Gateway without changing code"* → enable **Active Tracing**.

captureAWS() wraps SDK clients so all AWS SDK calls get traced automatically.

"Which service shows how a request travels through microservices?" → **X-Ray**

"Which shows CPU and memory usage?" → **CloudWatch Metrics**

Common cause of missing traces → **missing IAM permissions** for the X-Ray agent.

If question says *"trace a user request through multiple AWS services"* → X-Ray.

If it says *"monitor CPU/memory"* → CloudWatch.

If it says *"audit who made API call"* → CloudTrail.

AWS CloudTrail:

Amazon CloudTrail is a **governance, compliance, and auditing service** that records **every API call and console action** made in your AWS account.

CloudTrail captures **API activity** across **all AWS services**, including:

- **Console actions** (e.g., launching an EC2 instance)
- **AWS SDK / CLI / API calls**
- **IAM user or role actions**
- **Assumed roles, STS tokens**
- **Service-to-service actions** (e.g., Lambda calling DynamoDB)
- **Events triggered by AWS services**

Each event includes:

Field	Description
EventTime	When it happened
EventName	API action (e.g., CreateBucket)
UserIdentity	Who made the call

Field	Description
SourceIPAddress	Where it came from
Resources	Which AWS resources were affected
ErrorCode	If it failed

Event Type	Description
Management Events	Control plane actions — managing resources (e.g., <code>RunInstances</code> , <code>CreateUser</code>).
Data Events	Data plane actions — reading/writing data (e.g., <code>GetObject</code> in S3, <code>PutItem</code> in DynamoDB).
Insight Events	Detect unusual API activity (e.g., sudden surge in <code>AuthorizeSecurityGroupIngress</code>).

By default, only **management events** are recorded.

You must **enable data & insight events** manually.

CloudTrail logs can be:

- Stored in **S3 buckets** (default destination)
- Viewed in **CloudTrail console** → **Event history (last 90 days)**
- Sent to **CloudWatch Logs** for real-time alerts
- Delivered to **EventBridge** for automation

Type	Description	Scope
Event History (default)	Stores past 90 days of management events	Per region
One Trail (configured)	Continuous logging to S3	Regional or multi-region
Organization Trail	Single trail across multiple AWS accounts (via AWS Organizations)	Entire org

Feature	Description
IAM Permissions	Control who can create or view trails.
S3 Encryption	Logs encrypted (AES-256 or KMS).
Log File Integrity Validation	Detects if logs were modified or deleted.

Feature	Description
CloudTrail Logs Access Policy	S3 bucket must allow CloudTrail service principal (<code>cloudtrail.amazonaws.com</code>) to write.

CloudTrail **Insights** uses machine learning to detect unusual activity:

- Spikes in API calls
- Unusual IP addresses
- Policy changes
- **Organization Trail** collects logs for *all* accounts under AWS Organizations.
- Delivered to a **central S3 bucket** in the management account.
- Reduces cost and simplifies compliance.

Scenario	Correct Service / Feature
"Who deleted an S3 bucket?"	CloudTrail
"Detect abnormal spikes in API calls"	CloudTrail Insights
"Trigger alert when security group changes"	CloudTrail → CloudWatch → SNS
"Audit SDK/CLI/API usage"	CloudTrail
"View last 90 days of activity"	Event History
"Store all logs indefinitely"	Trail with S3 destination
"Single view across multiple accounts"	Organization Trail
"Ensure log integrity"	Log File Integrity Validation
"Query API events with SQL"	Athena on CloudTrail S3 logs

If you see *"identify who deleted a resource"* → **CloudTrail**.

If question says *"persist API logs for compliance"* → **CloudTrail + S3**.

If you see *"ensure CloudTrail logs cannot be tampered with"* → **Enable log file integrity validation**.

If question says *"alert when a user deletes an EC2 instance"* → **CloudTrail → CloudWatch Logs → SNS**

If question says *"detect abnormal behavior or API spikes"* → **CloudTrail Insights**.

AWS CDK:

AWS Cloud Development Kit (CDK) is an **open-source framework** that lets you **define your AWS infrastructure using programming languages** (like TypeScript, Python, Java, or C#) instead of writing raw YAML or JSON CloudFormation templates.

CDK converts your code into CloudFormation templates automatically and deploys them through AWS CloudFormation.

- **You write code** in your preferred language using CDK constructs.
- Run `cdk synth` → CDK **synthesizes** CloudFormation templates.
- Run `cdk deploy` → CDK **deploys** the stack via CloudFormation.
- You get real AWS resources created and managed.

Concept	Description
App	The root container for one or more stacks.
Stack	Equivalent to one CloudFormation stack (a deployable unit).
Construct	Reusable building block that defines one or more AWS resources.
Stage	Collection of stacks deployed together (used in pipelines).
Context	Runtime configuration data passed via <code>cdk.json</code> or CLI.

Command	Description
<code>cdk init app --language typescript</code>	Create a new CDK project
<code>cdk synth</code>	Generate (synthesize) CloudFormation template
<code>cdk diff</code>	Show differences between deployed and local stacks
<code>cdk deploy</code>	Deploy stack via CloudFormation
<code>cdk destroy</code>	Delete deployed stack
<code>cdk bootstrap</code>	Prepares your account for CDK (creates S3 buckets, roles)
<code>cdk list</code>	List all stacks in the app

- CDK uses **your AWS credentials (via AWS CLI or environment)**.
- **Each stack deployment** is handled by CloudFormation with its own IAM role.
- You can specify roles in CDK using properties like `role`, `policy`, `grantReadWrite()` etc.
- CDK enforces **least privilege** when creating service roles.

You can integrate CDK with **AWS CodePipeline** using **CDK Pipelines**

Every construct has a **logical ID** in CloudFormation.

- CDK **generates logical IDs** automatically.
- You can override them if needed.
- CDK manages **dependency ordering** automatically.

Scenario	Correct Solution
“Provision AWS infra using familiar languages”	AWS CDK
“Generate CloudFormation templates automatically”	<code>cdk synth</code>
“Automate stack deployment in pipeline”	CDK Pipelines
“Package and deploy Lambda code automatically”	CDK Assets
“Use pre-built constructs for patterns”	L3 Constructs
“Deploy stack fails due to missing resources”	Run <code>cdk bootstrap</code>
“View differences before deploying”	<code>cdk diff</code>
“Multiple environments with shared resources”	Use separate Stacks
“Track configuration with IaC”	CDK (under the hood: CloudFormation)

If you see “provision AWS resources using familiar programming languages” → **AWS CDK**.

If you see “generate CloudFormation templates automatically” → `cdk synth`

If it says “prepare environment before first deployment” → `cdk bootstrap` If question says

“deployment failed: missing bootstrap resources” → run `cdk bootstrap` .

If you see “*automate deployment of CDK apps through CI/CD*” → **CDK Pipelines**.

If the question says “*resource deployment order automatically managed*” → **CDK via CloudFormation** ensures it.

AWS CloudFront:

Amazon CloudFront is AWS’s **Content Delivery Network (CDN)** that delivers web content (HTML, CSS, JS, APIs, media, etc.) to users **with low latency and high transfer speeds** via **AWS edge locations** worldwide.

It caches content closer to the users, reducing the load on your origin servers (S3, EC2, ALB, etc.).

- User requests content (e.g., image, video, API).
- CloudFront checks local edge cache:
 - If **cached**, content is returned instantly.
 - If **not cached**, it retrieves from **origin** and stores a copy.
- Subsequent users near that location get **faster delivery**.

Origin Type	Example Use Case
Amazon S3 Bucket	Static website, images, downloads

Origin Type	Example Use Case
Application Load Balancer / EC2	Dynamic web apps
API Gateway / Lambda	APIs, serverless backends
Media Store / MediaPackage	Video streaming
Custom Origin (any HTTP endpoint)	Third-party or on-prem server

Term	Description
Distribution	The CloudFront configuration for your content delivery.
Origin	The backend source of truth (S3, ALB, etc.).
Edge Location	Data center where CloudFront caches content.
Cache Behavior	Rules controlling how CloudFront handles requests (TTL, query strings, headers).
TTL (Time To Live)	How long objects stay cached before revalidation.
Invalidation	Manual cache refresh for changed files.

Feature	Description
Caching	Stores content at edges for faster access.
Compression	Gzip/Brotli reduces bandwidth usage.
Lambda@Edge	Run code at edge locations (request/response manipulation).
Origin Shield	Adds an extra caching layer to reduce origin load.
HTTP/3 + QUIC	Faster connection protocol (modern browsers).

Feature	Description
HTTPS / TLS	CloudFront enforces HTTPS using ACM certificates.
AWS WAF Integration	Protect against SQL injection, XSS, bots.
Origin Access Control (OAC)	Securely connect CloudFront to S3 (replaces OAI).
Geo Restriction	Block or allow countries.
Signed URLs / Cookies	Restrict access to paid/private content.
Field-Level Encryption	Encrypt sensitive data before sending to origin.

Use Case	Signed URL	Signed Cookie
Access control for one object	✓	✗

Use Case	Signed URL	Signed Cookie
Access control for many objects	✗	✓
Used for private streaming/video downloads	✓	✓
Authentication context	Provided via query params or cookies	Same, but cookie-based

When you update your files in origin, cached copies might be stale.

You can **invalidate** objects manually:

```
aws cloudfront create-invalidation --distribution-id E123ABC456 --paths "/*"
```

- ✓ Each invalidation path removes content from edge caches immediately.
- ✓ Costs apply per invalidation (free for first 1,000 paths per month).

When using S3 as an origin:

- Disable **public access** to the S3 bucket.
- Use **Origin Access Control (OAC)** to grant CloudFront access only.
- S3 objects can remain private.

Type	Description
At Rest	S3-managed keys (SSE-S3) or KMS (SSE-KMS).
In Transit	CloudFront → HTTPS → Origin.
Field-Level Encryption	Encrypt form fields (e.g., credit card number) before reaching origin.

Lambda@Edge lets you run Lambda functions at CloudFront edge locations — **without managing servers**.

- Charged for:
 - Data transfer out to the internet.
 - HTTP/HTTPS requests.
 - Invalidation requests (first 1,000 free).
- Data transfer **into** CloudFront is free.
- Cheaper per GB than direct S3 public delivery.

CloudFront supports **primary & secondary origins** — if the primary fails, traffic automatically fails over.

Scenario	Correct Feature
"Speed up content delivery to global users"	CloudFront
"Serve private files securely"	Signed URLs or cookies
"Restrict S3 access only via CloudFront"	Origin Access Control (OAC)
"Users getting old data"	Create invalidation
"Add headers or auth logic at edge"	Lambda@Edge
"Restrict content by geography"	Geo Restriction
"Block malicious requests"	AWS WAF + CloudFront
"Improve API latency for global clients"	API Gateway + CloudFront
"Detect high 5xx errors"	CloudWatch Metrics
"Failover to backup origin"	Origin Groups

If you see *"deliver static and dynamic content globally"* → **CloudFront**.

If *"serve S3 files with lower latency"* → **CloudFront + S3 origin**.

If question says *"modify headers or URLs at the edge before reaching origin"* → use **Lambda@Edge**.

If you see *"restrict access to S3 content through CloudFront only"* → **Origin Access Control (OAC)** or **Origin Access Identity (OAI)** (older).

"Serve premium videos securely to authenticated users" → **CloudFront signed URLs/cookies**.

If a user sees outdated data → issue a **CloudFront invalidation**.

If question says *"users must not access S3 directly"* → enable **OAC** and block public access.

If question says *"customize requests globally with minimal latency"* → **Lambda@Edge**.

"Reduce S3 egress costs while improving performance" → **Serve via CloudFront**.

If question says *"automatically failover to backup S3 bucket if main origin fails"* → **Origin Group configuration**.

Amazon Route 53:

Amazon Route 53 is AWS's **highly available and scalable DNS (Domain Name System) service** that connects user requests to resources like **EC2, Load Balancers, S3, CloudFront**, or even **external domains**.

It handles:

-  **Domain registration**
-  **DNS resolution (name → IP)**
-  **Health checks & failover routing**

Function	Description
Domain Registration	Buy and manage domain names directly from AWS.
DNS Routing	Translate domain names into IP addresses.
Health Checks & Failover	Automatically redirect traffic away from unhealthy endpoints.
Traffic Routing Policies	Control <i>how</i> traffic is routed across multiple endpoints.

A **Hosted Zone** is a **container for DNS records** for a domain.

Type	Description
Public Hosted Zone	Routes traffic on the Internet (e.g., <code>myapp.com</code>).
Private Hosted Zone	Routes traffic <i>within a VPC</i> (e.g., <code>internal.myapp.local</code>).

Record Type	Purpose	Example
A	Maps hostname → IPv4 address	<code>api.myapp.com</code> → <code>192.0.2.1</code>
AAAA	Maps hostname → IPv6 address	<code>api.myapp.com</code> → <code>2001:db8::1</code>
CNAME	Alias to another domain	<code>images.myapp.com</code> → <code>assets.mycdn.com</code>
Alias Record	AWS-specific; maps to AWS resources	<code>www.myapp.com</code> → CloudFront / ELB / S3
MX	Mail server routing	For email setup
TXT	Metadata, SPF, DKIM, domain verification	
NS	Nameservers for hosted zone	
SOA	Authoritative info for zone (default)	

Alias vs CNAME:

- **Alias** = AWS-specific, **no extra DNS lookup**, can point to AWS resources (CloudFront, ELB, S3).
- **CNAME** = Standard DNS, can't use on root domain (`myapp.com`).

Policy Type	Description	Use Case
Simple Routing	One record, one target	<code>api.myapp.com</code> → EC2

Policy Type	Description	Use Case
Weighted Routing	Distribute traffic by percentage	A/B testing
Latency-Based Routing	Send user to region with lowest latency	Global apps
Failover Routing	Primary + Secondary; switches when health check fails	HA setup
Geolocation Routing	Route by user's geographic location	Country-based content
Geoproximity Routing	Route by user proximity <i>and bias</i> (Route 53 Traffic Flow)	Fine-tuned regional control
Multi-Value Answer	Return multiple healthy IPs (like simple + health checks)	Load balancing without ELB

Route 53 can monitor the **health and availability** of your endpoints.

Type	Description
Endpoint Health Check	Pings a URL or IP on HTTP/HTTPS/TCP.
Calculated Health Check	Combines results of multiple checks (e.g., "all must pass").
CloudWatch Alarm Health Check	Uses a CloudWatch alarm's status.

TTL defines how long DNS records are cached by resolvers.

Value	Meaning
Short TTL (60 s)	Quick updates, higher DNS traffic
Long TTL (1 hr+)	Slower updates, fewer lookups

Route 53 resolver handles **DNS resolution between on-prem and VPCs**.

Feature	Description
Inbound Endpoints	Allow on-prem DNS servers to resolve AWS private zones.
Outbound Endpoints	Allow VPCs to resolve on-prem DNS records.

Scenario	Correct Answer
"Distribute traffic globally based on latency"	Latency-based routing
"Split traffic 80/20 between endpoints"	Weighted routing
"Switch to backup region when health check fails"	Failover routing
"Direct users in US to one region, EU to another"	Geolocation routing
"Serve domain name within VPC only"	Private hosted zone
"Point root domain to CloudFront"	Alias record
"Detect if endpoint unhealthy"	Health check
"Hybrid DNS resolution"	Route 53 Resolver
"Audit DNS config changes"	CloudTrail

If you see *"route user traffic globally based on latency or health"* → **Route 53**.

If question says *"resolve domain names within a VPC only"* → **Private Hosted Zone**.

- *"Split traffic 70/30 between two regions"* → **Weighted**.
- *"Users in Asia get fastest response"* → **Latency-based**.
- *"Route to backup site on failure"* → **Failover**.
- *"Direct users by country"* → **Geolocation**.

If question says *"automatically route traffic away from unhealthy servers"* → **Route 53 Health Check + Failover**.

If it says *"map root domain to CloudFront distribution"* → **Alias record** (CNAME not allowed at root).

If "DNS updates take too long to propagate," shorten the **TTL**.

If question mentions *"prevent DNS spoofing or tampering"* → **DNSSEC**.

If question says *"hybrid architecture with on-prem DNS and VPC name resolution"* → **Route 53 Resolver**.

If question says *"audit DNS record changes"* → **CloudTrail**.

If it says *"track DNS queries and responses"* → **Route 53 Resolver Query Logging**.

AWS KMS:

AWS Key Management Service (KMS) is a **fully managed encryption service** that lets you **create, manage, and control encryption keys** used to protect your data across AWS.

Use Case	Example
Encrypt data at rest	S3, EBS, RDS, DynamoDB tables
Encrypt data in transit	TLS certificates + signed requests

Use Case	Example
Application-level encryption	Encrypt secrets before storing in DB
Envelope encryption	Use a data key to encrypt large data blobs
Cross-account encryption	Share CMKs securely between accounts

Type	Managed By	Description
AWS-Managed CMK	AWS	Created automatically by AWS (e.g., <code>/aws/s3</code> , <code>/aws/rds</code>)
Customer-Managed CMK	You	Full control: enable/disable, rotation, policy, alias
AWS-Owned Key	AWS internal	Used for internal encryption (not visible to you)
Data Key	Application	Used for envelope encryption; generated by CMK temporarily

Concept	Description
CMK (Customer Master Key)	Main KMS key used to encrypt/decrypt data keys.
Key ID / Alias	Identifier or friendly name for the CMK.
Key Policy	Resource-based policy defining <i>who can use or manage</i> the key.
Grants	Temporary permissions for other accounts/services to use a key.
Automatic Key Rotation	Every 1 year (for customer-managed keys).
Envelope Encryption	Encrypt data with a <i>data key</i> , and encrypt that data key with a CMK.

Large data isn't encrypted directly with a CMK because it's slow and costly.

✓ Process:

1. KMS generates a **data key**.
2. App uses it to encrypt data locally.
3. The **plaintext data key** is deleted; only the **encrypted version** (by CMK) is stored.
4. To decrypt, the app sends the encrypted key to KMS to retrieve plaintext.

KMS API Operations

Operation	Description
CreateKey	Create a new CMK.
Encrypt	Encrypt plaintext data using CMK.
Decrypt	Decrypt ciphertext using CMK.
GenerateDataKey	Generate a data key for envelope encryption.
ReEncrypt	Re-encrypt data under a new key.
ScheduleKeyDeletion	Delete CMK after 7–30 days (irreversible).
ListKeys / DescribeKey	Retrieve metadata about keys.

Method	Description	Uses KMS?
SSE-S3	S3-managed encryption	✗
SSE-KMS	S3 encryption using KMS CMK	✓
SSE-C	Customer provides key (not KMS)	✗
CSE (Client-Side Encryption)	Encrypt before uploading	Optional ✓

Type	Scope	Controls
Key Policy	Resource-based	Who can use/manage a CMK
IAM Policy	User-based	Which KMS actions the user can perform
Grants	Temporary delegation	Share access with other AWS accounts or roles

Automatic Key Rotation

- Available for **customer-managed CMKs only**.
- Rotates keys **once per year** automatically.
- AWS keeps older versions for decrypting older data.
- AWS-managed CMKs are rotated automatically (every 3 years, managed by AWS).

Action	Behavior
Disable Key	Key unusable but not deleted (can be re-enabled).
ScheduleKeyDeletion	Key permanently deleted after waiting period (7–30 days).
Immediate Delete	✗ Not allowed — waiting period is mandatory.

To allow another account to use your CMK:

1. Update the **key policy** with their account or role ARN.
2. Optionally create a **grant** for temporary access.

Limit	Value
Encrypt/Decrypt payload size	Up to 4 KB (use envelope encryption for larger)
GenerateDataKey	Use for larger data encryption
Rotation	1 year (customer CMK)
API throughput	~10,000 requests/sec per account (soft limit)

Scenario	Correct Solution
"Encrypt data at rest in S3 with auditable key usage"	SSE-KMS
"Lambda needs to decrypt secrets stored in S3"	KMS Decrypt API
"Rotate encryption keys annually"	Enable key rotation
"Share key across AWS accounts"	Key policy + Grant
"Encrypt large data efficiently"	Envelope encryption
"Prevent key deletion"	Disable key instead of deleting
"Audit all encryption operations"	CloudTrail logs
"Cross-region key replication"	Multi-Region CMK
"Temporary allow cross-account key use"	KMS Grant

If the question mentions *"encrypt data using a managed key service integrated with AWS"* → **KMS**.

If question says "developer needs to control who can use and rotate keys" → **Customer-Managed CMK**.

If the question mentions "encrypt large data efficiently using KMS" → **Envelope encryption**.

If you see "encrypt large dataset with KMS efficiently" → use `GenerateDataKey`, not `Encrypt`.

If question says "data should be encrypted at rest using a key controlled by customer" → **SSE-KMS with customer-managed CMK**.

If you see *"need to audit key usage"* → use **SSE-KMS** (since it logs in CloudTrail).

If the question says "temporarily allow a Lambda function in another account to use a key" → **KMS Grant**.

If "enable annual key rotation with no code changes" → **Enable key rotation** on CMK.

If question says *"prevent accidental data loss"* → **disable key** instead of deleting it.

If question says *"audit all encryption key usage for compliance"* → **enable CloudTrail for KMS**.

"Another AWS account needs to decrypt data with your CMK" → Add that account's role/user to **key policy**.

“Encrypt data in multiple AWS regions with the same key material” → **Multi-Region CMK**.

If question says “*encrypt 10 MB file directly with KMS*” → wrong.

Use **data key + envelope encryption** instead.

Amazon Cloudwatch:

Amazon CloudWatch is AWS’s **monitoring and observability service** for:

- **Metrics** (numerical data over time — CPU, memory, latency, etc.)
- **Logs** (system and app log data)
- **Alarms** (automated responses to thresholds)
- **Dashboards** (visual monitoring)
- **Events / Rules** (trigger automated actions)

Component	Purpose
Metrics	Quantitative data points (CPUUtilization, Invocations, etc.)
Logs	Text data from apps or AWS services
Alarms	Trigger actions when metrics exceed thresholds
Dashboards	Visualize metrics and logs
Events / EventBridge	React to state changes and automate workflows
Contributor Insights	Identify top “contributors” to issues or spikes
Anomaly Detection	ML-based pattern monitoring for metrics

Source	Example Metrics
EC2	CPUUtilization, NetworkIn, DiskReadOps
RDS	FreeStorageSpace, DBConnections
Lambda	Invocations, Duration, Errors, Throttles
API Gateway	4XXError, 5XXError, Latency
S3	NumberOfObjects, BucketSizeBytes
Custom (Your App)	OrdersProcessed , ActiveUsers , etc.

Namespaces

Each service or app stores metrics under a **namespace**.

Metrics can have **dimensions** (metadata labels)

CloudWatch Logs stores and manages **text-based log data** from AWS services and apps.

Concept	Description
Log Group	Container for related logs (e.g. <code>/aws/lambda/myFunction</code>)
Log Stream	Sequence of logs from a specific instance/process
Retention Policy	How long to keep logs (default = forever)
Metric Filters	Extract numeric values from logs (convert logs → metrics)

Alarms **watch a metric** and perform an **action** when a threshold is crossed.

State	Meaning
OK	Metric within normal range
ALARM	Threshold breached
INSUFFICIENT_DATA	Not enough data yet

Alarm Action	Example
SNS Notification	Email/SMS alert
Auto Scaling	Increase/decrease EC2 count
EC2 Action	Stop / terminate instance
Systems Manager Automation	Run remediation script

- ✅ **Composite Alarms:** Combine multiple alarms (e.g., CPU > 80% *and* Disk > 90%).
- ✅ **Anomaly Detection:** ML-based alarms automatically detect unusual metric behavior.

Cloudwatch Dashboard:

- Customizable **visual panels** showing metrics & alarms.
- Can include multiple services (EC2, Lambda, RDS, etc.).
- Share dashboards across accounts / via URL.
- JSON-defined → easy to version-control.

Cloudwatch Logs Insights - A **query language** for searching and analyzing logs quickly. Great for debugging Lambda, API Gateway, or EC2 logs.

Cloudwatch Agents:

To collect **OS-level metrics** (CPU, memory, disk usage) from EC2, on-prem, or containers:

Agent	Use Case
CloudWatch Agent	Custom metrics & logs from EC2/on-prem
Unified Agent (new)	Combines metrics + logs
ECS Container Insights	Collects container-level metrics

You can publish your own application metrics with the **PutMetricData** API:

```
aws cloudwatch put-metric-data \ --namespace "MyApp" \ --metric-name "ActiveUsers" \ --value 120 \ --unit Count
```

Scenario	Correct Answer
“Trigger Lambda when EC2 stops”	CloudWatch Event Rule
“Send alert when CPU > 80%”	CloudWatch Alarm
“Aggregate logs from multiple Lambdas”	CloudWatch Logs
“Search logs for errors”	CloudWatch Logs Insights
“Visualize app metrics”	CloudWatch Dashboard
“Detect sudden spike in requests”	Anomaly Detection
“Collect memory metrics on EC2”	Install CloudWatch Agent
“Automate scaling when load high”	Alarm + Auto Scaling
“Monitor API Gateway latency”	CloudWatch Metrics
“Audit API calls”	CloudTrail (not CloudWatch)

If you see *“monitor performance and trigger alerts automatically”* → **CloudWatch**.

If it says *“trace request path”* → **X-Ray** (different service).

If you see *“custom application metrics”* → use **PutMetricData** API.

“Monitor logs in near real time” → **CloudWatch Logs + Subscription Filters (to Lambda or Kinesis)**

“Automatically stop instance when idle CPU for 30 minutes” → CloudWatch Alarm + EC2 Action

If question says *“create unified monitoring view for multiple services”* → CloudWatch Dashboard.

If question says *“search specific error messages in Lambda logs”* → CloudWatch Logs Insights.

If **CloudWatch Logs not receiving data** → check IAM permissions + agent config.

If question says *“app emits custom performance metrics”* → CloudWatch Custom Metrics.

If **cost optimization** asked → **reduce** log retention **or** metric resolution**.

Metrics are **numerical data points** (like CPUUtilization, Duration, etc.), and their timing depends on the **source service** and **monitoring mode**:

Monitoring Type	Frequency (Granularity)	Notes
Basic Monitoring	Every 5 minutes	Default for EC2, RDS, etc.
Detailed Monitoring	Every 1 minute	Must be enabled (extra cost)
High-Resolution Custom Metrics	As low as 1 second	Only for <i>custom metrics</i> (developer sends via SDK/API)

- AWS services (like EC2, RDS, Lambda) typically publish metrics every **1 or 5 minutes**.
- You **can** push **custom metrics** with **1-second granularity** using the `PutMetricData` API.

Logs are **text-based data** (e.g., Lambda output, EC2 app logs, API Gateway logs).

Source	Delivery Frequency	Notes
Lambda	Real-time (a few seconds delay)	Automatically pushed after each invocation
EC2 (CloudWatch Agent)	Every 5–60 seconds , configurable	Depends on agent config
ECS / EKS (Container Insights)	~10–30 seconds delay	Batched for efficiency
CloudTrail / VPC Flow Logs	Every few minutes	Aggregated periodically
Custom Apps	You control frequency	Depends on when you push logs

✅ Logs are **streamed continuously**, not every second exactly — but usually **visible within seconds to tens of seconds** after generation.

- **Metrics:** Usually every **1–5 minutes**, unless you push **1-second custom metrics**.
- **Logs:** Streamed in near-real-time, but **not guaranteed every second** — they're batched for efficiency.

AWS CLI:

AWS Command Line Interface (CLI) is a **unified tool** to interact with **all AWS services** directly from your terminal or scripts.



Basic Command Syntax

bash

Copy code

```
aws <service> <operation> [parameters] [--options]
```

✓ Examples:

bash

Copy code

```
aws s3 mb s3://my-bucket          # make bucket
aws ec2 describe-instances        # list EC2s
aws dynamodb put-item --table-name MyTable --item '{"id": {"S": "1"}}'
aws lambda update-function-code --function-name myFunc --zip-file fileb://func.zip
```

Many services return paginated results.

Option	Description
<code>--max-items</code>	Limit total items
<code>--page-size</code>	Number of items per API call
<code>--starting-token</code>	Continue from previous page

Feature	Purpose
Pagination	Handle large result sets (list responses).
Waiters	Poll for resource state changes before proceeding.

Service	Example Command	Purpose
S3	<code>aws s3 cp file.txt s3://mybucket/</code>	Upload file
Lambda	<code>aws lambda invoke --function-name MyFunc out.json</code>	Run Lambda
CloudFormation	<code>aws cloudformation deploy --template-file stack.yaml --stack-name MyStack</code>	Deploy IaC
DynamoDB	<code>aws dynamodb scan --table-name Users</code>	Query table
SQS	<code>aws sqs send-message --queue-url URL --message-body "hello"</code>	Send message
SNS	<code>aws sns publish --topic-arn ARN --message "Hello!"</code>	Publish topic
CloudWatch	<code>aws cloudwatch put-metric-data --metric-name latency --value 1.3</code>	Custom metric

Service	Example Command	Purpose
ECR	<code>aws ecr get-login-password</code>	Authenticate Docker push
IAM	<code>aws iam list-users</code>	Manage users

Category	Feature	Exam Keyword
Tool	Command-line AWS interface	“Automate AWS tasks”
Auth Methods	IAM user, role, SSO, env vars, profiles	“Avoid hardcoded creds”
Output	JSON, YAML, Text, Table	“Readable results”
Region Control	<code>--region</code> , <code>AWS_DEFAULT_REGION</code>	“Deploy in us-east-1”
Profiles	Multi-account config	“Switch envs”
Waiters	Wait until state changes	“Instance running before next step”
Query Filters	JMESPath	“Extract field from output”
Scripting	Bash / PowerShell	“CI/CD automation”
Error Handling	IAM perms, region mismatch	“AccessDenied errors”
Integration	Works with SDKs, CodeBuild, pipelines	“Automation-ready”

If question says “*switch between multiple accounts or regions easily*” → use **profiles** (`AWS_PROFILE`).

If question says “*script should wait until resource is ready before proceeding*” → **CLI Waiter**.

If question says “*extract specific field from JSON output*” → use `--query` **option**.

If question says “*developer needs to access multiple AWS accounts with CLI*” → **profiles + role_arn**.

If a CLI command fails with “*Access Denied*” → missing **IAM permission** or wrong **profile**.

If question says “*deploy code to Lambda from pipeline*” → use **AWS CLI command**.

If CLI command fails silently, check:

- Credentials/profile
- Region mismatch
- IAM permissions

If question says “*developer script should not store credentials*” → use **IAM Role** or **AWS SSO**.

If question says “*automate actions using scripts*” → **CLI**

If it says “*integrate AWS operations in your app code*” → **SDK**

Amazon API Gateway:

Amazon API Gateway is a **fully managed service** that allows you to **create, publish, secure, monitor, and maintain APIs** at any scale.

It acts as a **front door** for applications to access **backend services**, such as:

- AWS Lambda functions
- EC2 instances
- AWS services (DynamoDB, S3, Step Functions, etc.)
- External HTTP endpoints

API Type	Description	Typical Use Case
REST API (Classic)	Full-featured API with caching, mapping, throttling, etc.	Legacy APIs, custom authorization
HTTP API (v2)	Faster, cheaper, simpler, fewer features	Modern serverless APIs with Lambda
WebSocket API	Persistent, bidirectional communication	Chat apps, dashboards, live games

Concept	Description
Endpoint	The API’s public URL (regional, edge-optimized, or private).
Resource	Path segment (<code>/users</code> , <code>/orders</code>).
Method	HTTP verb (GET, POST, PUT, DELETE, etc.).
Integration	Backend target (Lambda, HTTP, AWS Service).
Stage	Deployed version of your API (<code>/dev</code> , <code>/prod</code>).
Deployment	Snapshot of API configuration pushed to a stage.
Mapping Templates	Transform request/response payloads (using VTL).
Usage Plans / API Keys	Control access and rate limits per client.

Type	Description	Example
Lambda Integration	Invokes AWS Lambda function	<code>myFunc</code>
HTTP Integration	Proxies requests to external URL	<code>https://mybackend.com/api</code>
Mock Integration	Returns static responses (no backend)	Testing / prototyping
AWS Service Integration	Directly call AWS services via IAM	e.g., <code>DynamoDB.PutItem</code>

Auth Type	Description	Common Use Case
IAM	SigV4-signed requests (AWS users/roles)	Internal AWS-to-AWS API calls
Lambda Authorizer	Custom token validation via Lambda	Custom JWT or API key logic
Cognito Authorizer	Integrates with Cognito User Pool	User login for web/mobile apps
None (Public)	No auth	Public endpoints

Endpoint Types

Type	Description
Regional	Accessed directly in region; best for same-region clients.
Edge-Optimized	Uses CloudFront to reduce latency for global users.
Private	Accessible only from inside a VPC via PrivateLink.

API Gateway protects backends with built-in throttling.

Setting	Description
Account-level limit	Default = 10,000 req/sec across all APIs
Stage limit	Applied per stage
Usage plan limit	Applied per API key/client
Burst	Short-term spike allowed beyond steady rate

- API Gateway **can cache responses** to improve performance and reduce backend load.
- Caching is enabled **per stage**.
- You can configure **TTL (default 300 s)** and **invalidate cache manually**.

You can transform requests/responses using:

- **Mapping Templates** (Velocity Template Language – VTL)
- **Body Mapping Expressions**
- **Integration Request / Response**

✅ Example Use Cases:

- Rename JSON fields
- Add new headers
- Filter sensitive info before returning
- CORS enables browser apps from one domain to call APIs in another domain.
- Must return headers:
`Access-Control-Allow-Origin: *` `Access-Control-Allow-Methods: GET, POST`

Concept	Purpose
Integration Response	Defines how API Gateway handles backend response.
Method Response	Defines how API responds to client (status codes, headers, body).

Error Type	Description
4XX	Client-side errors (invalid request, auth fail)
5XX	Backend or integration errors

Feature	Purpose
API Keys	Identify & throttle clients
Usage Plans	Rate limits & quotas
IAM / Cognito / Lambda Authorizers	Control who can call API
Resource Policies	Restrict access by IP, VPC, or account
TLS / Custom Domain Names	Secure endpoint

- Each **Stage** can have its own settings (logging, caching, throttling).
- You can use **Stage Variables** like environment variables.

Caching Layer	Description	Benefit
API Gateway Cache	Stores full responses at edge	Faster client responses
Lambda Caching (Code)	Developer caches data in code	Reduces DB calls per invocation

- **Edge-Optimized APIs** automatically use CloudFront CDN.
- You can also front a **Regional API** with your own CloudFront distribution.

Scenario	Correct Feature
"Trigger Lambda on API call"	Lambda Integration
"Lower cost, no caching needed"	HTTP API
"Global low-latency API"	Edge-Optimized
"Convert JSON request to DynamoDB format"	Mapping Template
"Restrict by user token"	Cognito Authorizer
"Custom token validation logic"	Lambda Authorizer
"Restrict API by IP range"	Resource Policy
"Reduce repeated backend calls"	Enable Caching
"Different settings per environment"	Stages + Stage Variables
"Analyze API errors and latency"	CloudWatch Metrics & Logs

If the question says *"lower latency and lower cost for simple HTTP endpoints"* → **HTTP API (v2)**.

If it says *"API caching, usage plans, API keys"* → **REST API (v1)**.

If question says *"manage multiple environments for same API"* → use **Stages**.

If question says *"build API with no server code"* → **AWS Service Integration (e.g., DynamoDB)**.

- *"Allow only signed AWS users"* → **IAM authorization**
- *"Use JWT tokens from user pool"* → **Cognito Authorizer**
- *"Custom logic for header token"* → **Lambda Authorizer**
- *"Users across the globe"* → **Edge-Optimized**
- *"Internal microservice inside VPC"* → **Private**

If question says *"prevent abuse by limiting requests per client"* → **Usage Plan + API Key**.

If question says *"reduce latency and backend calls for identical requests"* → **Enable API Gateway cache**.

If question says *"convert incoming request JSON to match backend format"* → **Mapping Templates**.

If question says *"developer needs to troubleshoot 502 errors"* → **enable CloudWatch logs**.

If question says *"new version must not affect live users until tested"* → **Deploy to new stage**.

If question says *"browser app cannot call API due to CORS error"* → **Enable CORS on method/stage**.

If question says *"serverless backend with API and no server management"* → **API Gateway + Lambda**.

If question says *"return friendly message to client when backend errors occur"* →

Method/Integration Mapping.

If question says “*restrict API access to specific VPC*” → **Resource Policy**.

If question says “*use different Lambda versions in dev vs prod*” → **Stage Variables + Lambda Aliases**.

If question says “*reduce Lambda invocations for repeated API calls*” → **Enable API Gateway cache**.

If question says “*improve performance for users globally*” → **Edge-Optimized API**.

If question says “*reduce API Gateway cost for simple Lambda API*” → use **HTTP API**.

If question says “*automate deployment of APIs*” → **CLI + create-deployment** or **SAM template**.

AWS Elastic Beanstalk:

Elastic Beanstalk (EB) is a **Platform-as-a-Service (PaaS)** that automatically deploys your **application code** (Node.js, Python, Java, .NET, PHP, Ruby, Go, Docker, etc.) to AWS infrastructure.

Component	Description
Application	Logical container for your app versions and environments.
Application Version	Specific deployable package (ZIP, WAR, Docker image).
Environment	Set of AWS resources running one version of your app.
Environment Tier	Web Server or Worker.
Platform	Preconfigured stack (Node.js, Python, Java, etc.).
Configuration Template	Saved settings for future environments.

Tier	Purpose	Example
Web Server Environment	Handles HTTP requests (via ELB → EC2).	Web apps, APIs
Worker Environment	Handles background jobs (via SQS → EC2).	Email sending, batch jobs

Method	Description	Rollback Behavior
All-at-Once	Deploys new version to all instances simultaneously.	Fast but downtime risk.
Rolling	Updates instances in batches.	Old batch still serves traffic.
Rolling with	Adds new batch before terminating old	No capacity loss.

Method	Description	Rollback Behavior
Additional Batch	ones.	
Immutable	Launches new instances with new version; swaps after success.	Safest; easy rollback.
Blue/Green	Deploys to new environment, swaps CNAME if successful.	Instant rollback (swap back).

Deployment Workflow

- **Developer uploads code** (ZIP, WAR, Docker image)
 - EB stores it in **S3** as an application version
 - EB launches/updates EC2 instances and related resources
 - Health checks and monitoring start automatically
- Elastic Beanstalk lets you customize your environment using:

Method	Description
Configuration Files (<code>.ebextensions</code>)	YAML files in app root (e.g. <code>.ebextensions/options.config</code>)
Console / CLI Settings	Modify environment options directly
Saved Configuration Template	Reuse settings across environments

You can define **environment variables** in:

- AWS Console (Configuration → Software)
- `.ebextensions`
- CLI using `--envvars`

Tool	Purpose
AWS CLI	Full AWS control (<code>aws elasticbeanstalk ...</code>)
EB CLI	Simplified Beanstalk deployment (<code>eb init</code> , <code>eb deploy</code>)

```

eb init # Configure app
eb create # Create environment
eb deploy # Deploy latest version
eb status # Check environment status
eb logs # Retrieve logs
eb terminate # Delete environment

```

EB automatically creates **IAM roles**:

- **Service Role** → Allows EB to manage resources (EC2, ASG, ELB).
- **Instance Profile** → Permissions for EC2 instances (e.g., access S3, CloudWatch).
- Every deployment creates an **Application Version** (stored in S3).
- You can **rollback** to any previous version.
- You can customize EC2 instances or install additional software using `.ebextensions` or **Platform Hooks**.

Scenario	Correct Solution
"Deploy web app without managing infrastructure"	Elastic Beanstalk
"Handle background processing"	Worker environment
"Zero-downtime deployment"	Blue/Green or Immutable
"Customize EC2 instance software"	<code>.ebextensions</code> config
"Scale app automatically"	Auto Scaling settings
"App version rollback"	Redeploy previous version
"Persist DB after termination"	External RDS
"Debug failed environment"	Retrieve logs / check instance health
"Automate deploy via CI/CD"	CodePipeline + Beanstalk
"Restrict instance permissions"	IAM instance profile

If you see *"process background tasks asynchronously"* → **Worker environment**.

If question says *"zero downtime deployment"* → **Blue/Green** or **Immutable**.

If question says *"Beanstalk cannot find app version"* → Check **S3 bucket permissions** or **deleted version**.

If question says *"customize EC2 software packages"* → **use** `.ebextensions`.

If question says *"store API keys or DB credentials in Beanstalk"* → use **Environment Variables**

If question says *"deploy app from local system using command line"* → **EB CLI**.

If "environment health is Degraded" → check **load balancer**, **instance logs**, or **app errors**.

If question says *"Beanstalk can't write logs to CloudWatch"* → **missing permissions on instance profile**.

If question says *"database should persist after environment termination"* → **create RDS separately** (not inside EB environment).

If question says *"revert to previous version quickly after failed deploy"* → **redeploy previous Application Version**.

"Run custom script after app deployment" → use **container_commands**.

Amazon EBS:

Amazon Elastic Block Store (EBS) provides **block-level storage volumes** that attach to **EC2 instances**. It stores data **persistently** (even after instance stop/start) and supports:

- Snapshots (backups)
- Encryption (via KMS)
- Performance tuning (IOPS, throughput)
- Multi-AZ replication (for resilience)

Service	Type	Use Case
EBS	Block Storage	EC2 disks, databases, OS drives
EFS	File Storage	Shared file system across instances
S3	Object Storage	Backup, static files, data lake
Instance Store	Ephemeral Block Storage	Temporary scratch disk (lost on stop/terminate)

Feature	Description
Block Storage	Data stored in fixed-size blocks.
Persistent	Retains data after instance stop/start.
AZ-Scoped	Volumes live in a single Availability Zone.
Attach/Detach	Can move volume between instances (same AZ).
Resize on the fly	Increase size or change type dynamically.

Type	Performance	Ideal Use Case
gp3 (General Purpose SSD)	Balance of price/performance	Default for most workloads
gp2 (Legacy SSD)	Older gen (deprecated)	Historical only
io1 / io2 (Provisioned IOPS SSD)	High IOPS (up to 64K)	Databases needing consistent latency
st1 (Throughput-Optimized HDD)	High throughput, low cost	Big data, log processing
sc1 (Cold HDD)	Lowest cost	Archival, rarely accessed
io2 Block Express	Sub-ms latency, multi-attach	Mission-critical databases

Concept	Definition	Optimized For
IOPS	Input/Output Operations per second	Databases, random reads/writes
Throughput (MB/s)	Amount of data read/written per second	Large sequential workloads

- **Point-in-time backups** of EBS volumes stored in **S3** (incremental).
- Used for **backup**, **replication**, and **AMIs** (machine images).
- Snapshots are **region-wide** and can be copied across regions.

Feature	Description
Default Encryption	Can be enabled at account level.
KMS Integration	Uses AWS KMS CMK to encrypt at rest.
Auto-encryption	Snapshots and new volumes inherit encryption automatically.
In-transit encryption	Handled automatically during data transfer.

Operation	Encryption Result
Snapshot from encrypted volume	Snapshot is encrypted
Volume from encrypted snapshot	Volume is encrypted
Copy encrypted snapshot	Can re-encrypt with new CMK

Operation	Description
Create Volume	Via Console, CLI, SDK, or CloudFormation
Attach Volume	To EC2 instance (within same AZ)
Detach Volume	Gracefully stop instance or unmount filesystem first
Resize Volume	Increase size or change volume type online
Delete Volume	Data lost permanently (unless snapshot exists)

- Automated backups using **Data Lifecycle Manager (DLM)**.
- Snapshots are incremental — only changed blocks are stored.
- You can restore snapshot anytime to a new volume.

Metric	Description
VolumeReadOps / VolumeWriteOps	Number of I/O operations
VolumeQueueLength	Pending I/O requests (performance bottleneck indicator)
VolumeIdleTime	Seconds with no I/O
BurstBalance	Remaining burst credit balance for gp2/gp3

Feature	EBS	Instance Store
Persistence	Persistent	Ephemeral (lost on stop/terminate)
Attachable	Yes (same AZ)	No
Backup	Snapshots	None
Encryption	Supported	Not supported
Performance	Consistent	High, temporary
Use Case	Databases, long-term storage	Cache, temp files

If question says “*persistent storage for EC2*” → **EBS**.

If “*shared across multiple instances*” → **EFS**.

If question says “*copy EBS volume to another AZ*” → take **snapshot**, then **restore** to another AZ.

- “*High IOPS, low latency DB*” → **io1/io2**
- “*Cost-effective general use*” → **gp3**
- “*Throughput-heavy logs*” → **st1**

If question says “*optimize sequential read/write performance*” → focus on **Throughput (st1)**.

If question says “*copy EBS volume to another region*” → **create snapshot** → **copy** → **restore volume**.

If question says “*secure EBS data at rest with least effort*” → **enable EBS encryption by default**.

If “*EC2 instance terminated but data must persist*” → **disable “Delete on Termination”** on volume.

If “*volume performance is lower right after creation*” → due to **lazy loading** (pre-warm or access all blocks once).

If question says “*schedule daily EBS backups*” → **use Data Lifecycle Manager**.

High VolumeQueueLength → increase **IOPS** or switch to **io1/io2**.

If “*data must persist after instance stop*” → **EBS**.

If “temporary high-speed storage” → **Instance Store**.

“Create identical environment in new region” → **copy snapshot** → **create volume** → **attach to EC2**.

Amazon EFS:

Amazon Elastic File System (EFS) is a **fully managed, serverless, elastic NFS (Network File System)** that provides:

- **Shared access** to files across multiple EC2 instances, ECS tasks, EKS pods, and Lambda functions.
- **Automatic scaling** — grows and shrinks as files are added or removed.
- **No provisioning** — you pay only for what you use.

Property	Description
Elastic	Auto-scales up/down as you add/remove files.
Shared	Many clients can mount concurrently (POSIX-compliant).
Durable & Available	Data stored across multiple AZs in a region.
Serverless	AWS manages servers and scaling.
Compatible	Mount via NFSv4.1 — supports Linux, ECS, EKS, Lambda.

Storage Class	Description	Use Case
EFS Standard	Multi-AZ, high availability, low latency	Active workloads
EFS One Zone	Single AZ, lower cost, slightly less durable	Dev/test, temporary workloads

Lifecycle Tier	Description	Use Case
Standard-IA (Infrequent Access)	Automatically moves inactive files after defined period (default 30 days).	Cost optimization
One Zone-IA	Cheaper infrequent access for One Zone EFS	Cold data in dev/test

Mode	Description	Best For
General Purpose (default)	Low latency; most workloads	Web apps, home directories
Max I/O	Higher throughput, more latency	Big data, parallel compute, HPC

Mode	Description	Billing
Bursting (default)	Scales automatically with file size	Free
Provisioned	Set throughput manually (MB/s)	Extra cost

Access Mode	Description
Regional (Multi-AZ)	Data replicated across 3 AZs (default)
One Zone	Single-AZ, cheaper but less durable
Mount Targets	Network entry points (one per AZ per VPC)

EFS can be mounted on:

- EC2 instances (via NFS)
- ECS / EKS containers (via EFS CSI driver)
- AWS Lambda (direct mount, up to 10 GB temp cache)
- Use **AWS Backup** for automated snapshots of EFS.
- Snapshots are incremental.
- Can restore to same or new file system.
- Lambda can mount EFS to persist files across invocations.
- Files are accessible across concurrent Lambda invocations.
- Great for **large ML models, shared temporary data, or config files**.

Metric	Description
BurstCreditBalance	Remaining credits for burst throughput
PercentIOLimit	Percent of I/O limit used
DataReadIOBytes / DataWriteIOBytes	I/O usage over time
ClientConnections	Number of active mounts

Scenario	Solution
Multiple EC2s share same web app files	Mount EFS
Lambda functions share libraries/config	EFS mount
Microservices store shared logs/configs	EFS
Big data or ML workloads needing parallel file access	EFS Max I/O
Auto-scaling web servers storing assets	EFS (shared)
CI/CD build servers storing artifacts	EFS One Zone or Standard

EFS Access Points:

- Simplify shared access to EFS across multiple apps.
- Define **custom paths, POSIX permissions, and user IDs** for each access point.
- Lambda and ECS can mount via **Access Point ARN**.
- **Persistence:** Fully persistent — data remains even if EC2 instances are stopped or terminated.
- **When EC2 is terminated:** The **EFS file system is unaffected** — it's separate from the instance and can be mounted by other instances anytime.

Scenario	Correct Solution
"Multiple EC2 instances need shared file storage"	EFS
"Lambda function must access shared config files"	EFS mount
"Lower latency single-AZ file system"	EFS One Zone
"Reduce cost for infrequently accessed files"	Lifecycle Management → IA
"Encrypt EFS data at rest"	KMS integration
"Automate daily backups"	AWS Backup
"Restrict access to VPC only"	Resource Policy + Security Group
"Scale storage automatically"	EFS (auto-scaling)
"Custom user/group access"	POSIX permissions
"High parallel throughput"	Max I/O mode

If question says *"shared file system for multiple EC2 instances or Lambda functions"* → **EFS**.

If *"persistent disk for one instance"* → **EBS**.

If *"store static files or backups"* → **S3**.

If *"no need to manage storage capacity"* → **EFS (elastic auto-scaling)**.

If question says *"reduce cost for infrequently accessed files"* → **Enable EFS Lifecycle Management (move to IA)**.

"Large number of EC2 instances accessing same file system concurrently" → **Max I/O mode**.

If *"constant high throughput regardless of file size"* → **Provisioned throughput**.

If *"reduce latency for a specific AZ only"* → **One Zone**.

If *"resilient across multiple AZs"* → **Regional (Multi-AZ)**.

If *"multiple Lambda functions need access to same files"* → **EFS mount with Lambda** (serverless + persistent shared state).

If question says *"encrypt files in transit and at rest"* → **enable EFS encryption (KMS + TLS)**.

If *"automate daily EFS backups"* → **AWS Backup**, not CloudWatch or DLM (those are for EBS).

If *"reduce EFS cost for rarely accessed data"* → **Enable Lifecycle Management → IA class**.

“Lambda functions need to share 5 GB model files” → Use EFS (not /tmp or S3).

“Restrict EFS access to certain VPCs or accounts” → EFS Resource Policy.

“Simplify access control for multiple apps sharing EFS” → EFS Access Points.

Amazon S3:

Amazon S3 (Simple Storage Service) is a **fully managed, object-based storage service** used to store and retrieve any amount of data, from anywhere, at any time.

Concept	Description
Bucket	Top-level container for objects (unique name globally).
Object	File + metadata (key, value pairs).
Key	Full path to the object (<code>images/photo.jpg</code>).
Region	Buckets exist in one region.
Version ID	Unique identifier for each version when versioning is enabled.
Prefix	Virtual folder path within a bucket.
Storage Class	Defines cost, availability, and access frequency.

Storage Class	Durability	Availability	Use Case
Standard	99.999999999% (11 9s)	99.99%	Frequently accessed data
Intelligent-Tiering	Same	99.9%	Automatic cost optimization
Standard-IA (Infrequent Access)	Same	99.9%	Long-lived but infrequently accessed data
One Zone-IA	Same	99.5%	Cheap, single-AZ non-critical data
Glacier Instant Retrieval	Same	99.9%	Archival with ms retrieval
Glacier Flexible Retrieval	Same	99.99%	Archival (minutes to hours retrieval)
Glacier Deep Archive	Same	99.99%	Lowest cost, long-term archive (hours)
Reduced Redundancy (Legacy)	Deprecated	N/A	Avoid

Operation	Consistency Model
PUT / DELETE / LIST / GET	Strongly Consistent (immediate visibility across all regions)

Feature	Description
IAM Policies	User or role-based permissions.
Bucket Policies	Resource-based JSON policies applied at bucket level.
ACLs (Access Control Lists)	Legacy, per-object permissions (avoid using).
Block Public Access	Global toggle to prevent accidental exposure.
Object Ownership	Setting that ensures uploaded objects belong to bucket owner.

Type	Description	Managed By
SSE-S3 (AES-256)	Server-side encryption with S3-managed keys	AWS
SSE-KMS	Server-side encryption with KMS CMK	Customer
SSE-C	Customer-provided encryption key	You
CSE (Client-Side)	Encryption before upload	You
Default Encryption	Can enforce encryption at bucket level	AWS or KMS

- Keeps **multiple versions** of an object.
- Protects against accidental deletes and overwrites.
- Combined with **MFA Delete** for strong protection.

S3 Event Notifications:

Trigger actions when events happen in a bucket.

Trigger Example	Target
Object Created	Lambda, SNS, or SQS
Object Deleted	Lambda, SNS, or SQS

Feature	Description
Multipart Upload	Upload parts of large objects in parallel (>100 MB).
Transfer Acceleration	Uses CloudFront edge network for faster global uploads.
Byte-Range Fetch	Retrieve partial object data (parallel downloads).
Prefix Parallelization	Unlimited prefixes; no need to randomize filenames (S3 auto-scales).
Requester Pays	Download cost charged to requester, not bucket owner.

Feature	Description

S3 Access Logs:

- Record all requests to your bucket.
- Delivered to another S3 bucket (cannot be the same).
- Used for **analytics** or **auditing**.

Cross Region Replication:

- Automatically replicates objects between buckets in different regions.
- Requires:
 - Versioning enabled on both buckets.
 - Proper IAM permissions.
- Replication **asynchronous** but near real-time.

S3 Object Lock:

- Prevents deletion or modification of objects for a specified time period.
- **Governance Mode:** Only admins can override.
- **Compliance Mode:** No one can change (regulatory protection).

S3 Glacier:

Class	Retrieval Time	Use Case
Instant Retrieval	Milliseconds	Frequent archive access
Flexible Retrieval (Standard / Bulk)	Minutes-hours	Infrequent access
Deep Archive	Hours (12–48 h)	Long-term cold storage

Feature	Guarantee
Durability	99.999999999% (11 9s)
Availability (Standard)	99.99%
Consistency	Strong read-after-write globally
Data Stored	Across ≥3 AZs (Standard class)

Scenario	Correct Solution
“Host static website with global availability”	S3 + CloudFront

Scenario	Correct Solution
"Protect objects from accidental deletion"	Enable Versioning + MFA Delete
"Share bucket across accounts"	Bucket Policy + IAM roles
"Encrypt data with full audit trail"	SSE-KMS
"Lower cost for cold data"	Lifecycle → IA / Glacier
"Trigger Lambda on file upload"	S3 Event Notification
"Replicate bucket across regions"	Cross-Region Replication
"Prevent changes for compliance"	Object Lock (Compliance Mode)
"Analyze CSV subset in S3"	S3 Select
"Access logs for requests"	Server Access Logging

If *"store data for years at lowest cost"* → **Glacier Deep Archive**.

If *"frequently accessed data with unpredictable patterns"* → **Intelligent-Tiering**.

Older exams used to mention eventual consistency — not anymore. It's **always strong consistency** now.

If *"bucket accidentally exposed to public"* → check **Block Public Access**.

If *"objects uploaded by other accounts should remain owned by bucket owner"* → **Object Ownership** → **Bucket owner enforced**.

If *"need audit trail of who used key to decrypt"* → **SSE-KMS** (logged in CloudTrail).

If *"customer manages encryption keys manually"* → **SSE-C**.

If *"recover from accidental deletion"* → **Enable Versioning**.

If *"permanent delete protection"* → **Enable MFA Delete**.

If *"minimize cost for older data"* → **Lifecycle Policy**.

If *"automatically delete old log files"* → **Expiration Rule**.

If *"uploading large files slowly"* → use **Multipart Upload** or **Transfer Acceleration**.

If *"track who accessed which object"* → **Enable Server Access Logging** or **CloudTrail Data Events**.

If *"business continuity in another region"* → **CRR**.

If *"same region replication for compliance"* → **SRR (Same-Region Replication)**.

If *"analyze subset of data without downloading entire object"* → **S3 Select**.

If *"retain logs for 7 years due to compliance"* → **S3 Object Lock (Compliance Mode)**.

If *"run queries on S3 files without a DB"* → **Athena**.

If *"process uploaded files automatically"* → **Lambda trigger**.

Amazon DynamoDB:

Amazon DynamoDB is a **fully managed NoSQL database service** that provides **single-digit-millisecond latency** at any scale.

It's designed for **key-value** and **document** data models.

Feature	Description
Fully managed	No servers to manage, auto-scaling capacity
Serverless	Pay only for what you use (on-demand mode)
High performance	Millisecond latency, global scale
Scalable	Automatically scales partitions for throughput
Integrated security	KMS encryption, IAM access control
Event-driven	Streams trigger Lambda functions
Global Tables	Multi-region replication
Point-in-Time Recovery (PITR)	Restore to any second in last 35 days

Component	Description
Table	Collection of items (like a table in SQL)
Item	A record (like a row)
Attribute	A field (like a column)
Primary Key	Uniquely identifies each item
Partition Key	Hash key used for partitioning data
Sort Key	Optional; allows multiple items per partition
Index	Additional query structure (GSI or LSI)

Primary Keys

Type	Description	Example
Partition Key only	Simple primary key (unique hash)	<code>user_id</code>
Partition + Sort Key	Composite primary key	<code>user_id + timestamp</code>

Capacity Modes:

Mode	Description	When to Use
Provisioned	You set RCU/WCU manually	Predictable workload
On-Demand	Auto-scales with traffic	Unpredictable workload

1 RCU = one strongly consistent read per second for 4 KB

1 WCU = one write per second for 1 KB

Type	Description	Notes
Eventually Consistent Reads (Default)	May return stale data (low cost)	Uses 0.5 RCU per 4 KB
Strongly Consistent Reads	Always up-to-date (double cost)	1 RCU per 4 KB
Transactional Reads	ACID guarantees	2x cost of standard reads

Index Type	Key Structure	Query Behavior	Notes
Local Secondary Index (LSI)	Same Partition Key, different Sort Key	Query alternate attributes within same partition	Created at table creation only
Global Secondary Index (GSI)	Different Partition + Sort Keys	Query on completely different attributes	Created anytime , uses separate capacity

Operation	Description	Efficient?
Query	Retrieve items by Partition Key (and optional Sort Key condition)	✅ Very efficient
Scan	Reads entire table and filters results	❌ Inefficient

- **DynamoDB Streams** capture changes (insert, update, delete) in near real-time.
- Stored for **24 hours**.
- Can trigger **Lambda**, **Kinesis**, or **other consumers**.

TTL:

- Automatically deletes items after a specific timestamp (epoch format).
- Great for expiring sessions, temporary logs, etc.
- Expired items deleted asynchronously (not immediate).

Feature	Description
IAM Policies	Fine-grained access control (per table or item).
Encryption at Rest	AWS-managed KMS key (default) or customer CMK.

Feature	Description
Encryption in Transit	HTTPS (TLS).
VPC Endpoints	Private connectivity from VPC.

Global Tables:

- **Multi-Region, multi-master replication.**
- Enables **low-latency** global reads/writes.
- Automatic conflict resolution (last writer wins).
- Requires streams enabled.

Transactions:

- Support **ACID guarantees** across multiple items and tables.
- Use `TransactWriteItems` and `TransactGetItems` .
- **2× RCUs/WCUs** cost.

Feature	Description
On-Demand Backup	Full table snapshot
PITR (Point-in-Time Recovery)	Restore to any second within last 35 days

DynamoDB Accelerator (DAX) is an **in-memory cache** for DynamoDB.

Feature	Description
Purpose	Reduce latency for read-heavy workloads
Latency	Microseconds
Integration	Drop-in replacement for DynamoDB SDK
Managed Service	Fully managed by AWS
Consistency	Eventually consistent only

Operation	Purpose
PutItem	Create/replace item
GetItem	Retrieve by key
UpdateItem	Modify item attributes
DeleteItem	Delete by key

Operation	Purpose
Query	Retrieve items by Partition/Sort key
Scan	Read entire table
BatchWriteItem / BatchGetItem	Bulk operations
TransactWriteItems	Transactional write

Metric	Description
ConsumedReadCapacityUnits	RCUs used
ConsumedWriteCapacityUnits	WCUs used
ThrottledRequests	Requests exceeding capacity
SystemErrors	Internal issues
ReturnedItemCount	Items returned by query/scan

Scenario	Solution
"Retrieve all orders for a user sorted by date"	Partition = user_id, Sort = order_date
"Query by non-key attribute"	Create GSI
"Delete expired sessions automatically"	TTL
"Trigger process after insert"	DynamoDB Stream + Lambda
"Global replication"	Global Table
"Reduce read latency"	DAX
"Recover deleted items"	PITR
"ACID transaction across tables"	Transactional Writes
"Handle unpredictable load"	On-Demand mode
"Ensure item not overwritten"	Conditional Write

If question says *"query multiple items for same user ordered by time"* → use **composite key** with **Sort Key (timestamp)**.

If *"traffic spikes unpredictably"* → **On-Demand mode**.

If *"cost control for steady usage"* → **Provisioned mode + Auto Scaling**.

If *"read must reflect latest write immediately"* → **Strongly Consistent Read**.

- *"Query on non-key attribute"* → **GSI**.
- *"Query items by same partition but different sort attribute"* → **LSI**.

If *"app scans whole table and is slow"* → replace **Scan** with **Query** using indexed key.

If “*trigger Lambda every time new item added*” → **Enable DynamoDB Stream (NEW_IMAGE)**.

If “*auto-delete expired user sessions*” → **Enable TTL**.

If “*secure DynamoDB within private network*” → **use VPC Endpoint (Gateway type)**.

If “*application must read/write data from multiple regions with low latency*” → **Global Table**.

If “*ensure both updates succeed or none at all*” → **Transactional Write**.

If “*recover accidentally deleted data*” → **PITR**.

If “*create periodic backups*” → **On-Demand Backup**.

If “*perform atomic multi-item operation*” → **TransactWriteItems**.

If “*reads are throttled*” → **increase RCU** or enable **Auto Scaling**.

Primary Keys:

Every DynamoDB table *must* have a **Primary Key**, which defines how data is stored and retrieved.

There are two types:

Type	Structure	Example	Purpose
Partition Key (Hash Key)	Single attribute	user_id	Determines partition (physical storage)
Partition + Sort Key (Composite Key)	Two attributes	user_id + order_id	Allows multiple items per user, sorted by sort key

You can only query by the primary key (and sort key).

If you need to query by *any other attribute*, you need an **index**.

Indexes:

Indexes are like “alternate query views” of your data — they let you query efficiently by **non-primary-key attributes**.

DynamoDB supports **two types**:

LSI:

Feature	Description
Partition Key	Same as base table
Sort Key	Different attribute
Creation Time	Must be defined at table creation
Capacity Units	Uses same RCU/WCU as base table
Storage	Stored with base table, counts toward its size limit

Feature	Description
Consistency	Can be strongly consistent (same partition)

Query the same user (partition key) by multiple attributes — e.g., by *date*, *status*, or *amount*.

Example:

Table: `Orders`

- Partition Key → `CustomerID`
- Sort Key → `OrderDate`
- LSI → `CustomerID + OrderStatus`

You can now query *all orders for a customer* by `OrderStatus` .

GSI:

Feature	Description
Partition Key	Different from base table
Sort Key	Optional, can differ
Creation Time	Can be created anytime
Capacity Units	Separate RCUs/WCUs (independent scaling)
Storage	Separate physical index
Consistency	Eventually consistent only
Projection	Choose which attributes to copy into index

Query by completely different attributes — e.g., by `City` or `Email` .

Example:

Table: `Users`

- Primary Key → `UserID`
- GSI → `City` as Partition Key

Now you can query all users in `Chicago` .

ACID Transactions:

ACID = Atomic, Consistent, Isolated, Durable

It ensures **reliable, all-or-nothing operations** — similar to SQL databases.

Property	Meaning	Example
Atomic	All operations in transaction succeed or none do	Update both tables or roll back both

Property	Meaning	Example
Consistent	Data remains valid under all constraints	Enforces item conditions
Isolated	Transactions don't affect each other	Parallel updates won't see partial data
Durable	Once committed, data is safe	Survives system or network failure

API	Description
TransactWriteItems	Write multiple items atomically
TransactGetItems	Read multiple items atomically

- ✅ Supports up to **25 items or 4 MB** per transaction.
- ✅ **Costs 2× RCUs/WCUs** of normal operations.
- ✅ **Strongly consistent by default.**

AWS Lambda:

AWS Lambda lets you **run code without provisioning or managing servers**.

You upload your code (called a *function*), and Lambda automatically runs it in response to **events** such as:

- HTTP requests (via API Gateway)
- S3 uploads
- DynamoDB Streams
- EventBridge events
- SNS or SQS messages
- Step Functions tasks

Concept	Description
Function	Your code + runtime (Python, Node.js, Java, Go, .NET, etc.)
Handler	Entry point for your function (e.g., <code>index.handler</code>)
Event	JSON payload that triggers the function
Context Object	Info about invocation (timeout, request ID, memory)
Execution Role	IAM role that gives permissions to AWS services
Environment Variables	Config data available to function
Timeout	Max duration (default 3 s, max 15 min)

Concept	Description
Memory	128 MB – 10 GB (CPU scales linearly with memory)

Invocation Types:

Type	Behavior	Example
Synchronous	Caller waits for result	API Gateway, CLI, SDK
Asynchronous	Queued and retried automatically	S3, SNS, EventBridge
Event Source Mapping (Poll-based)	Lambda polls and processes records	SQS, Kinesis, DynamoDB Streams

Concept	Description
Concurrency	Number of executions running in parallel
Burst Scaling	Instantly scales up (up to 1,000 per region, then 500 per minute)
Reserved Concurrency	Caps or reserves concurrent executions
Provisioned Concurrency	Pre-warms environments for low latency

Type	Description
Cold Start	New environment initialized → higher latency
Warm Start	Reuse previous environment → fast

Lambda Destinations:

Type	Description
OnSuccess	Send result to SQS, SNS, Lambda, or EventBridge
OnFailure	Send failure info to same targets

Error Handling and Retries:

Invocation Type	Retry Behavior
Synchronous	Caller handles retry

Invocation Type	Retry Behavior
Asynchronous	Auto-retries twice with exponential backoff
Stream (Kinesis/DynamoDB)	Retries until success or data expires
SQS	Uses queue's visibility timeout + DLQ

Lambda Layers:

- Reusable **code or dependencies** shared across functions.
- Supports versioning.
- Example: Node.js modules, Python libs, SDKs.
- Up to **5 layers per function**, 50 MB each (250 MB total).

Role	Description
Execution Role	Grants Lambda permissions to AWS services
Resource-based Policy	Allows other AWS services to invoke Lambda
KMS Integration	Encrypt environment variables
VPC Access	Attach to subnets and security groups

Source	Integration Type
API Gateway	Synchronous (HTTP → Lambda)
S3	Asynchronous (on upload event)
SNS	Asynchronous
SQS	Poll-based via Event Source Mapping
DynamoDB Streams / Kinesis	Poll-based via Event Source Mapping
EventBridge	Asynchronous
CloudWatch Events / Logs	Scheduled or triggered

Deployment Options:

Method	Description
Zip upload	Direct upload (≤ 50 MB zip, ≤ 250 MB uncompressed)
Container Image	Up to 10 GB, supports custom runtimes
S3 Bucket	Store deployment packages for large codebases
Infrastructure as Code	CloudFormation, SAM, or CDK

Concept	Description
Version	Immutable snapshot of a Lambda function
Alias	Pointer to a specific version (like a tag)
Traffic Shifting	Gradually move traffic between versions (blue/green)

Setting	Impact
Memory (128 MB – 10 GB)	More memory → more CPU → faster execution
Timeout (max 15 min)	Prevents infinite loops
Ephemeral Storage (/tmp)	Up to 10 GB temporary local space
Retries	Vary by invocation type

Scenario	Solution
“Process uploaded images in S3”	S3 Event → Lambda
“API with no servers”	API Gateway + Lambda
“Process queue messages”	SQS Event Source Mapping
“Invoke on DynamoDB changes”	DynamoDB Streams + Lambda
“Run daily scheduled job”	EventBridge (Cron) → Lambda
“Share dependencies”	Lambda Layers
“Track errors & duration”	CloudWatch Metrics
“Persist logs”	CloudWatch Logs
“Avoid cold starts”	Provisioned Concurrency
“Roll out new version safely”	Aliases + Weighted Routing

If question says “no servers, automatic scaling, pay per request” → **Lambda**.

- “Retries automatically after failure” → **Asynchronous invocation**.
- “Lambda polls a stream and processes batches” → **Event Source Mapping**.
- “Prevent noisy-neighbor from starving other functions” → **Reserved Concurrency**.
- “Avoid cold starts in latency-sensitive API” → **Provisioned Concurrency**.

Reduce Cold Starts:

- Provisioned Concurrency
- Keep function warm (periodic invoke)
- Use smaller dependencies / layers

If “need to route failed async invocations to queue” → **Lambda Destination**.

If “*asynchronous function keeps failing*” → set **DLQ** or **Lambda Destination**.

If “*share common libraries across multiple functions*” → **Lambda Layers**.

If “*Lambda can’t access RDS in private subnet*” → attach **VPC config** (subnet + security group).

- “*Run code when new object uploaded to S3*” → **S3 Event** → **Lambda**.

- “*Process messages from SQS queue*” → **Event Source Mapping**.

If “*deploy 1 GB runtime with custom binaries*” → **Container-based Lambda**.

If “*safe deploy new code gradually*” → use **Alias + Weighted Traffic**.

If “*trace latency across microservices*” → **Enable X-Ray**.

If “*monitor number of invocations*” → **CloudWatch Metrics**.

If “*automate Lambda deployments safely*” → **CodeDeploy + Canary strategy**.

If “*function runs slow*” → increase **memory** (CPU scales automatically).

If “*optimize cost for low-usage workload*” → use **on-demand** (default).

If “*predictable load needing pre-warming*” → use **Provisioned Concurrency**.

AWS CloudFormation:

AWS CloudFormation (CFN) is a service that lets you **define and manage AWS infrastructure as code (IaC)** using **templates** written in **YAML** or **JSON**.

Instead of manually creating resources, you describe them in a template, and CloudFormation automatically **provisions, configures, and manages** them.

Concept	Description
Template	The YAML/JSON file that defines resources
Stack	A deployed instance of the template
StackSet	Deploys stacks across multiple accounts/regions
Change Set	Preview of modifications before applying
Drift Detection	Detects manual changes outside CloudFormation
Parameters	Inputs to make templates reusable
Outputs	Export values for other stacks
Resources	AWS components to create (e.g., S3, EC2, Lambda)
Mappings	Region or environment-specific values
Conditions	Logical controls (e.g., create only if Env=Prod)
Metadata	Extra information for tools like SAM or CLI

4 Template Anatomy (5 Main Sections)

yaml

 Copy code

```
AWSTemplateFormatVersion: '2010-09-09'
Description: My sample CFN template

Parameters:
  EnvType:
    Type: String
    Default: dev

Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: my-sample-bucket

Outputs:
  BucketName:
    Value: !Ref MyBucket
```

Section	Purpose
Parameters	User input values (e.g., EnvType)
Resources	Actual AWS resources
Outputs	Export values or show results
Mappings	Conditional value lookups
Conditions	Conditional logic for resources

Function	Purpose	Example
<code>!Ref</code>	Reference another resource or parameter	<code>!Ref MyBucket</code>
<code>!GetAtt</code>	Get an attribute from a resource	<code>!GetAtt MyEC2.PublicDnsName</code>
<code>!Sub</code>	Substitute variables	<code>!Sub arn:aws:s3:::\${MyBucket}</code>
<code>!Join</code>	Concatenate strings	<code>!Join ['-', ['app', !Ref Env]]</code>
<code>!If</code>	Conditional logic	<code>!If [IsProd, t3.large, t3.micro]</code>
<code>!FindInMap</code>	Lookup in mapping	<code>!FindInMap [RegionMap, us-east-1, AMI]</code>
<code>!ImportValue</code>	Import exported output from another stack	<code>!ImportValue VPCID</code>

Stack Lifecycle:

- **Create Stack** — CloudFormation provisions resources.
- **Update Stack** — Apply modified template (can use Change Set).
- **Delete Stack** — Removes all associated resources.
- **Rollback** — If failure occurs, deletes resources created in this attempt.
- A **Change Set** previews all modifications before applying an update.
- Shows which resources will be added, modified, or deleted.
- Drift Detection Detects **manual changes** to resources outside CloudFormation.
- Marks resources as **Drifted**, **In Sync**, or **Deleted**.
- Stack Policies Protect critical resources from accidental deletion or modification.
- Applied during stack updates.

-

Example:

```
json                                                                    Copy code

{
  "Statement": [{
    "Effect": "Deny",
    "Action": "Update:*",
    "Principal": "*",
    "Resource": "LogicalResourceId/MyDB"
  }]
}
```

- Nested Stacks - Use one CloudFormation template inside another.
- Great for modular, reusable architecture.

1 5

Cross-Stack References

- Share outputs between stacks.

yaml

Copy code

```
Outputs:
  VPCID:
    Value: !Ref MyVPC
    Export:
      Name: SharedVPC
```

yaml

Copy code

```
Resources:
  MySubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !ImportValue SharedVPC
```

Helper	Description
cfn-init	Sets up EC2 instance (installs software, configures files)
cfn-signal	Sends success/failure signal during creation
cfn-get-metadata	Retrieves metadata defined in template
cfn-hup	Watches for metadata changes and triggers updates

- CF StackSets Manage stacks across **multiple AWS accounts and regions**.
- Useful for enterprise multi-account setups.

Policy	Behavior
Delete (default)	Resource deleted
Retain	Resource preserved
Snapshot	Take backup before deletion (EBS, RDS)

Category	Feature	Exam Keyword
Type	Infrastructure as Code	“Automate resource creation”
Templates	YAML / JSON	“Reusable blueprints”
Resources	AWS::<Service>::<Resource>	“Define infrastructure”

Category	Feature	Exam Keyword
Stacks	Deployed templates	“Create / Update / Delete”
Change Sets	Preview updates	“Review before apply”
Rollback	Auto revert on failure	“Safe deployments”
Deletion Policy	Retain / Snapshot / Delete	“Preserve DB”
Drift Detection	Find manual changes	“Detect drift”
Cross-Stack	Export / ImportValue	“Share outputs”
Nested Stacks	Modular templates	“Reusable components”
StackSets	Multi-account / region	“Enterprise deployments”
Helpers	cfn-init, cfn-signal	“Wait until EC2 setup complete”
Integration	CodePipeline / SAM / CDK	“CI/CD automation”

If question says “*automate infrastructure deployment with version control*” → **CloudFormation**.

If “*template should work in multiple environments*” → use **Parameters**.

If “*region-specific AMI IDs*” → use **Mappings**.

If “*combine values dynamically*” → use **!Sub**.

If “*fetch attribute from created resource*” → **!GetAtt**.

- “*Re-use values across stacks*” → **Outputs + Export/ImportValue**.
- “*Template should accept user-specific input*” → **Parameters**.
If “*deployment partially succeeded and auto-reverted*” → **Stack rollback**.
If “*must review updates before applying*” → **Use Change Set**.
If “*resource state changed outside template*” → use **Drift Detection**.
If “*prevent database deletion during updates*” → **Stack Policy**.
If “*auto-revert update on error or high CPU*” → **Rollback Trigger**.
If “*keep database after stack deletion*” → **DeletionPolicy: Retain**.
If “*reuse template components across environments*” → **Nested Stacks**.
If “*reuse existing resources in another stack*” → **Export / ImportValue**.
If “*CloudFormation must wait until EC2 config is complete*” → use **cfn-signal**.
If “*deploy same infra to multiple accounts*” → **StackSets**.

AWS CodeGuru:

Amazon CodeGuru is a **machine learning–powered developer tool** that helps you:

- **Review code automatically** for security, performance, and best-practice issues.
- **Profile running applications** to identify performance bottlenecks and reduce compute cost.

It has two components:

1. **CodeGuru Reviewer** → for **static code analysis**
2. **CodeGuru Profiler** → for **runtime performance analysis**

Component	Purpose	Works On
CodeGuru Reviewer	ML-based static code reviews that find security issues, bugs, and best-practice violations	Source code (Java, Python, JavaScript, TypeScript)
CodeGuru Profiler	Analyzes running applications to find CPU, memory, and latency hotspots	Live applications (Java, Python, Lambda, ECS, EC2, etc.)

Reviewer:

Automatically reviews your source code (from **GitHub**, **Bitbucket**, or **AWS CodeCommit**) and provides intelligent recommendations for:

- Code quality
- Security vulnerabilities
- Resource leaks (e.g., open connections)
- Concurrency issues
- AWS API best practices
- Connect CodeGuru Reviewer to your **repository** (e.g., GitHub or CodeCommit).
- It runs **machine learning models trained on thousands of codebases**.
- Provides recommendations via the **AWS Console**, **CodeGuru Console**, or **Pull Requests**.
- Supports **code scanning** during **pull requests** or **full repository analysis**.

Profiler:

Finds **performance inefficiencies** in **live, running applications**.

It helps you:

- Reduce **CPU utilization**
- Lower **compute cost**
- Identify **high-latency code paths**
- Add the **CodeGuru Profiler agent** to your app (Java, Python, Lambda).
- The agent sends runtime metrics to **CodeGuru Profiler**.
- The service analyzes data and creates a **flame graph** showing where CPU time is spent.

Scenario	Use
“Find code security issues and best-practice violations automatically”	CodeGuru Reviewer
“Reduce Lambda function CPU time and cost”	CodeGuru Profiler
“Identify thread contention in EC2 app”	CodeGuru Profiler
“Get ML-based recommendations for code style and AWS API usage”	CodeGuru Reviewer
“Visualize CPU hotspots and optimize performance”	CodeGuru Profiler

Component	Supported Languages
Reviewer	Java, Python, JavaScript, TypeScript
Profiler	Java, Python (Lambda + containerized apps)

If question says “*developer wants automated code review that checks for security and performance issues*” → **CodeGuru Reviewer**.

If “*automated reviewer identifies unclosed streams or inefficient queries*” → **CodeGuru Reviewer**.

If question says “*application consumes too much CPU or cost on EC2/Lambda*” → **CodeGuru Profiler**

If “*company wants to cut compute cost of Java app without changing infra*” → **CodeGuru Profiler**.

If “*automate review + runtime optimization in CI/CD*” → **CodeGuru Reviewer + Profiler**.

AWS Certificate Manager

AWS Certificate Manager (ACM) is a service that:

- **Provisions, manages, and deploys SSL/TLS certificates**
- Secures communication between **clients and AWS resources**
- **Automatically renews** certificates
- Secure APIs, websites, and applications with HTTPS (SSL/TLS)
- Integrate easily with AWS services like **CloudFront, ALB, API Gateway, App Runner**, and **Elastic Beanstalk**
- Avoid manual certificate installation and renewal

Type	Description	Use Case
Public Certificates	Issued by ACM at no cost, validated via DNS or email	Websites, APIs with public endpoints
Private Certificates	Issued via ACM Private CA , not public	Internal apps, private networks
Imported Certificates	Bring your own cert (BYOC)	Third-party or custom CA

Method	Description	Notes
DNS Validation	Add a CNAME record in Route 53 or your DNS provider	Recommended (auto-renews)
Email Validation	Confirmation link sent to domain owner's email	Slower, manual
Private CA	Internal PKI validation	For internal use only

Service	How It Uses ACM
Elastic Load Balancer (ALB/NLB)	Attach ACM cert for HTTPS listener
CloudFront	Use ACM cert (in us-east-1) for HTTPS distribution
API Gateway	Secure custom domain with ACM cert
App Runner / Elastic Beanstalk	HTTPS via ACM
AWS Global Accelerator	TLS termination
Amazon Lightsail	HTTPS for managed websites

Lifecycle:

- **Request certificate** (public, private, or import)
- **Validate ownership** (DNS/email)
- **ACM issues and stores cert**
- **Attach cert** to service (CloudFront, ALB, etc.)
- **ACM auto-renews** before expiration

Without ACM → you manually buy a certificate, install it on servers, renew yearly.

With ACM → AWS handles the entire lifecycle.

AWS Service	How to Attach
ALB/NLB	Add HTTPS listener → choose ACM cert ARN
CloudFront	Select ACM cert (must be in us-east-1)
API Gateway	Attach ACM cert to custom domain
Elastic Beanstalk	Via load balancer configuration
App Runner	Via console or CLI setting for HTTPS

Limitation	Explanation
CloudFront region	ACM cert must be in us-east-1
Import cert renewals	Manual (ACM won't auto-renew imported certs)
No direct EC2 usage	You can't directly install ACM certs on EC2 — only attach via ELB/CloudFront
Custom Domains	Domain ownership must be validated
Private CA	Costs extra; not for public internet

AWS Private Certificate Authority:

- A **managed private CA service** for issuing internal certificates.
- Used for **internal apps, IoT devices, or VPNs**.
- Allows you to **create and sign** private SSL/TLS certs.
- **Costs extra** (unlike public certs, which are free).

✅ Exam Tip:

If “*issue internal SSL certificates for corporate apps*” → **ACM Private CA**.

If “*secure public website*” → **Public Certificate via ACM**.

AWS Cognito:

Amazon Cognito is a **user identity and authentication service** that lets you:

- Add **sign-up / sign-in** and **access control** to web & mobile apps.
- Authenticate users via **email/password, social logins, or SAML/OIDC** identity providers.
- Issue **JWT tokens** (ID, access, refresh) to authorize requests.

Component	Description	Used For
User Pool	Manages user sign-up/sign-in	Authentication

Component	Description	Used For
Identity Pool (Federated Identities)	Grants AWS credentials to users	Authorization to AWS resources
Cognito Sync (deprecated)	Used to sync user data across devices	Replaced by AppSync

User Pool:

- Handles **sign-up, sign-in, password reset, and MFA**.
- Issues **JWT tokens** after successful authentication.
- Can act as an **OAuth 2.0 authorization server**.
- Supports **custom authentication flows** via **Lambda triggers**.

Token	Purpose	Example Use
ID Token	User identity claims (name, email, etc.)	App UI / personalization
Access Token	Authorize access to APIs	API Gateway
Refresh Token	Get new tokens without login	Session renewal

Cognito User Pools support:

- **Username/password**
- **Social logins** (Google, Facebook, Apple)
- **SAML / OIDC enterprise logins**

Identity Pool:

- Provides **temporary AWS credentials** to authenticated or guest users using **STS (Security Token Service)**.
- Maps authenticated identities (from Cognito or external IdPs) to **IAM roles**.

Identity Flow

- 1 User logs into User Pool (or federated provider).
- 2 Cognito issues a token.
- 3 Identity Pool exchanges token for **temporary AWS credentials** via **STS**.
- 4 App uses these credentials to access AWS (e.g., S3, DynamoDB).

Feature	Description
Multi-Factor Authentication (MFA)	SMS / TOTP support

Feature	Description
Device Tracking	Remember devices for faster login
Account Recovery	Email/SMS-based
Token Revocation	API to invalidate tokens
Advanced Security (Adaptive Auth)	Detects suspicious logins (device/IP anomalies)

Service	Integration	Description
API Gateway	Authorizer	Validates JWT tokens from Cognito
AppSync	Auth mode	Supports Cognito User Pools
Lambda	Custom auth flows or triggers	Custom logic
S3 / DynamoDB	Access via Identity Pool roles	Temporary credentials
CloudFront	Secure access using Cognito JWT	Protect content

Type	Description
Cognito User Pool Authorizer	Validates JWT tokens automatically
Lambda Authorizer	Custom logic for token validation
IAM Authorization	Signed requests with credentials

- 1 User signs up / logs in (User Pool)
- 2 Cognito returns JWT tokens
- 3 Tokens sent to API Gateway → validated via Cognito Authorizer
- 4 Identity Pool exchanges token for AWS credentials
- 5 User accesses AWS resources (e.g., S3, DynamoDB)

Aspect	Cognito Token	STS Credentials
Format	JWT	Temporary IAM creds
Used By	APIs, app logic	AWS SDK (S3, DynamoDB)
Duration	1 hour (ID/Access)	1 hour (default)
Source	User Pool	Identity Pool

AWS Secrets Manager:

AWS Secrets Manager is a **fully managed service** that helps you:

- **Store, encrypt, and retrieve secrets** (like DB passwords, API keys, OAuth tokens).

- **Rotate secrets automatically** using AWS Lambda.
- **Securely access secrets** from apps or AWS services (Lambda, EC2, ECS, etc.).

Developers often need to manage credentials securely — for example:

- Database connection strings
- Third-party API keys
- JWT signing keys
- OAuth tokens
- Database usernames/passwords (RDS, Aurora, MySQL, etc.)
- API keys or OAuth tokens
- SSH keys or app secrets
- JSON documents with multiple key/value pairs

One of the most important features for the exam 🙋

- Secrets Manager can **automatically rotate secrets** on a schedule.
- Uses a **Lambda rotation function** (pre-built templates exist for RDS).
- Example: Rotate RDS password every 30 days.
 - ✅ Steps:
 - 1 Enable rotation in the Secrets Manager console.
 - 2 Select or create a rotation Lambda.
 - 3 Secrets Manager calls Lambda periodically to update the secret and the database password.

You can retrieve secrets from:

- **AWS SDKs (boto3, AWS SDK for Node.js, etc.)**
- **AWS CLI**
- **Lambda environment**
- **ECS task definitions**

Feature	Secrets Manager	SSM Parameter Store
Use Case	Store sensitive secrets	Store config parameters
Automatic Rotation	✅ Yes	❌ No
KMS Encryption	✅ Default	✅ Optional
Versioning	✅	✅
Pricing	Paid per secret + API call	Free (standard tier)
Integrations	RDS, Lambda, ECS	SSM, EC2, CloudFormation

Service	Integration
RDS / Aurora	Automatic rotation of DB credentials
Lambda	Retrieve secrets securely at runtime
ECS / Fargate	Inject secrets into container env vars
EC2	Retrieve secrets with SDK
CloudFormation	Reference secrets via dynamic values (<code>{{resolve:secretsmanager:MySecret:SecretString:password}}</code>)
API Gateway / AppSync	Call downstream APIs with secrets pulled securely

Each secret can have multiple versions labeled as:

Label	Meaning
AWSCURRENT	Active version used now
AWSPREVIOUS	Last version before rotation
AWSPENDING	Version being rotated (not active yet)

- 1 Store DB password in Secrets Manager
- 2 Lambda retrieves it at runtime via AWS SDK
- 3 Secrets Manager auto-rotates password every 30 days
- 4 Lambda continues retrieving updated password automatically

Scenario	Solution
Secure DB credentials in Lambda	Store in Secrets Manager
Rotate RDS passwords automatically	Enable rotation with Lambda
Store third-party API key	Secrets Manager
Store non-sensitive config	Parameter Store
Restrict access to one team	IAM policy with resource ARN
Use secrets in CloudFormation template	Dynamic reference

If the question says “*store and automatically rotate credentials securely*” → **Secrets Manager**.
If it says “*store non-sensitive config data*” → **Systems Manager Parameter Store**.
If “*ensure database credentials are encrypted and rotated automatically*” → Secrets Manager + KMS.

If “company wants automatic password rotation for RDS without downtime” → **Secrets Manager rotation Lambda**.

If “Lambda needs DB password securely” → retrieve via **Secrets Manager SDK**, not env vars.

If “rotate DB credentials automatically” → **Secrets Manager**.

If “store app configuration (non-sensitive)” → **Parameter Store**.

If “Lambda retrieves credentials for connecting to RDS” → **Secrets Manager + GetSecretValue**.

If “Lambda rotation function updates secret but not yet active” → version labeled **AWSPENDING**.

If “company wants free configuration storage” → **Parameter Store**.

If “secure automatic rotation needed” → **Secrets Manager (paid)**.

AWS Security Token Service (STS):

AWS Security Token Service (STS) is an AWS service that issues **temporary, limited-privilege credentials** for users, roles, or federated identities.

As a developer, you use STS when:

- Your **app or Lambda** needs **temporary AWS credentials** (not hardcoded keys).
- Users from another AWS account or external IdP (like Google, SAML, or Cognito) must access AWS resources.
- You need **cross-account access** without creating permanent IAM users.

1 Caller requests temporary credentials (via STS API, SDK, or federation).

2 STS validates the request and issues temporary credentials:

- Access Key ID
- Secret Access Key
- Session Token

3 Caller uses those credentials to access AWS resources.

4 Credentials automatically expire (usually within **15 min – 12 hrs**, depending on role/session policy).

API Action	Purpose	Typical Use Case
<code>AssumeRole</code>	Get temporary creds for a role in same or another AWS account	Cross-account access
<code>AssumeRoleWithSAML</code>	For SAML 2.0 federated users	Enterprise federation (Active Directory, Okta)

API Action	Purpose	Typical Use Case
<code>AssumeRoleWithWebIdentity</code>	For OIDC (web/social) providers	Cognito, Google, Facebook login
<code>GetSessionToken</code>	For IAM user temporary creds (MFA)	CLI/console session with MFA
<code>GetFederationToken</code>	For federated users from custom identity systems	Temporary access for 3rd party users
<code>DecodeAuthorizationMessage</code>	Decode encoded error messages	Debugging IAM access denied errors

Scenario	Description	STS API Used
Cross-account access	Allow Account A to access Account B's resources	<code>AssumeRole</code>
Federated identity (Google, Facebook)	Cognito exchanges user token for AWS creds	<code>AssumeRoleWithWebIdentity</code>
Enterprise SSO (SAML)	Corporate users access AWS via IdP	<code>AssumeRoleWithSAML</code>
Temporary credentials for IAM users	For MFA or short-term CLI access	<code>GetSessionToken</code>
Third-party federation	Custom authentication system issues access	<code>GetFederationToken</code>

Service	Integration with STS
AWS Cognito	Uses STS under the hood for temporary AWS creds via Identity Pools
Lambda / EC2 Roles	Automatically assume roles using STS
AWS CLI with MFA	Uses <code>GetSessionToken</code>
Cross-account IAM roles	Use STS to <code>AssumeRole</code> into target account
S3 Signed URLs	Often generated using temporary credentials
App Federation	Web/mobile apps use STS tokens for secure access to S3/DynamoDB

Method	Default Duration	Max Duration
<code>AssumeRole</code>	1 hour	12 hours
<code>AssumeRoleWithSAML</code>	1 hour	12 hours

Method	Default Duration	Max Duration
AssumeRoleWithWebIdentity	1 hour	12 hours
GetSessionToken	12 hours	36 hours (non-MFA), 1 hour (MFA)
GetFederationToken	12 hours	36 hours

Feature	STS	IAM Roles
Purpose	Issues temporary credentials	Defines permissions & trust
Lifetime	Short (15 min–12 hrs)	Persistent
Usage	Used to assume roles or federate users	Used by EC2/Lambda, etc.
Management	API-based	IAM-managed

Mechanism	Purpose
Trust Policy	Defines who can assume the role
Session Policy	Optionally limits session permissions
MFA with STS	Adds multi-factor protection
CloudTrail	Logs all STS API calls

Scenario	Use
Cross-account Lambda invokes S3 in another account	AssumeRole
Temporary AWS creds for web/mobile users	AssumeRoleWithWebIdentity
MFA-protected CLI access	GetSessionToken
AD/SAML federation for corporate users	AssumeRoleWithSAML
Custom federation from on-prem apps	GetFederationToken

If the question says *“temporary credentials”, “assume role”, or “federated user access”* → it’s **STS**.

- *“Cross-account Lambda execution”* → AssumeRole
- *“Temporary credentials for Cognito web users”* → AssumeRoleWithWebIdentity
- *“Active Directory / SAML login”* → AssumeRoleWithSAML

If *“mobile app uploads directly to S3 after login”* → Cognito + STS temporary credentials.

If *“allow Account A to access resources in Account B securely”* → **STS AssumeRole + trust policy**.

If *“Cognito app accesses AWS resources using temporary creds”* → **STS Web Identity Federation**.

If “*use third-party IdP (like Facebook) for AWS access*” → STS

`AssumeRoleWithWebIdentity` .

If “*restrict access for temporary credentials*” → attach **Session Policy** when calling

`AssumeRole` .

AWS WAF:

AWS WAF (Web Application Firewall) helps protect your web applications from:

- **Common exploits** such as SQL injection, cross-site scripting (XSS), bad bots
- **Unwanted traffic** from specific IPs, countries, or patterns
- **Layer 7 (HTTP/S)** web attacks

Integration	Description
CloudFront	Global CDN protection (edge level)
Application Load Balancer (ALB)	Regional protection for web apps
API Gateway	Protects REST APIs
AppSync	Protects GraphQL APIs
AWS Verified Access	(Advanced use case) Protects private apps

Concept	Description
Web ACL (Access Control List)	The main container that holds rules; attached to a CloudFront, ALB, or API Gateway
Rules	Define what to block, allow, or count
Rule Groups	Collections of reusable rules
Managed Rule Groups	Pre-built rule sets by AWS or vendors
Custom Rules	Your own IPs, patterns, or conditions
Conditions / Statements	Logic inside a rule (e.g., IP match, regex, SQL injection)

Rule Type	Description	Example
IP Set Match	Block or allow based on IPs	Block known attacker IPs
Geo Match	Filter traffic by country	Allow only US users
String Match	Match a header, query, or body string	Block “DROP TABLE”
Regex Match	Match using regular expressions	Detect malicious patterns

Rule Type	Description	Example
Rate-based Rule	Block IPs sending too many requests	Limit 1,000 requests/5 minutes
SQL Injection / XSS Match	Detect malicious code	Prevent injection attacks
Bot Control	Detect and block bots	Stop automated scraping

Action	Meaning
ALLOW	Pass traffic through
BLOCK	Deny the request
COUNT	Log the request only (used for testing rules)
CAPTCHA / Challenge	Force the user to pass a CAPTCHA (bot mitigation)

Rules are **evaluated in order** (priority number).
When a rule matches, its **action** is executed.
If no rules match, the **default action** of the Web ACL is applied.

✅ **Default actions:**

- **ALLOW** all traffic
- **BLOCK** all traffic

🧩 CloudFront + WAF

Protects globally distributed websites at the edge.
Best for: Global latency-sensitive apps.

🧩 ALB + WAF

Protects backend web apps regionally.
Best for: Internal corporate apps or APIs behind ALB.

🧩 API Gateway + WAF

Protects REST APIs from abuse or injection.
Best for: Serverless API protection.

Scenario	WAF Feature Used
Block IPs after 1000 requests in 5 min	Rate-based rule

Scenario	WAF Feature Used
Block traffic from China	Geo match rule
Allow only requests containing specific header	String match rule
Stop SQL/XSS attacks	AWS Managed Rules
Prevent scraping bots	CAPTCHA / Bot Control
Test new rules safely	COUNT action
Analyze traffic patterns	WAF logs via Kinesis

Scenario:

A developer hosts a REST API on API Gateway that is being spammed with requests.

✓ Solution:

- 1 Create a Web ACL.
 - 2 Add a **Rate-based rule** (1000 requests / 5 min).
 - 3 Attach the Web ACL to API Gateway.
 - 4 Monitor logs via CloudWatch or Firehose.
- ✓ **Result:** Requests exceeding threshold → automatically blocked.

AWS Amplify:

AWS Amplify is a **development platform** that makes it easier to build, configure, and deploy **full-stack apps** powered by AWS services.

It provides:

- A **CLI & SDK** to configure cloud backends (Auth, API, Storage, etc.)
- A **UI library** to connect your frontend easily
- **Hosting** for web apps (frontend + backend)

Component	Description	Exam Keywords
Amplify CLI	Configure & deploy AWS backend resources via command line	“amplify add auth”, “amplify push”
Amplify Libraries (SDKs)	Frontend SDKs (JS, Android, iOS, Flutter) for connecting to backend	“import { Auth } from ‘aws-amplify’”
Amplify UI Components	Prebuilt React/Vue components (e.g., Authenticator)	“Add login UI quickly”
Amplify Hosting	Static web hosting (build + deploy pipeline)	“Continuous deployment from GitHub”

Component	Description	Exam Keywords
Amplify Studio	Visual development environment for frontend + data	“Low-code interface”

Amplify Category	Underlying AWS Service
Auth	Amazon Cognito
API (REST)	Amazon API Gateway + AWS Lambda
API (GraphQL)	AWS AppSync
Storage	Amazon S3
Database	DynamoDB (via AppSync resolvers)
Functions	AWS Lambda
Hosting	CloudFront + S3
Notifications	Amazon Pinpoint / SNS

Typical developer workflow:

```

amplify init # initialize Amplify project
amplify add auth # add Cognito authentication
amplify add api # add REST or GraphQL API
amplify add storage # add S3 storage
amplify push # deploy backend to AWS

```

Amplify Hosting provides **CI/CD for static web apps** (React, Vue, Angular, Next.js, etc.).

- Continuous deployment from **GitHub, GitLab, Bitbucket, or CodeCommit**
- Custom domains + HTTPS (ACM certificates)
- Backend environments (dev/stage/prod)
- Branch-based deploys (auto previews for PRs)

Amplify Studio is a **visual UI builder** that connects to your backend data models and authentication.

- Lets you drag-and-drop UI components (like tables, forms, auth flows)
- Generates React code automatically
- Integrates with **Figma** designs
- Still allows developers to edit generated code manually

Amplify Auth uses **Amazon Cognito** under the hood.

Amplify Storage = **Amazon S3 integration**.

REST API

- Uses **API Gateway + Lambda**
- `amplify add api` → prompts you for REST config

GraphQL API

- Uses **AWS AppSync**
- Auto-generates schema and resolvers for DynamoDB
- Supports real-time subscriptions

Layer	Underlying AWS Service
Frontend	S3 (hosting) + CloudFront (CDN)
Backend	CloudFormation stack
APIs	API Gateway / AppSync
Auth	Cognito
Functions	Lambda
Storage	S3 / DynamoDB

If the question says “*developer wants to add user authentication, APIs, and storage with minimal setup*” → **AWS Amplify**.

If “*deploy static frontend with backend, continuous updates on git push*” → **Amplify Hosting**.

If “*low-code tool to visually design UI bound to backend data*” → **Amplify Studio**.

If “*developer wants to add user sign-in quickly to React app*” → Amplify Auth + Cognito.

If “*developer wants real-time data sync in React app*” → **Amplify + AppSync**.

If “*user uploads files through the frontend securely*” → Amplify Storage (S3 + Cognito identity).

If “*developer wants to add custom backend logic without manual Lambda setup*” → **Amplify Function**

If “*team needs isolated backends for dev and prod*” → **Amplify environments**.

If “*Amplify hosting pipeline deploys both frontend and backend*” → it uses **CloudFront + S3 + CloudFormation**.

If “*frontend users access S3 via Amplify*” → they get **temporary credentials via STS** through **Cognito**.

If “*frontend-centric app with CI/CD and serverless backend*” → **Amplify**.

If “*backend-centric app (Node, Python) deployment*” → **Elastic Beanstalk**.

AWS CloudShell:

AWS CloudShell is a **browser-based, pre-authenticated shell environment** that lets you run **AWS CLI commands**, **Python scripts**, and **shell tools** directly from the AWS Management Console — *without installing or configuring anything locally*.

Feature	Description
Browser-based shell	Access directly from AWS Console — no setup required
Pre-authenticated	Uses your console login credentials (IAM user or role)
Persistent storage	1 GB of home directory storage per Region
AWS CLI preinstalled	Latest AWS CLI v2 + AWS SDKs (Python, Node.js, etc.)
Linux tools	Includes Git, Python, Node.js, Bash, jq, pip, zip/unzip, etc.
Multi-region support	Separate environment per AWS Region
No extra cost	Free — you pay only for the resources you interact with

Use Case	Example
Test AWS CLI commands	<code>aws ec2 describe-instances</code>
Manage AWS resources quickly	Create/delete S3 buckets, Lambda functions
Run SDK scripts	Python (boto3) or Node.js code
Clone repos from CodeCommit / GitHub	<code>git clone</code> then deploy with CLI
Edit & upload files	Drag-and-drop from browser, use <code>nano</code> or <code>vim</code>
Deploy infrastructure	Run CloudFormation or CDK commands
Debug IAM / permissions	Test assumed roles with CLI
Access same environment across Regions	Each Region has its own persistent <code>/home/cloudshell-user</code> directory

Installed Tools

Tool	Version / Purpose
AWS CLI v2	Preinstalled and configured
Python 3 + boto3	AWS SDK for scripting
Node.js + npm	Run JavaScript SDK apps
Git	Clone/push repositories
Terraform & AWS CDK (some regions)	Infrastructure as code
jq, unzip, curl, wget	Data manipulation and HTTP tools

- You get **1 GB of persistent storage per Region** under `/home/cloudshell-user/`.
- Files stored here **persist between sessions**, but only within the same Region.
- Temporary directories (e.g., `/tmp`) are **ephemeral** and cleared after session ends.

Limitations:

Limitation	Description
Timeout	Session auto-closes after ~20–30 min of inactivity
No VPC access	Cannot access private subnets/resources without public endpoints
Storage limit	1 GB only, Region-specific
No GUI apps	Command-line only
Limited compute power	Intended for CLI tasks, not builds/tests

Use Case	Example
Quick CLI tests	Test <code>aws</code> commands without setup
View CloudWatch logs	<code>aws logs tail /aws/lambda/MyFunc</code>
Git pull & deploy Lambda	Clone repo and use <code>aws lambda update-function-code</code>
Run CDK/CloudFormation deploys	<code>cdk deploy</code> or <code>aws cloudformation deploy</code>
Debug IAM access	Use <code>aws sts get-caller-identity</code>
Manage S3 or DynamoDB	Quick queries and data inspection

If you see “*developer needs to quickly run AWS CLI commands without setup or credentials file*” → answer **AWS CloudShell**.

If “*developer saved script in CloudShell us-east-1 but can’t find it in us-west-2*” → persistence is **Region-specific**.

If “*CloudShell command fails due to permission error*” → check IAM policy, not CloudShell config.

If “*CloudShell can’t reach RDS in private subnet*” → that’s expected — it doesn’t run inside your VPC.

Amazon MemoryDB:

Amazon MemoryDB for Redis is a **durable, in-memory database service** that’s **compatible with Redis**.

It combines the **speed of an in-memory cache** with the **durability and consistency of a database**.

Feature	MemoryDB for Redis	ElastiCache for Redis
Primary Purpose	Durable, in-memory database	Fast, in-memory cache
Persistence	✅ Data stored durably in multi-AZ cluster volume	⚠️ Optional snapshotting (mostly ephemeral)
Storage Type	Multi-AZ transactional log-based (distributed)	In-memory only (RAM)
Use Case	System-of-record data store	Caching layer
Durability	Designed for 99.99% durability	Data may be lost on node failure
Consistency	Strong consistency (replicated across AZs)	Eventually consistent (async replicas)
Engine Compatibility	Redis 6.x compatible	Redis 6.x compatible
Failover	Automatic, Multi-AZ	Automatic (with primary-replica setup)
Backups	Continuous, durable log-based backups	Optional snapshots

- **In-Memory Primary Cluster:** Fast reads/writes served from memory.
- **Transactional Log (Distributed Storage Layer):** All writes are durably stored across multiple Availability Zones.
- **Replicas:** Read replicas improve scalability and failover.

Use Case	Description
Session Stores	Persistent web session data for apps
Gaming Leaderboards	High-speed, durable in-memory operations
E-commerce carts	Fast read/write, no data loss after restart
Real-time analytics	In-memory computations with persistence
Microservice coordination	Shared durable state between services

MemoryDB durability is achieved by:

- **Synchronous replication** across **multiple AZs**.
- **Distributed transactional log** stored on **SSD-backed storage**.
- Every write operation = **recorded and acknowledged** only after being stored durably.

Scaling Type	Description
Vertical Scaling	Change node type for more memory/CPU
Horizontal Scaling	Add shards to distribute key space
Read Scaling	Add replicas for read-heavy workloads

- **Automatic continuous backups** stored durably in **multi-AZ distributed storage**.
- **Point-in-time recovery (PITR)** supported (similar to RDS).
- **Manual snapshots** also possible.

Feature	MemoryDB	DynamoDB + DAX
Purpose	In-memory database	In-memory cache for DynamoDB
Durability	✅ Persistent (database of record)	❌ Ephemeral (cache only)
Interface	Redis API	DynamoDB API
Use Case	Real-time durable DB	Cache to reduce DynamoDB latency

If the question says “store session state with high availability and data durability even after a node failure” → **MemoryDB**.

If it says “temporary cache to reduce latency” → **ElastiCache**.

If “in-memory performance required but cannot afford data loss” → **MemoryDB**.

If “cache layer only for performance optimization” → **ElastiCache**.

If the cluster fails, MemoryDB can **rebuild from durable log**, ensuring **no data loss**.

If “developer wants to migrate from open-source Redis with durability” → **MemoryDB (Redis compatible)**.

If “must encrypt data in transit and at rest by default” → **MemoryDB** (security-first design).

If “restore MemoryDB to a specific time before data corruption” → **PITR** supported.

ElastiCache does **not** offer this level of durability.

If “developer must use IAM + KMS to protect in-memory database data” → **MemoryDB** supports both.

If “serverless app requires Redis-compatible durable data store” → **MemoryDB** is the answer.

If “Redis API required” → **MemoryDB**.

If “accelerate DynamoDB reads” → **DynamoDB DAX**.

Amazon Aurora:

Amazon Aurora is a **fully managed relational database** service under **Amazon RDS (Relational Database Service)** that is **MySQL- and PostgreSQL-compatible** — but up to **5×**

faster than MySQL and **3× faster than PostgreSQL**, thanks to a **custom distributed storage engine** built by AWS.

Feature	Amazon Aurora	RDS (MySQL/PostgreSQL)
Storage	Distributed, auto-scaling storage up to 128 TB	Fixed-size EBS volumes
Availability	6-way replication across 3 AZs	Typically single-AZ or multi-AZ
Performance	Up to 5× faster (MySQL) / 3× faster (PostgreSQL)	Standard performance
Failover	30 seconds or less	Up to minutes
Backups	Continuous + PITR	Daily snapshots + PITR
Cost	Slightly higher	Lower
Replication	Aurora Replicas (reader endpoints)	Read Replicas (slower async)
Serverless Option	✅ Aurora Serverless	❌ No native serverless
Global Option	✅ Aurora Global Database	⚠️ Limited (manual cross-region)

Aurora separates **compute** from **storage**.

- **Storage layer** = distributed, replicated across 3 AZs (6 copies total).
- **Compute layer** = DB instances (writer + replicas).
- Storage auto-scales in **10 GB increments** up to **128 TB**.
- **Fault-tolerant**: writes committed only after majority (4 of 6) confirmation.
- **Distributed**:
Aurora's data is **split into chunks (10 GB segments)** and **replicated six times** across **three Availability Zones**.
This gives high durability — even if an AZ fails, the data is safe elsewhere.
- **Auto-scaling**:
You **don't need to pre-allocate storage** (like in RDS).
Aurora automatically adds more storage **as your data grows**, in 10 GB increments, up to a maximum of **128 TB** — without downtime.

Component	Description
Primary Instance (Writer)	Handles all write operations
Aurora Replicas (Readers)	Read-only replicas for scaling
Cluster Volume	Shared distributed storage across AZs

Component	Description
Endpoints	Simplify connection management

Endpoint Type	Function
Cluster Endpoint (Writer)	Always points to the current writer
Reader Endpoint	Load-balances across replicas for read queries
Custom Endpoint	User-defined subset of instances

- Up to **15 replicas per cluster**.
- Replicas share the same underlying storage volume — **no replication lag**.
- Failover happens **automatically** to a replica if primary fails.

Aurora Global Database:

- Replicates data **across regions** with **<1 second latency**.
- Used for **disaster recovery** or **low-latency global reads**.
- Read replicas in secondary regions can be promoted for DR.

Version	Description
Aurora Serverless v1	Auto-scales compute capacity (ACUs), ideal for intermittent workloads
Aurora Serverless v2	Scales seamlessly in fine-grained increments, supports multi-AZ

Automatically:

- Starts when queried
- Scales capacity based on load
- Pauses when idle (you only pay when active)

Security Feature	Description
Encryption at Rest	KMS-integrated
Encryption in Transit	SSL/TLS
IAM Authentication	IAM tokens instead of DB passwords
Network Isolation	Run inside VPC
Audit Logging	via CloudWatch or CloudTrail

Type	Description
Read Scaling	Add Aurora Replicas
Write Scaling	One writer per cluster (v2 supports scaling improvements)
Storage Scaling	Auto up to 128 TB
Serverless Scaling	Dynamic compute capacity (ACUs)

Failover Process:

- 1 Primary instance fails.
- 2 Aurora automatically promotes the best replica.
- 3 Cluster endpoint points to new writer.
- 4 Minimal downtime (<30 s).

Feature	Aurora	DynamoDB
Type	Relational	NoSQL
Schema	Fixed schema (SQL)	Schema-less (Key-Value)
Transactions	ACID	ACID (with Transaction API)
Scaling	Vertical + Read Replicas	Fully horizontal
Use Case	Complex queries, joins	Key-value access, scale-out apps

If you see “*relational database with high performance, automatic scaling, and multi-AZ durability*” → **Aurora**.

If “*data replicated automatically across multiple AZs for durability*” → that’s **Aurora**, not standard RDS.

If “*automatically route read/write queries to correct instance*” → use **Reader and Cluster endpoints**.

If “*RDS database must support cross-region disaster recovery with low latency*” → **Aurora Global Database**.

If “*application has unpredictable or infrequent database usage*” → **Aurora Serverless**.

If “*recover Aurora DB to exact time before error*” → **PITR**.

If “*long-term backup storage*” → **manual snapshots**.

If “*developer must connect using IAM credentials instead of static passwords*” → **Aurora IAM Auth**.

If “*database must fail over automatically to standby within 30 seconds*” → **Aurora**.

If “*Lambda functions need persistent DB connections efficiently*” → **RDS Proxy + Aurora**.

If “*Lambda function must query Aurora Serverless without managing connections*” → use **Aurora Data API**.

If “*relational schema + joins + durability*” → Aurora.

If “*massive scale, key-value*” → DynamoDB.

AWS Copilot:

AWS Copilot is an **open-source command-line tool** that simplifies **building, releasing, and operating containerized applications** on **Amazon ECS (Elastic Container Service)** and **AWS Fargate**.

Copilot helps you **go from Dockerfile → production** with **a few CLI commands**.

It automates:

- ECS cluster creation
- VPC, subnets, and networking setup
- Load balancer configuration
- CI/CD pipelines (CodePipeline + CodeBuild)
- Service scaling and monitoring

1 You start with a Dockerized app.

2 Run `copilot init` — Copilot:

- Detects your Dockerfile
- Asks what type of service (web, worker, job, etc.)
- Creates ECS task definitions, ALBs, and required infrastructure

3 Run `copilot deploy` — it:

- Builds and pushes your Docker image to ECR
- Deploys your app to ECS (on Fargate by default)
- Creates CloudFormation stacks for environments and services

Concept	Description
Application	Logical group of microservices deployed together
Environment	Separate stage (dev, test, prod) in a VPC
Service	A long-running task (web, backend, worker)
Job	One-time or scheduled ECS task
Pipeline	CI/CD pipeline for auto-deploying new versions

Command	Purpose
<code>copilot init</code>	Initialize new app (detects Dockerfile)
<code>copilot env init</code>	Create a new environment (e.g., prod, test)

Command	Purpose
<code>copilot deploy</code>	Deploy app to ECS Fargate
<code>copilot svc show</code>	Show service details
<code>copilot pipeline init</code>	Create CI/CD pipeline
<code>copilot job run</code>	Run one-off or scheduled task
<code>copilot storage init</code>	Attach S3 or DynamoDB storage
<code>copilot app delete</code>	Clean up all resources

Copilot automatically:

- Creates **CodePipeline + CodeBuild**
- Connects your **GitHub/CodeCommit repo**
- Deploys to **dev** → **prod** environments after tests

AWS AppSync:

AWS AppSync is a **fully managed service for building GraphQL APIs** that connects your application to multiple AWS data sources — such as **DynamoDB, Lambda, RDS**, or **Elasticsearch/OpenSearch** — and keeps data **real-time and synchronized** across devices.

Capability	Description
GraphQL API	You define schema and resolvers; AppSync handles queries, mutations, subscriptions
Data Sources	Connects to DynamoDB, Lambda, RDS (via Aurora), HTTP APIs, and OpenSearch
Real-Time Subscriptions	Clients can subscribe to live data changes (WebSocket-based)
Offline Sync	Works with Amplify SDK to sync when device reconnects
Authorization	Supports multiple auth modes — Cognito, IAM, API Key, OpenID Connect
Caching	Built-in API-level caching to reduce backend load
Security	Integrated with IAM + Cognito; supports fine-grained access
Monitoring	CloudWatch metrics, logs, X-Ray tracing

- 1 **Define a GraphQL schema** (types, queries, mutations, subscriptions).
- 2 **Connect resolvers** to AWS data sources (DynamoDB, Lambda, etc.).
- 3 **Clients** (web/mobile apps) use GraphQL queries to fetch or mutate data.

4 AppSync **resolves and aggregates** data from one or many sources automatically.

5 Optionally, clients subscribe to **real-time updates** (using GraphQL subscriptions).

Data Source	Common Use
DynamoDB	Direct, fast read/write access (NoSQL backend)
Lambda	Custom business logic or access to multiple services
RDS (Aurora Serverless)	For SQL data queries
OpenSearch Service	Full-text search APIs
HTTP Endpoint	Connect external REST APIs

- AppSync has **built-in caching layer** (using DynamoDB DAX-like service).
- You can enable **per-field caching** for resolvers.
- Reduces DynamoDB read capacity usage and latency.

Feature	AppSync	API Gateway
API Type	GraphQL	REST, WebSocket, HTTP
Real-time	Built-in (subscriptions)	WebSocket APIs only
Multiple Data Sources	Yes (DynamoDB, Lambda, etc.)	Usually one per route
Offline Sync	Yes (Amplify DataStore)	No
Schema-based	Yes (GraphQL Schema)	No
Best for	Modern apps, data aggregation	REST-based microservices

Use Case	Description
Mobile apps	Real-time + offline-ready APIs
Dashboards	Combine multiple data sources
Chat apps	Real-time messaging via subscriptions
E-commerce	GraphQL to aggregate products, orders, users
IoT	Live device telemetry updates

If the question says “*developer needs to create a real-time API that integrates multiple data sources*” → **AppSync** is the answer.

If “*developer needs GraphQL API integrating DynamoDB and Lambda*” → **AppSync**.

If “*needs REST API with transformations*” → **API Gateway**.

If “*AppSync API needs to be accessed by both public users (API key) and authenticated users (Cognito)*” → **multi-auth mode**.

If “reduce read load and latency in GraphQL API” → **AppSync Caching**.

If “real-time updates to multiple connected clients when data changes” → **AppSync Subscription**.

If “offline-first mobile app with data sync to backend” → **AppSync + Amplify DataStore**.

If “user-based access control in GraphQL schema” → use **Cognito groups + resolver authorization**.

If “developer wants to trace AppSync resolver performance” → **AWS X-Ray**.

If “developer wants to query multiple backends in one request” → **AppSync**.

If “needs REST API for microservices” → **API Gateway**.

If “Amplify project adds a GraphQL API” → that’s **AppSync** under the hood.

AWS Serverless Application Model (SAM):

AWS SAM (Serverless Application Model) is an **open-source framework** for building, testing, and deploying **serverless applications** on AWS.

It extends **AWS CloudFormation** with **simplified syntax** for defining serverless resources like:

- AWS Lambda functions
- Amazon API Gateway APIs
- Amazon DynamoDB tables
- Amazon S3 buckets
- AWS Step Functions
- EventBridge, SNS, and SQS triggers

Feature	AWS SAM	AWS CloudFormation
Purpose	Build & deploy serverless apps	Define any AWS infrastructure
Syntax	Simplified (shortcuts for Lambda, API Gateway, etc.)	Verbose, YAML/JSON
Engine	Runs on CloudFormation	Native CloudFormation
Local Testing	✅ Yes (SAM CLI)	❌ No
Deploy Command	<code>sam deploy</code>	<code>aws cloudformation deploy</code>
Template File	<code>template.yaml</code>	<code>template.yaml</code>
Packaging	Automatically uploads Lambda code to S3	Must be done manually

SAM templates are written in **YAML** and use the **AWS::Serverless transform**.

Example header:

```
Transform: AWS::Serverless-2016-10-31
```



Basic Template Example

yaml

Copy code

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: Simple serverless API

Resources:
  MyFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler: app.lambda_handler
      Runtime: python3.9
      CodeUri: src/
    Events:
      ApiEvent:
        Type: Api
        Properties:
          Path: /hello
          Method: get
```

✓ This single template:

- Creates a **Lambda function**
- Creates an **API Gateway endpoint (/hello)**
- Connects the two automatically



Resource	Description
AWS::Serverless::Function	Defines a Lambda function
AWS::Serverless::Api	Defines an API Gateway REST API
AWS::Serverless::HttpApi	Defines an HTTP API (v2)
AWS::Serverless::SimpleTable	Creates a DynamoDB table
AWS::Serverless::StateMachine	Defines a Step Functions state machine
AWS::Serverless::LayerVersion	Defines a Lambda Layer
AWS::Serverless::Application	References another SAM/CFN stack

Command	Description
<code>sam init</code>	Initialize a new SAM project (creates sample template + code)
<code>sam build</code>	Package dependencies and prepare for deployment
<code>sam local invoke</code>	Run Lambda locally with test events

Command	Description
<code>sam local start-api</code>	Run entire API locally (simulates API Gateway)
<code>sam package</code>	Upload Lambda code to S3 and package the template
<code>sam deploy</code>	Deploys the packaged stack to AWS
<code>sam logs -n <function></code>	View CloudWatch logs
<code>sam delete</code>	Deletes the CloudFormation stack and related resources

SAM lets you define **environment variables** and **permissions** directly:

```

yaml
Properties:
  Environment:
    Variables:
      TABLE_NAME: MyTable
Policies:
  - DynamoDBCrudPolicy:
      TableName: !Ref MyTable
  
```

 Copy code

SAM provides **managed policy templates** for common permissions — no need to write full IAM policies.

Examples:

- `AmazonDynamoDBFullAccess`
- `S3ReadPolicy`
- `S3CrudPolicy`
- `SQSPollerPolicy`
- `LambdaInvokePolicy`
- `CloudWatchPutMetricPolicy`

SAM allows **local simulation** of Lambda + API Gateway: `sam local start-api`

SAM integrates seamlessly with:

- **CodePipeline / CodeBuild**
- **GitHub Actions**
- **Bitbucket Pipelines**

SAM Accelerate (`--watch` flag) enables **incremental, near real-time deployments** — speeding up the dev cycle.

If “developer wants to test Lambda locally and deploy via IaC” → **AWS SAM**.

If “define general AWS infrastructure (VPCs, EC2)” → **CloudFormation**.

If “serverless app includes Lambda, API Gateway, and DynamoDB table” → you can define all three in **one SAM template**.

If “developer needs to test and debug Lambda locally before deployment” → use `sam local invoke`.

If “Lambda code must be uploaded to S3 before deployment” → SAM **does this automatically**.

If “Lambda needs access to DynamoDB” → use a **managed policy template** like `DynamoDBCrudPolicy`.

If “Lambda triggers when a file is uploaded to S3” → define an **S3 event source** in SAM.

If “Lambda requires permission to read/write S3” → use `S3CrudPolicy`.

If “multiple functions share dependencies” → use **Lambda Layers**.

If “developer needs to test API locally without deploying to AWS” → **SAM Local**.

If “developer wants faster code updates during development” → use **SAM Accelerate**.

If “developer wants to automate serverless deployments via pipeline” → **SAM CLI in CodeBuild**.

If “developer wants code-defined infra” → CDK.

If “YAML-based infra with serverless focus” → SAM.

If “deployment failed with rollback” → check **CloudFormation events**.

If “developer asks about SAM cost” → SAM = free, **you only pay for the AWS resources**.

AWS ECR:

Amazon Elastic Container Registry (ECR) is a **fully managed Docker-compatible container image registry** that lets you **store, manage, share, and deploy container images** securely.

Feature	Description
Private & Public Repositories	Store container images privately or publicly (ECR Public Gallery)
Fully Managed	No servers — AWS manages scaling, storage, and security
Integration	Works with ECS, EKS, Fargate, Lambda (container images), and CodeBuild
Image Scanning	Detects vulnerabilities using Amazon Inspector
Encryption	Images are encrypted at rest (KMS) and in transit (HTTPS)
Lifecycle Policies	Auto-delete old/unneeded images
Cross-Region Replication	Replicate repositories for latency or DR
Authentication	Uses IAM or temporary auth tokens
Immutable Tags	Prevents tag overwriting for version control

Type	Description
Private Repository	Default; accessible only via IAM-based auth
Public Repository	Publicly shareable; hosted in <code>gallery.ecr.aws</code>
Cross-Account Access	Grant permissions via repository policy

Mechanism	Description
IAM Policies	Control push/pull/delete actions
Repository Policies	Fine-grained access (cross-account)
ECR Authorization Token	Used for Docker login (valid 12 hours)
Encryption at Rest	AWS-managed KMS key (default)
Encryption in Transit	HTTPS endpoints
Image Scanning	Finds vulnerabilities using Amazon Inspector
VPC Endpoints	Access ECR privately (no public internet)

- Each image can have multiple **tags** (e.g., `v1` , `latest`).
- Tags are just **aliases** — deleting a tag doesn't delete the image.
- Enable **immutable tags** to prevent overwriting (production best practice).
Lifecycle policies let you **automatically delete unused images** to save cost.

Cross Region Replication:

- Automatically **replicates images** to other AWS Regions.
- Ensures **low-latency pulls** and **regional availability**.
- Use **replication rules** in repository settings.

Service	Integration with ECR
ECS / Fargate	Pulls images directly from ECR
EKS (Kubernetes)	Pods reference ECR image URIs
Lambda	Supports container image deployment (≤ 10 GB)
CodeBuild	Builds and pushes images to ECR
CodePipeline	Automates CI/CD for containers
Copilot CLI	Uses ECR automatically when deploying to ECS
App Runner	Pulls images directly from ECR
Cloud9 / EC2	Developers can push/pull via Docker CLI

Action	Description
<code>ecr:CreateRepository</code>	Create repo
<code>ecr:PutImage</code>	Push image
<code>ecr:GetDownloadUrlForLayer</code>	Pull image
<code>ecr:BatchCheckLayerAvailability</code>	Validate image layers
<code>ecr:DescribeRepositories</code>	List repositories
<code>ecr>DeleteRepository</code>	Delete repo
<code>ecr:GetAuthorizationToken</code>	Authenticate docker client

Public ECR:

- Hosted at gallery.ecr.aws.
- Anyone can **pull images** without authentication.
- Useful for sharing public container images.
- Integrated with **CloudFront CDN** for global delivery.

Issue	Possible Cause
no basic auth credentials	Not logged in → run <code>aws ecr get-login-password</code>
ECS task fails to start	IAM role lacks <code>ecr:GetAuthorizationToken</code> or <code>ecr:GetDownloadUrlForLayer</code>
Cannot push image	Repo policy denies <code>PutImage</code>
Slow pull	Missing cross-region replication or private link configuration

A developer wants to automatically scan Docker images for vulnerabilities when they are pushed to AWS.

✅ **Answer:** Enable **ECR image scanning on push**.

A Lambda function packaged as a container image (7 GB) must be deployed.

✅ **Answer:** Push image to **ECR** and reference its URI in Lambda.

Two AWS accounts need to share container images securely.

✅ **Answer:** Use **ECR repository policy** to grant cross-account access.

Developer wants old, unused image versions deleted automatically.

✅ **Answer:** Configure an **ECR lifecycle policy**.

ECS tasks cannot pull images from ECR inside a private subnet.

✅ **Answer:** Create **VPC interface endpoints** for ECR and S3.

If “developer needs a secure, private registry for Docker images integrated with ECS or EKS” → **ECR** is the correct answer.

If “Lambda or ECS needs to retrieve image automatically” → use the **ECR image URI**.

If “developer needs to share container images across accounts securely” → use **repository policy (JSON IAM policy)**.

If “developer needs private access to ECR from VPC” → configure **ECR VPC Endpoint** (Interface endpoint).

If “developer wants to detect vulnerabilities in Docker images automatically after push” → **enable image scanning on push**.

If “prevent overwriting existing image versions” → use **immutable tags**.

If “repository is full of old images” → configure an **ECR lifecycle policy**.

If “disaster recovery or multi-region container pull performance required” → use **cross-region replication**.

If “ECS task fails because it can’t pull image” → likely **ECR permission or login issue**.

If “AccessDenied when docker push to ECR” → check `ecr:PutImage` permission.

If “developer wants to share container images publicly without using Docker Hub” → use **ECR Public**.

Amazon ECS:

Amazon ECS (Elastic Container Service) is a **fully managed container orchestration service** that helps you **run, stop, and manage Docker containers** on AWS — either on **EC2 instances** or **AWS Fargate** (serverless).

Launch Type	Description	Key Points
Fargate	<i>Serverless containers</i> — AWS manages infrastructure	You don’t manage EC2 or scaling. Pay per vCPU & memory.
EC2	You manage EC2 instances (container hosts)	You must patch, scale, and manage instances.
External (ECS Anywhere)	Run ECS tasks on on-prem or other clouds	Connects via SSM Agent — hybrid model.

Component	Description	Example
Cluster	Logical grouping of ECS resources (Fargate or EC2)	“prod-cluster”
Task Definition	Blueprint describing container(s), ports, memory, etc.	“myapp-task:v3”
Task	Running instance of a task definition	1 running container app

Component	Description	Example
Service	Ensures tasks are always running (desired count)	“web-service”
Container Agent	Runs on EC2 instances to manage tasks	Installed automatically
Scheduler	Decides where tasks run (based on resources)	“place 2 tasks on cluster”

A **Task Definition** is a **JSON blueprint** that defines:

- Docker image (from ECR or Docker Hub)
- CPU & memory
- Environment variables
- Networking mode (bridge, awsvpc, host)
- IAM roles
- Logging configuration

ECS **Service** ensures your **tasks stay running** and supports:

- **Load balancing** (ALB/NLB)
- **Auto Scaling**
- **Health checks**
- **Rolling updates**

Network Mode	Description	Used With
awsvpc	Each task gets its own ENI, IP address	Fargate, EC2
bridge	Docker default — shared host port mapping	EC2 only
host	Containers share host’s network stack	EC2 only
none	No network access	Special use cases

Role	Purpose	Required For
Task Role	Permissions for the container itself	When container needs AWS access (S3, DynamoDB, etc.)
Execution Role	Permissions for ECS to start tasks (pull images, write logs)	Required for Fargate tasks
Service Role	Permissions for ECS Service to manage scaling and load balancers	ECS service-level ops

ECS Anywhere:

- Run ECS tasks on **on-prem servers** or **other clouds**.
- Uses **Systems Manager (SSM Agent)** for control and registration.
- Hybrid architecture.

Error	Root Cause
CannotPullContainerError	Missing ECR permissions or wrong image URI
AccessDeniedException	Missing task or execution role permissions
Task stopped immediately	App exited (non-0 code) or missing health check
Service stuck in “steady state”	Desired count misconfigured

If “*developer wants to run containers without managing servers*” → **Fargate**.

If “*needs custom AMI, GPU, or specialized networking*” → **EC2 launch type**.

If “*ECS must ensure N containers are always running*” → that’s an **ECS Service**.

If “*developer modifies container image, env vars, or memory limits*” → must **register a new task definition revision**.

If “*service must always maintain 3 running tasks even after failure*” → ECS Service maintains **desired count**.

If “*Fargate task requires public IP*” → must use **awsvpc mode**.

If “*containers need dynamic port mapping*” → use **ALB + ECS + awsvpc mode**.

If “*ECS task cannot access S3 from inside container*” → **missing Task Role**.

If “*task fails to pull image from ECR*” → **missing Execution Role**.

If “*task must have a dedicated IP for security or routing*” → use **awsvpc**.

If “*ECS task fails to start due to missing ECR access*” → check **task execution role**.

“A developer wants each ECS task to have its own IP address and be directly associated with a security group.”

✅ **Answer:** Use **awsvpc** network mode.

“An ECS EC2 service runs multiple containers on the same host, each listening on port 8080.”

✅ **Answer:** Use **bridge** mode with dynamic port mapping.

“A gaming company runs UDP-based game servers that need ultra-low latency networking.”

✅ **Answer:** Use **host** network mode.

“A data-processing ECS task should have no inbound or outbound internet access.”

✅ **Answer:** Use **none** network mode.

AWS EKS:

Amazon Elastic Kubernetes Service (EKS) is a **fully managed Kubernetes control plane** that lets you **run containerized applications using Kubernetes** on AWS — without managing

the Kubernetes master nodes.

Control Plane (Managed by AWS)

- Runs the **Kubernetes API server**, **etcd**, and **controller manager**.
- Deployed across multiple **Availability Zones** for HA.
- AWS automatically scales and patches it.

Worker Nodes (Managed by You or AWS Fargate)

- Run your **pods and containers**.
- Can be:
 - **Managed Node Groups** → AWS creates and manages EC2 nodes.
 - **Self-managed Nodes** → You manage EC2 lifecycle.
 - **AWS Fargate** → Serverless pods, no EC2.

Option	Description	Use Case
EKS on EC2	You manage EC2 worker nodes.	Custom workloads, GPUs, DaemonSets
EKS on Fargate	Fully serverless pods, no node mgmt.	Simpler workloads
EKS Anywhere	Run EKS on-prem or other clouds.	Hybrid or multi-cloud
EKS Add-ons	AWS-managed core K8s components (CoreDNS, kube-proxy, VPC CNI).	Managed operational setup

Component	Description
Cluster	The control plane and worker nodes together
Node Group	Set of EC2 instances managed as Kubernetes nodes
Pod	Smallest deployable unit — runs one or more containers
Service	Exposes pods internally or externally (LoadBalancer, ClusterIP, NodePort)
Deployment	Manages rolling updates, replicas
Ingress	HTTP routing layer (often backed by ALB Ingress Controller)
ConfigMap & Secret	Store config and credentials
kubectl CLI	Main Kubernetes client for interacting with EKS

Component	Description
IAM Roles for Service Accounts (IRSA)	Maps AWS IAM permissions to Kubernetes service accounts

IRSA = IAM Role + Kubernetes Service Account mapping.

Instead of giving node-wide IAM permissions, each pod can assume a specific IAM role via its **service account**.

- 1 Create IAM role with AWS permissions.
- 2 Annotate Kubernetes service account with that role ARN.
- 3 Pod uses that service account → gets temporary credentials via AWS STS.

Type	Description
ClusterIP	Internal-only service (default)
NodePort	Exposes pod on node IP + static port
LoadBalancer	Creates an AWS ALB or NLB
Ingress	HTTP routing via ALB Ingress Controller

EKS Add-Ons:

EKS can automatically manage key K8s components:

- **CoreDNS** → internal DNS resolution for pods
- **kube-proxy** → routes traffic
- **AWS VPC CNI** → pod networking
- **Amazon EBS CSI Driver** → persistent volumes
- **AWS Load Balancer Controller** → ALB/NLB integration

Feature	Description
IAM Integration (IRSA)	Fine-grained pod permissions
Security Groups for Pods (SGP)	Attach SG directly to pods
Network Policies	Restrict pod-to-pod traffic
Encryption	Secrets encrypted with AWS KMS
Audit Logging	Enabled via EKS control plane
Private Endpoint	Restrict API access to VPC only

“A developer needs to run Kubernetes workloads without managing control plane nodes.”

✓ **Answer:** Use **Amazon EKS**.

“Each Kubernetes pod must assume an IAM role to access S3 securely.”

✅ **Answer:** Use **IAM Roles for Service Accounts (IRSA)**.

“Developer wants to deploy Kubernetes workloads without provisioning EC2 instances.”

✅ **Answer:** Use **EKS on Fargate**.

“Kubernetes Service must expose pods publicly over HTTPS.”

✅ **Answer:** Create a **Service of type LoadBalancer** (provisions ALB/NLB).

“Pods need persistent shared storage across multiple nodes.”

✅ **Answer:** Use **EFS CSI driver**.

If a question says “*developer wants to use existing Kubernetes manifests and tools*” → **EKS**.

If it says “*developer wants a simple AWS-native container service*” → **ECS**.

If “*which service manages Kubernetes control plane for you?*” → **EKS**.

If “*developer wants to run containers on Kubernetes without managing EC2 instances*” → **EKS on Fargate**.

If “*Kubernetes pod needs AWS API access (e.g., S3)*” → use **IRSA** (IAM Role for Service Account).

If “*developer wants to integrate Kubernetes with AWS IAM for fine-grained pod permissions*” → **IRSA**.

If “*developer wants to expose a Kubernetes app to the internet*” → create a **Service of type LoadBalancer** (uses ALB/NLB).

If “*developer needs EBS volumes for pods*” → install the **EBS CSI driver**.

If “*multiple pods across nodes need shared storage*” → use **EFS CSI driver**.

If “*developer wants to monitor container CPU/memory usage*” → **CloudWatch Container Insights**.

If “*developer wants to deploy containers using Kubernetes manifests*” → use **EKS + kubectl**.

If “*EKS pods should run without managing EC2 nodes*” → use **EKS on Fargate**.

If “*restrict access to EKS control plane to VPC only*” → **Private Endpoint**.

If “*developer wants to deploy Kubernetes manifests as part of pipeline*” → **CodeBuild + kubectl apply**

If “*EKS pricing*” → pay for **control plane + compute resources**, unlike ECS (control plane free).

AWS Systems Manager:

AWS Systems Manager (SSM) is a **unified management service** that helps you **view, control, automate, and secure your AWS and on-premise infrastructure**.

Developers interact with SSM mainly for:

- **Parameter Store** → storing app configuration/secrets
- **AppConfig** → deploying runtime configs safely
- **Run Command / Automation** → automating scripts on EC2
- **Session Manager** → connecting to EC2 securely (no SSH keys)

- **Inventory / Patch Manager** → less tested, but good to know

SSM Feature	Description	Exam Relevance
Parameter Store	Store and retrieve config values & secrets	★ Very High
AppConfig	Safe config deployment with validation	★ Very High
Run Command	Run shell commands/scripts remotely on instances	★ Medium
Session Manager	Secure browser/CLI access to EC2 (no SSH)	★ Medium
Automation	Run multi-step workflows (patching, snapshots, etc.)	Medium
Inventory / Patch Manager	Track and patch instance software	Low
State Manager	Enforce configuration compliance	Low
OpsCenter	Centralize operational issues	Low

AWS SSM Parameter Store securely stores configuration data and secrets as **key-value pairs**, such as:

- Database connection strings
- API keys
- Environment variables
- Feature flags

Tier	Description
Standard	≤ 4 KB size, 10 000 parameters per account
Advanced	≤ 8 KB size, higher quota, supports policies and expiration

Feature	SSM Parameter Store	Secrets Manager
Encryption	KMS	KMS
Auto rotation	✗ No	✓ Yes
Versioning	✓ Yes	✓ Yes
Integration	EC2, Lambda, ECS	Lambda, RDS
Pricing	Free (Standard)	Paid
Best for	Configs & small secrets	Rotating DB credentials

SSM Run Command - Run remote shell commands or scripts across EC2, on-prem, or hybrid instances without SSH. **Use Cases:**

- Restart apps
- Pull code updates
- Apply patches or scripts

Session Manager - Connect to EC2 securely **without SSH keys, bastion hosts, or open ports.**

Feature	Description
Browser / CLI access	Start session via AWS Console or CLI
Audit logs	Commands logged to CloudWatch or S3
IAM-based access	No inbound SSH required
Encryption	All traffic encrypted by SSM Agent

SSM Automation - Automates multi-step operational tasks using **Automation Documents (runbooks)** — JSON/YAML scripts executed by SSM.

Examples:

- Stop/start EC2 on schedule
- Create AMI backups
- Patch instances

SSM Agent:

- Small agent installed on EC2, on-prem, or hybrid servers.
- Communicates with SSM service endpoints.
- Required for Run Command, Session Manager, Patch Manager, etc.

State Manager - Keeps instances in a **desired configuration state** (install software, maintain compliance).

Example:

Ensure all EC2 instances have CloudWatch Agent installed.

A Lambda function must access database credentials securely without storing them in code.

✅ **Answer:** Use **SSM Parameter Store (SecureString + KMS)**.

An EC2 instance must be accessed securely without SSH key pairs.

✅ **Answer:** Use **Session Manager**.

Developer wants to execute a shell script on all EC2 instances nightly.

✅ **Answer:** Use **Run Command** or **Automation** with schedule.

Config changes must roll out gradually across environments with rollback on errors.

✅ **Answer:** **AppConfig**.

Instance fails to receive SSM commands.

✅ **Answer:** Check **SSM Agent installation** and **IAM role permissions**.

If “*developer needs to securely store configuration data or secrets without hardcoding*” → **SSM Parameter Store**.

If “*needs to automate updates or configuration scripts on EC2*” → **SSM Run Command**.

If “*store and encrypt secrets such as API keys*” → use **SecureString + KMS**.

If “*Lambda function must read secrets securely*” → use **SSM Parameter Store (SecureString)** with `kms:Decrypt` permissions.

If “*store and auto-rotate database passwords*” → **Secrets Manager**.

If “*store static app config securely*” → **Parameter Store**.

If “*roll out feature toggles gradually without redeploying code*” → **AppConfig**.

If “*developer must run a command on multiple EC2 instances remotely*” → **SSM Run Command**.

If “*secure EC2 access required without SSH*” → **Session Manager**.

If “*automate EC2 snapshot creation daily*” → **SSM Automation runbook**.

If “*Run Command fails on EC2*” → ensure **SSM Agent is installed** and **IAM role allows SSM actions**.

If “*enforce consistent configuration across EC2 fleet*” → **State Manager**.

If “*SSM commands fail in private subnet*” → add **VPC Interface Endpoints for SSM**.

AWS AppConfig:

AWS AppConfig is a feature of **AWS Systems Manager (SSM)** that lets you **create, manage, and deploy application configurations safely and quickly** — *without redeploying or restarting your application*.

Feature	Description
Config Management	Store config data (key-value pairs, JSON, etc.)
Validation	Automatically validate before deployment (JSON schema, Lambda checks, etc.)
Safe Rollout	Gradual deployment (e.g., 10%, then 50%, then 100%)
Monitoring & Rollback	Integrates with CloudWatch alarms to rollback if errors spike
Versioning	Each config has versions (easy rollback)
Integration	Works with EC2, ECS, Lambda, EKS, or any app via SDK/API
Feature Flags	Enable/disable features dynamically

Component	Description
Application	Logical container for related configs (e.g., “MyWebApp”)
Environment	Represents deployment target (Dev, Test, Prod)
Configuration Profile	Defines where configs come from (S3, SSM Parameter Store, AppConfig hosted)
Validators	Ensure configs are valid before deployment (JSON schema or Lambda)
Deployment Strategy	Defines how fast to roll out (linear, exponential, all-at-once)
Configuration Version	Versioned snapshot of your configuration
Hosted Configuration	Store config data directly in AppConfig (no S3 needed)

- 1 Create an Application** — e.g., “ShoppingApp”
- 2 Create an Environment** — e.g., “Prod” or “Staging”
- 3 Create a Configuration Profile** — where config is stored (S3, SSM, or hosted)
- 4 Add Validators** — to catch bad configs
- 5 Deploy the Configuration** — choose strategy (e.g., 10% per minute)
- 6 Monitor Rollout** — rollback automatically if CloudWatch alarm triggers

Type	Description
JSON Schema Validation	Ensures structure and data types are correct
Lambda Validation	Custom logic (e.g., check for missing required fields)

Strategy	Description
AllAtOnce	Instantly deploy to all targets
Linear	Deploy in equal increments over time (e.g., 20% every 5 minutes)
Exponential	Doubles deployment rate each interval
Custom	Define your own intervals, bake time, or rollback criteria

A Lambda function uses configuration values to connect to APIs.

Step 1: Store API URLs and limits in AppConfig (hosted or Parameter Store).

Step 2: Add JSON schema validator.

Step 3: Deploy config to *Staging*, then *Prod* gradually.

Step 4: Lambda fetches latest config via AppConfig extension.

“A developer wants to change an application’s API endpoint at runtime without redeploying the app.”

✅ **Answer:** Use **AWS AppConfig**.

“Before deploying new configuration, developer must ensure JSON structure is correct.”

✅ **Answer:** Add **JSON schema validator** to AppConfig.

“App configuration should roll out gradually to 10%, then 50%, then 100% of users.”

✅ **Answer:** Use **Linear or Exponential Deployment Strategy**.

“Application should automatically rollback if error rates increase during rollout.”

✅ **Answer:** Use **AppConfig + CloudWatch Alarm**.

“A Lambda function needs to update behavior dynamically without redeploy.”

✅ **Answer:** Use **AppConfig Lambda Extension** to pull latest configs.

AWS Elastic Load Balancer:

Elastic Load Balancing (ELB) automatically distributes **incoming traffic across multiple targets** — such as EC2 instances, containers, IP addresses, or Lambda functions — to achieve **high availability, fault tolerance, and scalability**.

Type	OSI Layer	Ideal For	Example Use
Application Load Balancer (ALB)	Layer 7 (HTTP/HTTPS)	Web apps, microservices, HTTP routing	ECS, EKS, Lambda, API-based apps
Network Load Balancer (NLB)	Layer 4 (TCP/UDP/TLS)	High-performance, low-latency traffic	Gaming, VoIP, IoT
Gateway Load Balancer (GLB)	Layer 3 (Network Gateway)	Deploying third-party virtual appliances	Firewalls, intrusion detection

Feature	Description
Health Checks	Monitors targets; only routes to healthy ones
Cross-Zone Load Balancing	Distributes traffic evenly across AZs
SSL/TLS Termination	Manages HTTPS certificates (via ACM)
Target Groups	Logical grouping of instances, IPs, or Lambdas
Sticky Sessions (Session Affinity)	Route user to same target using cookies
Security Groups	ALB supports; NLB does not (uses NACLs)
Access Logs	Store request logs in S3

Feature	Description
Connection Draining / Deregistration Delay	Allows active requests to complete before removing target

- 1 **Client** connects to ELB DNS name (not static IP).
- 2 ELB chooses a **healthy target** from a **target group**.
- 3 Request routed according to **routing rules** (path, host, etc.).
- 4 ELB continuously performs **health checks** to remove unhealthy targets.


5 Application Load Balancer (ALB) —  *Most Tested*


Layer 7 (HTTP/HTTPS)

Understands **URLs, headers, cookies**, and can make routing decisions based on them.


Features

Feature	Description
Host-Based Routing	Route by domain name (e.g., api.example.com → API service)
Path-Based Routing	Route by URL path (e.g., /images → image service)
Query/Header Routing	Route by request header or query parameter
Redirects & Fixed Responses	Send custom responses directly from ALB
WebSocket & HTTP/2	Supported
Lambda Targets	Invoke Lambda functions directly (serverless routing)
Authentication	Supports OIDC, Cognito user pool auth at the load balancer layer
Container Support	Integrates tightly with ECS using dynamic port mapping

✓ Target Types

- **Instance** → EC2 instances (by instance ID)
 - **IP** → Private IP addresses (ECS tasks in awsvpc mode)
 - **Lambda** → Invokes Lambda functions directly
-

✓ Use Cases

Use Case	Example
Web apps	HTTP/HTTPS frontend
ECS (Fargate or EC2)	Distribute traffic to containers
Lambda	Serverless backend
Microservices	Route requests by path or domain
Cognito Auth	ALB performs user login before request hits backend

✓ Exam Tip:

If *"application uses multiple microservices exposed via unique paths"* → use **ALB with path-based routing**.

⚙️ Sticky Sessions (ALB)

- Uses an **Application Cookie (AWSALB)** or custom cookie.
- Keeps users connected to the same target for session consistency.

✓ Exam Tip:

If *"user's session data stored locally on EC2"* → enable **sticky sessions**.

⚙️ Cross-Zone Load Balancing

- Enabled by default for ALB.
 - Ensures equal traffic distribution across all AZs.
-

6 Network Load Balancer (NLB)

Layer 4 (TCP/UDP/TLS)

Routes traffic purely based on IP protocol data (no inspection of headers or URLs).

Features

Feature	Description
High performance	Millions of requests/sec, ultra-low latency
Static IPs	Each AZ has one static IP
Elastic IP Support	Assign your own EIPs
TLS Offloading	Can handle SSL termination at NLB
Zonal Isolation	One AZ can fail without affecting others
Cross-Zone LB	Optional
Target Types	Instance, IP, or ALB as target

Exam Tip:

If “needs static IP address or extreme performance” → NLB.

Use Cases

Use Case	Example
Real-time apps	Gaming, trading
IoT	MQTT, UDP
Hybrid	On-prem connections via Direct Connect
Load balancing across ALBs	ALB → NLB chaining

7 Gateway Load Balancer (GLB)

Layer 3 (Network Gateway Level)

Designed for deploying, scaling, and managing **virtual network appliances** such as:

- Firewalls
- Intrusion Detection Systems (IDS)
- Deep Packet Inspection (DPI)

Exam Tip:

If *"traffic must pass through third-party virtual appliances transparently"* → **Gateway Load Balancer**.

8 Target Groups

Target Groups = collections of endpoints registered with a load balancer.

Field	Purpose
Target type	instance, IP, Lambda
Health checks	Customizable (path, port, thresholds)
Protocol	HTTP, HTTPS, TCP, UDP
Port	Port to route traffic to
Matcher	Defines success response (e.g., 200–299)

Exam Tip:

If *"ALB health checks failing for ECS service"* → verify **container port mapping** and **health check path**.

9 Health Checks

Property	Description
Interval	How often to check
Timeout	Response wait time
Healthy / Unhealthy Thresholds	# of successes/failures before marking target up/down
Protocol & Path	e.g.,  HTTP:80 /health

Cross Zone Load Balancing:

Load Balancer	Default	Can Disable?
ALB	Enabled	No
NLB	Disabled	Yes (optional)

Load Balancer	Default	Can Disable?
GLB	Enabled	No

- Load balancer handles **SSL decryption** and forwards plain HTTP to targets.
- Certificates managed by **AWS Certificate Manager (ACM)**.
- Simplifies backend and centralizes certificate management.

Feature	ALB	NLB	GLB
Security Groups	✓ Yes	✗ No (use NACLs)	✗ No
ACM Integration	✓ Yes	✓ Yes	✗ No
Private LB Support	✓ Yes	✓ Yes	✓ Yes
Access Logs	✓ Yes (S3)	✓ Yes (S3)	✗ No

Feature	ALB	NLB
Stickiness Support	✓ Yes	✓ Yes (via TCP connection)
Cookie Type	AWSALB	None (TCP persistence)
Use Case	Web apps needing session persistence	Stateful TCP apps

- Load Balancers integrate seamlessly with **EC2 Auto Scaling Groups** and **ECS Services**.
- Targets register/deregister automatically.
- Health checks from ALB/NLB can trigger scaling events.

Scenario 1

"Application requires routing requests to microservices based on URL path."

✓ Answer: Application Load Balancer (path-based routing)

Scenario 2

"Application requires static IPs and extreme throughput."

✓ Answer: Network Load Balancer

Scenario 3

"Developer needs to expose Lambda function via HTTPS endpoint."

✓ Answer: ALB with Lambda target

🗨 Ask ChatGPT

Scenario 4

"Web app needs HTTPS offload and Cognito-based authentication at load balancer."

✓ Answer: ALB with ACM + Cognito auth

Scenario 5

"Containers on ECS need to be registered dynamically with a load balancer."

✓ Answer: ALB target groups + ECS integration

AWS Kinesis:

Amazon Kinesis is a **fully managed service for real-time streaming data** — it lets you collect, process, and analyze large volumes of data as they arrive.

Service	Purpose	Key Exam Use
Kinesis Data Streams (KDS)	Build custom real-time streaming apps	Core DVA focus
Kinesis Data Firehose	Load streaming data into AWS destinations automatically	DVA focus
Kinesis Data Analytics	Run SQL queries or analytics on streaming data	Moderate
Kinesis Video Streams	Stream video data for analytics	Rarely tested

Kinesis Data Streams:

3 Kinesis Data Streams (KDS)

Purpose

Ingest and process real-time data — e.g., user clickstreams, application logs, IoT data.

Architecture

arduino

 Copy code

Producers → Kinesis **Stream** → Shards → **Consumers** (Apps / Lambda)

- **Producer:** sends records (data payloads) into the stream.
- **Stream:** made up of one or more **shards** (units of throughput).
- **Shard:** holds ordered records.
- **Consumer:** application that processes data (Lambda, EC2, or Kinesis Client Library).

Shard Throughput

Operation	Limit
Write	1 MB/sec or 1,000 records/sec per shard
Read	2 MB/sec per shard

-  Scale by **adding shards**.
-  Each shard can be **split or merged** to scale capacity.

Data Retention

- Default: **24 hours**
- Can be increased up to **7 days** (or **365 days** with extended retention)

Exam Tip:

If “*application replays old streaming data*” → increase **data retention period**.

⚙️ Consumers

Type	Description
Classic Consumers	Poll data using Kinesis API (<code>GetRecords</code>)
Enhanced Fan-Out	Push model, each consumer gets its own throughput (2 MB/sec per shard per consumer)
Lambda Integration	Serverless stream processing (event source mapping)

✅ Exam Tip:

If “multiple consumers need same data with low latency” → **Enhanced Fan-Out**.

⚙️ Checkpointing

- Consumers track which records they've already processed.
- Usually handled by **Kinesis Client Library (KCL)** or **Lambda** automatically.

⚙️ Security

- Encrypted at rest via **KMS**.
- Encrypted in transit via **TLS**.
- Fine-grained access via **IAM policies**.

✅ Use Cases

Use Case	Example
Real-time analytics	Web clickstream → process user behavior
IoT data ingestion	Device metrics → Lambda processing
Log aggregation	App logs → S3/Redshift via Firehose
Metrics pipeline	Custom CloudWatch-like system



Kinesis Data Firehose:

✓ Purpose

Fully managed, **no-code streaming ingestion and delivery service**.

👉 Think of it as:

"Send streaming data → Firehose buffers it → automatically delivers to a destination."

✓ Destinations

- Amazon S3
- Amazon Redshift
- Amazon OpenSearch Service
- Splunk / DataDog
- Custom HTTP endpoints

✓ Key Features

Feature	Description
No manual scaling	Auto-scales to match data throughput
Data transformation	Optional — can use Lambda for inline transformations
Buffering	Collects data before delivering (based on size/time)
Compression & Encryption	Before delivery to S3
Delivery retry policy	Retries on failure automatically

✓ Exam Tip:

If "developer wants to stream data to S3 or Redshift with no management" → use **Kinesis Firehose**.

Feature	Kinesis Data Streams	Kinesis Firehose
Processing	Custom (build your own app)	Managed, automatic
Scaling	Manual (add shards)	Auto
Latency	~1 second	~60 seconds
Destinations	Custom consumers (Lambda, EC2)	S3, Redshift, OpenSearch
Transformation	App logic	Lambda optional
Replay capability	Yes (data retained)	No (real-time only)

Kinesis Data Analytics:

Run **SQL queries or analytics** on streaming data in real time (from KDS or Firehose).

- Filter, transform, and aggregate data without building custom code.

Kinesis Video Streams:

- Ingest, store, and process **live video streams**.
- Commonly used for **machine learning**, **security cameras**, or **vision analytics**.

Scenario	Correct Answer
Real-time log processing with custom business logic	Kinesis Data Streams + Lambda
Deliver log data to S3 automatically	Kinesis Firehose
Query streaming IoT data with SQL	Kinesis Data Analytics
High-volume, low-latency ingestion	Kinesis Data Streams (shard scaling)
Replay past 3 hours of data	Increase retention in Kinesis Data Streams
Buffer and batch transform before storing	Firehose with Lambda transform
Handle multiple consumers needing same data	Enhanced Fan-Out
Auto-scale ingestion without shard management	Firehose

AWS IAM:

AWS Identity and Access Management (IAM) lets you **securely control access to AWS services and resources**.

Component	Description	Example
Users	Individual identities (person or app)	alice, developer_bot
Groups	Collection of users with same permissions	dev-team, admin-team
Roles	Temporary permissions that AWS resources assume	EC2 role, Lambda role
Policies	JSON documents defining permissions	<code>"Effect": "Allow", "Action": "s3:GetObject"</code>

IAM Policies:

Field	Meaning
Effect	Allow or Deny

Field	Meaning
Action	API operation(s) (e.g., <code>s3:GetObject</code> , <code>dynamodb:PutItem</code>)
Resource	Target ARN(s)
Condition	Optional rules (e.g., by IP, time, tag)

Type	Description	Attached To
AWS Managed	Predefined by AWS	Users, groups, roles
Customer Managed	Created by you	Users, groups, roles
Inline	Embedded in a single identity	Specific user or role only
Resource-based	Attached directly to resource (e.g., S3 bucket policy)	Resource

Roles = temporary credentials (no passwords or keys).

They define “**what actions a service or entity can perform on your behalf.**”

Role Type	Purpose	Example
Service Role	AWS service acts on your behalf	Lambda → S3, ECS → CloudWatch
Assumed Role (STS)	Federated or cross-account access	User in Account A → Role in Account B
Instance Profile	Attach role to EC2 instance	EC2 access to S3
Lambda Execution Role	Grants Lambda function AWS API access	Lambda → DynamoDB

Rule	Meaning
Default Deny	Access denied unless explicitly allowed
Explicit Allow	Grants permission
Explicit Deny	Always overrides any allow

Feature	IAM Policy	Resource-Based Policy
Attached To	IAM user/role/group	AWS resource (e.g., S3, Lambda, SNS)
Defines	What the principal can do	Who can access the resource

Feature	IAM Policy	Resource-Based Policy
Cross-Account Access	No (alone)	Yes (easily)
Example	"User can read S3 objects"	"Anyone from Account B can read this bucket"

A **trust policy** defines *who can assume a role*.

IAM Access Analyzer - Analyzes policies to identify **overly permissive access** or **cross-account exposure**.

IAM Identity Center:

- Centralized SSO access across AWS accounts.
- Supports Active Directory, Okta, etc.

IAM Best Practices:

Category	Best Practice
Credentials	Never embed access keys in code; use roles
Principle of Least Privilege	Only grant necessary permissions
MFA	Require Multi-Factor Authentication
Groups	Assign permissions to groups, not users
Rotation	Rotate access keys regularly
Monitoring	Use CloudTrail to audit IAM actions
Cross-Account Access	Use roles, not shared keys

When you create a Lambda function:

- It assumes an **IAM execution role**.
- That role defines what AWS resources the function can access (e.g., S3, DynamoDB).
- ECS tasks (Fargate or EC2) assume **task roles**.
- EKS pods use **IAM Roles for Service Accounts (IRSA)** to get credentials securely.

Key Example	Meaning
<code>aws:SourceIp</code>	Restrict by IP or CIDR
<code>aws:RequestedRegion</code>	Restrict by AWS Region
<code>aws:CurrentTime</code>	Restrict by time

Key Example	Meaning
<code>aws:MultiFactorAuthPresent</code>	Require MFA
<code>s3:prefix</code>	Restrict S3 prefixes

Credential Type	Description
Console Password	For AWS Management Console
Access Keys (AKIA...)	For CLI/SDK access
Temporary Keys	From IAM Role or STS
MFA Token	For extra security

Scenario	Correct Answer
Lambda needs S3 access	Create IAM Role for Lambda
Cross-account EC2 access	Use Role + Trust Policy
Fine-grained access by IP	Policy Condition <code>aws:SourceIp</code>
Temporary credentials	Use AWS STS
AWS service acts on your behalf	Service Role
Federated login	Use IAM Identity Center or STS AssumeRoleWithSAML
S3 cross-account sharing	Resource-based bucket policy
Security audit	IAM Access Analyzer
Find “who did what”	CloudTrail

If “*application or service needs AWS access without static credentials*” → use **IAM Roles**.

If “*access should be restricted by IP or time*” → use **policy Condition keys** (like `aws:SourceIp`).

If “*permission must be shared across accounts*” → use **resource-based policy** (e.g., S3 bucket policy or Lambda permission).

If “*application running on EC2 or Lambda needs access to AWS resources*” → **assign IAM role**, not keys.

If “*another AWS account needs access to your resource*” → attach a **resource-based policy**.

If “*Lambda cannot assume role*” → check **trust policy** (must include service principal).

If “*short-term cross-account access required*” → use **STS AssumeRole**.

If “*centralized login and user management for multiple AWS accounts*” → **IAM Identity Center** (AWS SSO).

If “*organization needs to find resources accessible from outside account*” → use **IAM Access Analyzer**.

If “*Lambda fails to write to DynamoDB*” → check **execution role policy**.

If “*CodePipeline cannot deploy to ECS*” → missing **IAM permissions** on the service role.

If “*access must be limited to specific IP range*” → include **Condition** → **aws:SourceIp**.

If “*developer’s access keys leaked*” → **deactivate and rotate immediately**.

AWS EC2:

Amazon Elastic Compute Cloud (EC2) provides **resizable, on-demand compute capacity** in the AWS cloud.

It’s essentially **virtual servers** you can start, stop, and scale when needed.

Concept	Description
Instance	A virtual server (VM) running an OS (Amazon Linux, Ubuntu, Windows, etc.)
AMI (Amazon Machine Image)	Template that defines OS, software, and configuration
Instance Type	Defines CPU, memory, storage, and network capacity (e.g., t3.micro, m5.large)
Region & AZ	Determines physical location and redundancy
Key Pair	Used for SSH or RDP access
Security Group	Virtual firewall controlling inbound/outbound traffic
Elastic IP	Static public IP you can reassign between instances

State	Meaning
Pending	Starting up
Running	Active and billable
Stopping / Stopped	Persistent (EBS volumes remain, instance not billed)
Terminated	Permanently deleted

Model	Description	Use Case
On-Demand	Pay per second; no commitment	Dev/test, unpredictable usage
Reserved Instance	1- or 3-year term; 72% cheaper	Steady, predictable workloads
Savings Plans	Flexible pricing model replacing some reserved needs	Continuous usage but flexible instance types
Spot Instance	Unused capacity up to 90% off; interruptible	Fault-tolerant, batch jobs

Model	Description	Use Case
Dedicated Host/Instance	Physical isolation	Compliance or licensing requirements

Type	Description	Persistent?	Example Use
EBS (Elastic Block Store)	Block storage attached to instance	✅ Yes	OS disk, databases
Instance Store (Ephemeral)	Local physical disk attached to host	❌ No (deleted when stopped)	Cache, temporary files
EFS (Elastic File System)	Shared file storage across instances	✅ Yes	Multi-instance workloads
S3	Object storage (not directly mounted)	✅ Yes	Backups, static assets

Feature	Description
Security Group	Stateful firewall; allows/blocks inbound/outbound
NACL (Network ACL)	Stateless firewall at subnet level
Elastic IP	Static public IP
Placement Group	Controls how instances are placed across hardware (cluster/spread/partition)

Feature	Description	Access
Instance Metadata	Info about instance (ID, IP, role creds)	<code>http://169.254.169.254/latest/meta-data/</code>
User Data	Script that runs at launch	<code>#!/bin/bash</code> install commands

Feature	Description
Launch Template	Defines AMI, type, role, security group
Scaling Policy	Rule for when to scale up/down (e.g., CPU > 70%)
Health Checks	Replace unhealthy instances automatically

Control	Description
Security Groups	Allow traffic; stateful
NACLs	Deny/allow rules; stateless

Control	Description
SSH Key Pairs	Authentication
IAM Role	Temporary access to AWS APIs
Encryption	EBS & snapshots via KMS

Feature	Description
EBS Snapshot	Backup of volume (stored in S3, incremental)
AMI	Image built from snapshot + metadata
Copy / Share	Can share AMI across regions or accounts

Type	Description	Use Case
Cluster	Instances placed close together on same rack	Low latency, high throughput (HPC, analytics)
Spread	Instances placed on separate racks	High availability (avoid simultaneous failure)
Partition	Instances divided into logical partitions, each isolated	Big data clusters (HDFS, Cassandra, Kafka)

Action	RAM State	EBS State	Use Case
Stop	Lost	Retained	Regular stop/start
Hibernate	Saved to EBS	Retained	Resume same session

Type	Description	Flexibility
Compute Savings Plan	Applies to any compute (EC2, Fargate, Lambda) regardless of region, family, or OS	✅ Most flexible
EC2 Instance Savings Plan	Applies to a specific instance family in a specific region (e.g., m5 in us-east-1)	⚙️ Less flexible, cheaper

Scenario	Correct Answer
Run script at boot time	User Data
EC2 app must access S3 securely	IAM Role for EC2
EC2 behind LB failing health checks	Check SG + health path
EC2 cannot connect to DB in private subnet	NACL or SG restriction
Move EC2 setup to another region	Create and copy AMI

Scenario	Correct Answer
Need low latency HPC cluster	Placement Group (Cluster)
Store temporary files only	Instance Store
Persist data after stop	EBS Volume
No SSH access	Missing SG rule or wrong key pair
App needs auto scaling	ASG + ELB integration

If “*application must retain public IP after instance reboot*” → use an **Elastic IP**, not auto-assigned public IP.

When *terminated*, **EBS root volume may be deleted** unless “Delete on Termination” is unchecked.

If “*developer wants lowest cost for fault-tolerant batch processing*” → **Spot Instances**.

If “*data must persist after instance stop/restart*” → use **EBS** or **EFS**, not instance store.

If “*app instances need low-latency, high throughput between each other*” → use **Placement Group (Cluster)**.

If “*application on EC2 needs to access S3 or DynamoDB without storing keys*” → assign **IAM Role**.

If “*run script automatically when instance starts*” → use **User Data**.

If “*retrieve temporary credentials*” → use **Instance Metadata**.

If “*app must automatically handle variable traffic*” → use **Auto Scaling Group + Load Balancer**.

If “*EC2 behind load balancer stops responding*” → **ELB routes traffic only to healthy instances**.

If “*EC2 instance unreachable but OS logs look fine*” → check **System Status Check** (hardware failure).

If “*developer cannot SSH into EC2*” → check **inbound rule** in **Security Group** and **key pair**.

If “*developer wants to duplicate an EC2 setup in another region*” → **create AMI** → **copy AMI to new region** → **launch instance**.

If “*need low latency between nodes*” → **Cluster**.

If “*fault isolation across hardware*” → **Spread** or **Partition**.

If “*need instance to boot faster with in-memory state*” → **Hibernate**.

If “*need to deploy code updates to EC2 automatically*” → use **CodeDeploy with EC2 AppSpec file**.

If “*instances need ultra-low latency networking*” → **Cluster**.

If “*instances must not share hardware*” → **Spread**.

If “*large distributed system across fault domains*” → **Partition**.

If “*steady compute workload with flexibility to change instance types or regions*” → **Compute Savings Plan**.

If “*predictable workload on specific instance type*” → **EC2 Instance Savings Plan**.

AWS VPC:

A **VPC (Virtual Private Cloud)** is a **logically isolated network** in AWS where you can launch and control your resources — similar to having your own private data center inside AWS.

Component	Description	Example
VPC CIDR Block	The IP address range for your VPC	10.0.0.0/16
Subnet	Divides VPC into smaller networks	10.0.1.0/24
Route Table	Defines traffic flow (where packets go)	Send 0.0.0.0/0 to Internet Gateway
Internet Gateway (IGW)	Enables internet access for public subnets	Web servers
NAT Gateway	Allows private subnets to access internet outbound only	App servers download packages
Security Group (SG)	Instance-level firewall (stateful)	Allows port 443
Network ACL (NACL)	Subnet-level firewall (stateless)	Deny inbound port 22
VPC Endpoint	Connect privately to AWS services without internet	S3, DynamoDB
Peering / Transit Gateway	Connect multiple VPCs	Multi-VPC apps

Type	Description	Example
Public Subnet	Has route to Internet Gateway	Web servers, ALBs
Private Subnet	Has route only to NAT Gateway	App servers, DBs

Route Tables:

- Define **where traffic goes** from each subnet.
- Each subnet is associated with one **Route Table**.

Internet Gateway:

- Horizontally scaled, redundant gateway that allows **public traffic** to enter/exit your VPC.
- Must be **attached to the VPC** and referenced in the route table.

Security Groups:

- **Stateful** firewalls (remember connections).
- Attached to **ENIs** (Elastic Network Interfaces).
- Only **allow rules** — no denies.

NACLs:

- **Stateless** firewalls applied at **subnet level**.
- Have explicit **Allow** and **Deny** rules.
- Evaluated in **order (lowest rule number first)**.

VPC Endpoints:

Enable private access to AWS services **without internet, NAT, or IGW**.

Type	Description	Example
Interface Endpoint (PrivateLink)	Creates ENI with private IP inside VPC	SSM, CloudWatch, API Gateway
Gateway Endpoint	Adds route entry in route table	S3, DynamoDB

VPC Peering:

Connects two VPCs privately using internal IPs (same or different accounts/regions).

- **One-to-one** relationship (no transitive routing).
- Must update route tables on both sides.

Transit Gateway - Central hub to connect multiple VPCs and on-premises networks.

VPC Flow Logs

Capture **IP traffic metadata** (not packet content) for debugging or auditing.

- Can send to **CloudWatch Logs** or **S3**.
- Captures accepted/rejected traffic per ENI, subnet, or VPC.

Feature	Description
PrivateLink	Underlying tech that lets you privately connect to AWS or 3rd-party services via Interface Endpoints
VPC Endpoint	Implementation of PrivateLink for your VPC (S3, DynamoDB, etc.)

Feature	Default VPC	Custom VPC
Created automatically	✓	✗
Internet access	✓	Must configure
Public subnet	✓	Must define manually
Route tables / SGs	Auto-configured	Custom setup

IPv6 Support:

- Dual-stack (IPv4 + IPv6) supported.
- **No NAT Gateway for IPv6** — public by default.
- Uses **Egress-Only Internet Gateway** for outbound-only IPv6 traffic.

Scenario	Correct Answer
EC2 can't access internet	Missing IGW or route
Private instance needs internet	NAT Gateway
Lambda needs to reach RDS privately	Run in same VPC/subnet
Lambda needs S3 access but no NAT	Gateway Endpoint for S3
VPC A ↔ VPC B can't communicate	Update route tables after Peering
Connect multiple VPCs	Transit Gateway
Monitor dropped traffic	VPC Flow Logs
Restrict access by subnet	NACL rules
Securely connect to AWS API privately	Interface Endpoint (PrivateLink)

If “*Lambda or EC2 must access the internet but is in a private subnet*” → attach a **NAT Gateway** in a **public subnet**.

If “*EC2 has no internet access even with Elastic IP*” → check that route table includes 0.0.0.0/0 → Internet Gateway .

If “*cannot connect to EC2 public IP*” → either IGW not attached or route missing.

If “*private instance needs to download updates from internet*” → **NAT Gateway in public subnet**.

If “*EC2 can ping out but not receive responses*” → security group inbound rules missing.

If “*connection blocked despite SG allowing it*” → check **NACL deny rule**.

If “*Lambda in private subnet must write to S3 but no internet access*” → **use VPC Gateway Endpoint (S3)**.

If “*VPC A can't reach VPC B after peering*” → missing **route entries**.

If *“need to connect 5+ VPCs efficiently”* → **Transit Gateway** (instead of multiple peerings).

If *“instance must have multiple IPs or SGs”* → add **additional ENI**.

If *“need to monitor traffic rejected by NACL”* → **enable VPC Flow Logs**.

If *“need fixed public IP for EC2 reboot or redeploy”* → **Elastic IP**.

If *“connect privately to a partner SaaS without public internet”* → **PrivateLink (Interface Endpoint)**.

If *“instances can’t resolve internal names”* → **enable DNS hostnames and DNS resolution** in VPC settings.

If *“EC2 launched in default VPC has internet access automatically”* → because **IGW, routes, and SGs** are pre-set.

If *“private instances need outbound IPv6 only”* → use **Egress-Only Internet Gateway**.

If *“Lambda in VPC can’t reach internet”* → attach **NAT Gateway**.